Kubernetes: A Complete CKA Technical Reference

Covering CKA 2026, Container Interaction, Kustomize, and Cluster Operations

Author: Compufreq

> ***Author's Note:*** *This guide is a comprehensive technical reference for engineers, DevOps practitioners, SREs, and CKA candidates. Every command includes context, purpose, flags, common pitfalls, and real-world examples.*
> *Chapters are arranged in a logical learning progression from cluster architecture through container operations, debugging, advanced configuration, and certification.*

Table of Contents

PART 1 — CLUSTER ARCHITECTURE & SETUP

Chapter 1 — Kubernetes Architecture: A Complete Overview 3
Chapter 2 — Cluster Installation from Scratch with kubeadm 22
Chapter 3 — Highly-Available Control Plane Architecture 31
Chapter 4 — Extension Interfaces: CNI, CSI, and CRI 38

PART 2 — WORKLOADS & CONFIGURATION

Chapter 5 — Understanding Pods and Containers 45
Chapter 6 — ConfigMaps and Secrets: Complete Reference 53
Chapter 7 — StorageClasses and Dynamic Volume Provisioning 64
Chapter 8 — Workload Autoscaling with HPA and VPA 72
Chapter 9 — Self-Healing Primitives: Probes and PodDisruptionBudgets 80
Chapter 10 — Custom Resource Definitions and Operators 87
Chapter 11 — Gateway API: Modern Ingress Traffic Management 92

PART 3 — NETWORKING & SECURITY

Chapter 12 — Kubernetes Services: ClusterIP, NodePort, LoadBalancer, and Beyond 97
Chapter 13 — Ingress: HTTP Routing and TLS Termination 104
Chapter 14 — NetworkPolicy: Traffic Control and Microsegmentation 112
Chapter 15 — RBAC: Role-Based Access Control 122
Chapter 16 — Kubernetes Security: Authentication, Pod Security, and TLS 130

PART 4 — CONTAINER INTERACTION

Chapter 17 — Executing Commands Inside Containers 136
Chapter 18 — Viewing and Streaming Container Logs 142
Chapter 19 — Copying Files To and From Containers 146
Chapter 20 — Inspecting and Describing Containers 149
Chapter 21 — Port Forwarding to Containers 153
Chapter 22 — Resource Monitoring Inside Containers 156
Chapter 23 — Removing and Modifying Containers in Pods 160

PART 5 — DEBUGGING & OBSERVABILITY

Chapter 24 — On-the-Spot Test Container Creation and Deletion 168
Chapter 25 — Network Testing from Ephemeral Containers 174
Chapter 26 — CoreDNS: Testing, Diagnosing, and Verifying DNS Resolution 178
Chapter 27 — Advanced Debugging with Ephemeral Containers 188
Chapter 28 — Namespace-Aware Commands and Context Management 192

PART 6 — ADVANCED CONFIGURATION & CKA

Chapter 29 — kubectl Kustomize: Configuration Management at Scale 196
Chapter 30 — CKA Certification 2026: Complete Study and Command Reference 210

APPENDICES

Appendix A — Quick Reference Cheat Sheet 234
Appendix B — CKA 2026 Command Speed Reference 237
Appendix C — Kustomize Pattern Reference 240
Appendix D — CKA Gap-Fill Quick Reference 243

Chapter 1 — Kubernetes Architecture: A Complete Overview

1.1 What Is Kubernetes?

Kubernetes (often abbreviated K8s — there are 8 letters between K and s) is an open-source container orchestration system originally designed by Google and donated to the Cloud Native Computing Foundation (CNCF) in 2014. Today it is the industry standard for deploying, scaling, and managing containerized workloads.

At its core, Kubernetes solves three fundamental problems that arise when running containers at scale:

Placement — Where should a container run? Kubernetes has a sophisticated scheduler that considers CPU, memory, node affinity, taints, tolerations, and custom policies to decide which node should run which container.

Lifecycle management — What happens if a container crashes? Kubernetes continuously compares
the *desired state* you declare with the *actual state* of the cluster. If a container dies, it restarts it. If a node goes offline, it reschedules the Pods that were on it.

Networking and discovery — How do containers find each other? Kubernetes provides stable DNS names and virtual IPs for groups of containers, abstracting away the ephemeral nature of individual container IP addresses.

1.2 The Cluster Model

A Kubernetes cluster consists of one or more **nodes** — physical machines or virtual machines. Every cluster has two logical layers:

```
┌──────────────────────────────────────────────────────────┐
│                     CONTROL PLANE                        │
│  ┌──────────────────┐  ┌──────────┐  ┌──────────────────┐  │
│  │  API Server      │  │   etcd   │  │  Scheduler       │  │
│  │ (kube-apiserver) │  │(key-value│  │(kube-scheduler)  │  │
│  │                  │  │store)    │  │                  │  │
│  └──────────────────┘  └──────────┘  └──────────────────┘  │
│  ┌──────────────────────────┐  ┌──────────────────────┐  │
│  │  Controller Manager      │  │  Cloud Controller    │  │
│  │  (kube-controller-manager) │  │  Manager (optional) │  │
│  └──────────────────────────┘  └──────────────────────┘  │
└──────────────────────────────────────────────────────────┘
              │              │              │
┌──────────────────────────────────────────────────────────┐
│            WORKER NODES (1 to thousands)                 │
│  ┌──────────────────────────────────────────────────┐  │
│  │  kubelet   │  kube-proxy  │  Container Runtime (CRI) │  │
│  │            │              │  (containerd / CRI-O)    │  │
│  │  ┌──────────────────────────────────────────┐  │  │
│  │  │    Pod    │    Pod    │    Pod                │  │  │
│  │  │ Container │ Container │ Container + Container │  │  │
│  │  └──────────────────────────────────────────┘  │  │
│  └──────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────┘
```

1.3 Control Plane Components In Depth

kube-apiserver

The **API server** is the central hub of Kubernetes. Every action in the cluster flows through it — `kubectl` commands, controller watches, kubelet heartbeats, and scheduler decisions all talk to the API server.

The API server:

- Validates and persists resource definitions into etcd
- Exposes the Kubernetes REST API on port `6443` (HTTPS)
- Authenticates requests (certificates, service account tokens, OIDC, webhook)
- Authorizes requests via RBAC, ABAC, or webhook
- Runs admission controllers that can mutate or validate resources before persistence

How kubectl works: When you run `kubectl apply -f deployment.yaml`, `kubectl` reads your `~/.kube/config`, authenticates to the API server using the embedded certificate, serializes your YAML into JSON, and sends a `PUT` or `POST` to the appropriate REST endpoint (e.g., `/apis/apps/v1/namespaces/default/deployments`).

```
# Watch the actual HTTP calls kubectl makes
kubectl apply -f deployment.yaml -v=8

# Directly query the API server REST API
kubectl get --raw /api/v1/namespaces/default/pods | python3 -m json.tool
```

etcd

etcd is a distributed key-value store that is the only persistent storage in Kubernetes. Every resource object (Pod, Deployment, ConfigMap, Service, Secret, etc.) is serialized to Protocol Buffers and stored in etcd.

Key properties:

- Uses the **Raft consensus algorithm** for distributed consistency
- Stores data at paths like `/registry/pods/default/mypod`
- Should run with **3 or 5 members** in production for quorum fault tolerance
- Requires fast disks — etcd latency directly impacts cluster responsiveness

```
# Inspect etcd data directly (on a control-plane node)
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  get /registry/pods/default/ --prefix --keys-only

# Check etcd cluster health
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  endpoint health --cluster
```

kube-scheduler

The **scheduler** watches for newly created Pods that have no `nodeName` assigned and selects the best node for them to run on. The selection process has two phases:

Filtering — Remove nodes that cannot satisfy the Pod's requirements:

- Not enough CPU or memory
- Node has a taint the Pod doesn't tolerate
- NodeAffinity rules exclude the node
- Pod's requested hostPort is already in use

Scoring — Rank remaining nodes by multiple criteria (spread, resource balance, affinity preferences, etc.) and choose the highest-scoring node.

The scheduler then writes the chosen `nodeName` back to the Pod spec via the API server, and the kubelet on that node picks it up.

```
# See which node the scheduler chose and why
kubectl get pod mypod -o wide
kubectl describe pod mypod | grep -A 5 "Events"

# Override scheduler for a specific Pod (manual scheduling)
kubectl patch pod mypod -p '{"spec":{"nodeName":"worker-2"}}'
```

kube-controller-manager

The **controller manager** is a single binary that runs many independent control loops ("controllers"),
each responsible for one resource type:

Controller	Responsibility
Deployment controller	Ensures the right number of ReplicaSet revisions exist
ReplicaSet controller	Ensures the right number of Pods are running
Node controller	Detects node failures; evicts Pods after timeout
Job controller	Tracks Job Pod completions; creates new Pods if needed
Endpoints controller	Populates Endpoint objects from Service + Pod selectors
ServiceAccount controller	Creates default ServiceAccounts in new namespaces
Namespace controller	Handles namespace deletion and cleanup
CronJob controller	Fires Jobs on cron schedule

Each controller runs a tight reconciliation loop:

1. Watch the API server for changes to its resource type
2. Compare actual state vs desired state
3. Take actions to converge (create/delete/update resources)

```
# See the controllers running
kubectl -n kube-system get pod | grep kube-controller-manager

# Look at controller-manager logs for a specific reconciliation
kubectl -n kube-system logs kube-controller-manager-controlplane \
  --since=5m | grep -i "deployment controller"
```

1.4 Worker Node Components In Depth

kubelet

The **kubelet** is the primary node agent. It runs on every node (including the control-plane node in single-node clusters) and is responsible for:

- Watching the API server for Pods assigned to its node
- Calling the container runtime (via CRI) to create/delete containers
- Mounting volumes (ConfigMaps, Secrets, PVCs, emptyDir)
- Running liveness/readiness/startup probes
- Reporting node and Pod status back to the API server
- Managing static Pods from `/etc/kubernetes/manifests/`

The kubelet is the only Kubernetes component that does NOT run inside Kubernetes — it runs directly as a systemd service on the host.

```
# kubelet status and logs on a node
sudo systemctl status kubelet
sudo journalctl -u kubelet -f
sudo journalctl -u kubelet --since="5 minutes ago" | grep -i error

# kubelet configuration
cat /var/lib/kubelet/config.yaml
cat /etc/kubernetes/kubelet.conf    # kubeconfig for kubelet

# Which Pods this node's kubelet manages
ls /var/lib/kubelet/pods/
```

kube-proxy

kube-proxy runs on every node and implements the Service abstraction by maintaining network rules (typically iptables or IPVS rules) that route traffic destined for a Service ClusterIP to one of the backing Pod IPs.

When a Service is created:

1. The Endpoints controller populates an Endpoints object with the IPs of matching Pods
2. kube-proxy watches the Endpoints object
3. kube-proxy programs iptables/IPVS rules to DNAT traffic from ClusterIP:port to PodIP:port

```
# See kube-proxy mode and status
kubectl -n kube-system get pod | grep kube-proxy
kubectl -n kube-system logs kube-proxy-xxxxx

# View the iptables rules kube-proxy creates
sudo iptables -t nat -L KUBE-SERVICES -n | grep -i my-service

# kube-proxy configmap
kubectl -n kube-system get configmap kube-proxy -o yaml
```

Container Runtime (CRI)

The **container runtime** is responsible for actually pulling images and running containers. Modern Kubernetes uses the **Container Runtime Interface (CRI)** to decouple from any specific runtime. Common runtimes:

Runtime	Notes
containerd	Default in most Kubernetes distributions; minimal, fast
CRI-O	Purpose-built CRI runtime for Kubernetes; used by OpenShift
Docker Engine	Supported via `cri-dockerd` shim (Dockershim removed in K8s 1.24)

```
# Interact with containerd directly (replaces docker CLI)
sudo crictl ps                        # List running containers
sudo crictl images                    # List pulled images
sudo crictl logs <container-id>       # Container logs
sudo crictl inspect <container-id>    # Container details
sudo crictl pods                      # List pod sandboxes

# containerd directly
sudo ctr images list                  # All images in containerd
sudo ctr containers list              # All containerd containers
```

1.5 The Kubernetes Object Model

Everything in Kubernetes is a **resource object** with the same four top-level fields:

```
apiVersion: apps/v1          # API group/version
kind: Deployment             # Resource type
metadata:                    # Identity and labels
  name: my-app
  namespace: production
  labels:
    app: my-app
    version: "1.0"
  annotations:
    description: "Main application deployment"
spec:                        # Desired state (you write this)
  replicas: 3
  ...
# status:                    # Actual state (Kubernetes writes this, read-only for users)
#   availableReplicas: 3
#   conditions: [...]
```

The **spec/status split** is fundamental: you declare intent in `spec`, Kubernetes reconciles toward it and reports reality in `status`.

API Groups and Versions

Resources are organized into API groups. The path structure is:

```
/api/v1                                  # Core group (Pods, Services, ConfigMaps, Namespaces)
/apis/<group>/<version>                  # Named groups
/apis/apps/v1                            # Deployments, StatefulSets, DaemonSets, ReplicaSets
/apis/batch/v1                           # Jobs, CronJobs
/apis/networking.k8s.io/v1              # Ingress, NetworkPolicy
/apis/storage.k8s.io/v1                 # StorageClass, PersistentVolume
/apis/rbac.authorization.k8s.io/v1      # Role, ClusterRole, RoleBinding
/apis/gateway.networking.k8s.io/v1      # Gateway, HTTPRoute (Gateway API)
/apis/apiextensions.k8s.io/v1           # CustomResourceDefinition
```

```
# Discover all resource types in the cluster
kubectl api-resources
kubectl api-resources --namespaced=true   # Only namespace-scoped resources
kubectl api-resources --namespaced=false  # Only cluster-scoped resources

# Discover all API versions
kubectl api-versions

# Get the schema for any resource
kubectl explain pod
kubectl explain pod.spec.containers
kubectl explain deployment.spec.template.spec.containers.resources
```

1.6 Namespaces

Namespaces provide a logical partition within a cluster. Resources in different namespaces are isolated from each other by name (you can have two Deployments both named `web` in different namespaces), and RBAC and ResourceQuotas can be scoped per-namespace.

```
# Built-in namespaces
kubectl get namespaces
# NAME              STATUS   AGE
# default           Active   30d   ← Where your resources go if you don't specify
# kube-system       Active   30d   ← Kubernetes system components
# kube-public       Active   30d   ← Publicly readable ConfigMap (cluster-info)
# kube-node-lease   Active   30d   ← Node heartbeat Lease objects

# Create a namespace
kubectl create namespace staging

# Work in a namespace
kubectl get pods -n staging
kubectl apply -f app.yaml -n staging

# Set your current context to a namespace permanently
kubectl config set-context --current --namespace=staging

# List resources across ALL namespaces
kubectl get pods --all-namespaces
kubectl get pods -A               # shorthand
```

What is NOT namespaced (cluster-scoped resources):

- Nodes
- PersistentVolumes
- StorageClasses
- ClusterRoles, ClusterRoleBindings
- Namespaces themselves
- CustomResourceDefinitions

1.7 Labels, Selectors, and Annotations

Labels are key-value pairs attached to objects. They are the glue that connects Kubernetes objects together — Services find their Pods via labels, Deployments manage their ReplicaSets via labels, and NetworkPolicies select targets via labels.

```
metadata:
  labels:
    app: frontend          # Common labels by convention
    version: "2.3.1"
    environment: production
    tier: web
```

Selectors find objects matching a set of labels:

```
# Equality-based selector
kubectl get pods -l app=frontend
kubectl get pods -l app=frontend,environment=production

# Set-based selector
kubectl get pods -l 'environment in (production, staging)'
kubectl get pods -l 'tier notin (backend)'
kubectl get pods -l '!beta'                # Pods that don't have the 'beta' label

# Show labels on each pod
kubectl get pods --show-labels
```

Annotations are also key-value pairs, but they are for machine-readable metadata — not used for selection. Use annotations for build info, documentation links, tool configuration, etc.

```
metadata:
  annotations:
    kubernetes.io/change-cause: "Upgraded to nginx 1.25.3 for CVE-2023-xxxx"
    prometheus.io/scrape: "true"
    prometheus.io/port: "9090"
    deployment.kubernetes.io/revision: "3"
```

1.8 The Reconciliation Loop: Desired vs Actual State

The most important concept in Kubernetes is the **reconciliation loop**. Kubernetes is a **declarative** system — you tell it *what* you want, not *how* to do it.

```
You declare:                              Kubernetes ensures:
┌──────────────────────┐                 ┌──────────────────────┐
│ spec.replicas: 3     │  ──────────────▶ │ 3 Pods running       │
└──────────────────────┘                 └──────────────────────┘
       ▲                                          │
       │           If Pod crashes:                │
       │           Controller detects deviation   │
       └──────────────────────────────────────────┘
                   Creates a new Pod
```

This is fundamentally different from imperative systems (like running `docker run` three times). If your desired state says `replicas: 3` and one Pod crashes, Kubernetes automatically creates a new one. If you manually delete a Pod, Kubernetes creates a new one.

```
# Watch the reconciliation in action
kubectl run testpod --image=nginx
kubectl delete pod testpod &    # Delete immediately
kubectl get pods -w             # Watch Kubernetes recreate it (if managed by a controller)

# The "why" of any resource is in its events
kubectl describe deployment my-app | tail -30

# Force a reconciliation by triggering a rollout
kubectl rollout restart deployment/my-app
```

1.9 Core Resource Relationships

Understanding how Kubernetes objects relate to each other is critical:

```
Deployment
    └ manages → ReplicaSet
                └ manages → Pod(s)
                            └ contains → Container(s)
                            └ mounts → Volume(s)
                                       └ backed by → PVC → PV

Service ──→ selects via labels ──→ Pod(s)
    └── exposes ──→ Endpoints object

ConfigMap ──→ mounted by ──→ Pod(s)
Secret ──→ mounted by ──→ Pod(s)

ServiceAccount ──→ used by ──→ Pod(s)
    └── bound via ──→ RoleBinding ──→ Role (RBAC permissions)

NetworkPolicy ──→ selects via labels ──→ Pod(s) (restricts ingress/egress)
```

1.10 kubectl: The Primary CLI

`kubectl` is your interface to everything in Kubernetes. It reads cluster credentials from `~/.kube/config` (the kubeconfig file).

Essential kubectl Patterns

```
# CRUD operations
kubectl get <resource> [name] [-n namespace] [-o yaml|json|wide]
kubectl describe <resource> <name>
kubectl apply -f <file-or-dir>
kubectl delete <resource> <name>

# Imperative creation (exam speed)
kubectl create deployment nginx --image=nginx --replicas=3
kubectl expose deployment nginx --port=80 --target-port=80 --type=ClusterIP
kubectl run test --image=busybox --restart=Never -- sleep 3600
kubectl create configmap myconfig --from-literal=key=value
kubectl create secret generic mysecret --from-literal=password=s3cret

# Edit in-place
kubectl edit deployment nginx      # Opens $EDITOR (vim by default on exam)
kubectl patch deployment nginx -p '{"spec":{"replicas":5}}'

# Imperative resource generation (--dry-run for templates)
kubectl create deployment nginx --image=nginx --dry-run=client -o yaml > deploy.yaml
kubectl run test --image=busybox --dry-run=client -o yaml > pod.yaml

# Watch and wait
kubectl get pods -w                  # Watch for changes
kubectl wait --for=condition=ready pod -l app=nginx --timeout=60s

# Rollout management
kubectl rollout status deployment/nginx
kubectl rollout history deployment/nginx
kubectl rollout undo deployment/nginx
kubectl rollout undo deployment/nginx --to-revision=2

# Contexts and clusters
kubectl config get-contexts
kubectl config use-context production
kubectl config set-context --current --namespace=staging
```

Output Formats

```
# JSON path for specific fields
kubectl get pod mypod -o jsonpath='{.status.podIP}'
kubectl get nodes -o jsonpath='{.items[*].metadata.name}'
kubectl get pods -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.podIP}{"\n"}{end}'

# Custom columns
kubectl get pods -o custom-columns='NAME:.metadata.name,IP:.status.podIP,NODE:.spec.nodeName'

# Sort output
kubectl get pods --sort-by=.metadata.creationTimestamp
kubectl get pods --sort-by=.status.podIP

# YAML/JSON output for all resources in a namespace
kubectl get all -n production -o yaml
```

1.11 kubeconfig Deep Dive

The **kubeconfig** file (default: `~/.kube/config`) stores credentials and connection info.
It can contain multiple clusters, users, and contexts.

```
apiVersion: v1
kind: Config
preferences: {}

clusters:
- name: production-cluster
  cluster:
    server: https://api.production.example.com:6443
    certificate-authority-data: <base64-ca-cert>

- name: staging-cluster
  cluster:
    server: https://api.staging.example.com:6443
    certificate-authority-data: <base64-ca-cert>

users:
- name: admin-user
  user:
    client-certificate-data: <base64-client-cert>
    client-key-data: <base64-client-key>

- name: readonly-user
  user:
    token: eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...

contexts:
- name: prod
  context:
    cluster: production-cluster
    user: admin-user
    namespace: default

- name: staging
  context:
    cluster: staging-cluster
    user: admin-user
    namespace: staging

current-context: prod
```

```
# View current context details
kubectl config view
kubectl config view --minify                     # Only current context
kubectl config current-context

# Merge multiple kubeconfigs
KUBECONFIG=~/.kube/config:~/extra-cluster.yaml kubectl config view --flatten \
  > ~/.kube/merged-config
export KUBECONFIG=~/.kube/config:~/cluster2.yaml  # Use multiple files

# Extract a single context to a new file
kubectl config view --minify --context=staging --kubeconfig=~/.kube/config \
  > staging-kubeconfig.yaml
```

1.12 Pod Lifecycle States

Understanding Pod lifecycle states is essential for debugging:

Phase	Description
Pending	Pod has been accepted by the cluster but containers are not yet running. Could be waiting for scheduling, image pull, or volume attachment.
Running	At least one container has started and no container has failed yet.
Succeeded	All containers have terminated successfully (exit code 0). Terminal state for Jobs.
Failed	At least one container terminated with a non-zero exit code.
Unknown	The node running the Pod is unreachable; status cannot be determined.

Container states within a Pod:

State	Description
Waiting	Not running yet; reasons: ContainerCreating, PodInitializing, ImagePullBackOff, CrashLoopBackOff
Running	Container is executing
Terminated	Container exited; check exit code and reason

```
# See Pod phase
kubectl get pod mypod -o jsonpath='{.status.phase}'

# See container states
kubectl get pod mypod -o jsonpath='{.status.containerStatuses[*].state}'

# Full state breakdown
kubectl describe pod mypod | grep -A 5 "State:"

# Common failure states and diagnosis
kubectl get pods | grep -v Running | grep -v Completed  # Non-healthy pods
kubectl describe pod <pod> | grep -A 20 "Events:"       # What went wrong
kubectl logs <pod> --previous                            # Logs from crashed container
```

CrashLoopBackOff Explained

CrashLoopBackOff means the container keeps crashing and Kubernetes is backing off before restarting it again (10s, 20s, 40s, up to 5 minutes). Common causes:

- Application exits immediately (configuration error, missing file, bad entrypoint)
- Liveness probe failing repeatedly
- OOM kill (Out Of Memory)

```
# Diagnose CrashLoopBackOff
kubectl describe pod <pod>               # Check "Last State" and exit code
kubectl logs <pod> --previous            # Logs from the previous crashed container
kubectl logs <pod> -c <container>        # Specific container

# OOM kill: exit code 137, reason OOMKilled
kubectl get pod <pod> -o jsonpath='{.status.containerStatuses[0].lastState.terminated}'
```

1.13 Resource Requests and Limits

Every container should define resource requests and limits. This affects both scheduling and runtime behavior.

```
containers:
- name: app
  image: myapp:1.0
  resources:
    requests:
      memory: "128Mi"    # Scheduler uses this; guaranteed minimum
      cpu: "250m"        # 250 millicores = 0.25 CPU cores
    limits:
      memory: "512Mi"    # Container is killed (OOMKilled) if it exceeds this
      cpu: "1000m"       # Container is throttled (not killed) if it exceeds this
```

Quality of Service (QoS) classes are automatically assigned based on requests/limits:

QoS Class	Condition	Eviction Priority
Guaranteed	requests == limits for CPU and memory	Last evicted
Burstable	At least one request/limit set, but not equal	Medium priority
BestEffort	No requests or limits set	First evicted under pressure

```
# Check QoS class of a Pod
kubectl get pod mypod -o jsonpath='{.status.qosClass}'

# See resource usage vs limits on nodes
kubectl describe node <node-name> | grep -A 20 "Allocated resources"

# See top resource consumers
kubectl top pods --all-namespaces --sort-by=memory
kubectl top nodes
```

1.14 The Network Model

Kubernetes imposes a flat networking model:

1. **Pod-to-Pod** — every Pod can reach every other Pod directly by IP without NAT, even across nodes
2. **Pod-to-Service** — Pods reach Services via their ClusterIP:port; kube-proxy does DNAT to a Pod
3. **External-to-Service** — External traffic enters via NodePort, LoadBalancer, or Ingress

```
External Traffic
      │
      ▼
[Ingress / LoadBalancer Service]
      │
      ▼
[Service (ClusterIP)]
      │
      ├──▶ Pod 1 (192.168.1.5)
      ├──▶ Pod 2 (192.168.1.6)
      └──▶ Pod 3 (192.168.2.3)

Pod-to-Pod direct:
192.168.1.5 ──────────────────▶ 192.168.2.3 (no NAT)
```

This flat model is implemented by CNI plugins (Calico, Flannel, Weave, Cilium).

1.15 Security Model Overview

Authentication and Authorization

```
Request ──▶ Authentication ──▶ Authorization ──▶ Admission Control ──▶ etcd
             (who are you?)     (can you do it?)   (is it valid?)
```

Authentication methods:

- **X.509 certificates** — embedded in kubeconfig; used by kubectl and kubeadm-created users
- **Bearer tokens** — used by ServiceAccounts inside Pods
- **OIDC tokens** — integration with external identity providers (Google, GitHub, etc.)
- **Webhook** — external authentication service

Authorization (RBAC):

```yaml
# Role: defines permissions in a namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: staging
  name: pod-reader
rules:
- apiGroups: [""]          # "" = core API group
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
---
# RoleBinding: connects a subject to a Role
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  namespace: staging
  name: read-pods-binding
subjects:
- kind: User
  name: developer1
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

```bash
# Test what you can do
kubectl auth can-i get pods
kubectl auth can-i get pods --as=developer1 --namespace=staging
kubectl auth can-i '*' '*' --all-namespaces           # Check superuser

# See what a serviceaccount can do
kubectl auth can-i list secrets --as=system:serviceaccount:default:my-sa
```

1.16 Summary: The 10 Things You Must Understand About Kubernetes

1. **The API server is the hub** — every operation goes through it; etcd is the only storage
2. **Declarative model** — you declare desired state; controllers reconcile toward it continuously
3. **Pods are ephemeral** — never rely on a Pod's IP or name; use Services and labels
4. **Labels are the glue** — they connect Services to Pods, Deployments to ReplicaSets
5. **Namespaces partition** — use them for environment/team isolation, RBAC scoping, quotas
6. **The kubelet is the node agent** — it's the only K8s component not running inside K8s
7. **kube-proxy implements Services** — iptables/IPVS rules on every node DNAT ClusterIP to PodIP
8. **CRI/CNI/CSI are extension points** — container runtime, network, and storage are pluggable
9. **Requests are for scheduling; limits are enforced at runtime** — QoS classes flow from this
10. **RBAC controls everything** — every API server request is authenticated and authorized

1.17 Troubleshooting the API Server

The API server is the most critical component. When it fails, `kubectl` commands hang or return connection errors.

Common Error: Unable to connect to API server

```
$ kubectl get pods
The connection to the server 192.168.1.10:6443 was refused - did you specify the right host or port?

# Diagnosis steps:
# Step 1: Can you reach the server at all?
curl -k https://192.168.1.10:6443/healthz

# Step 2: Is the API server pod running?
ssh control-plane-node
sudo crictl ps | grep kube-apiserver
# If empty → static pod manifest problem or certificate issue

# Step 3: Check API server logs
sudo crictl logs $(sudo crictl ps -q --name=kube-apiserver)

# Step 4: Check if the static pod manifest is valid
sudo cat /etc/kubernetes/manifests/kube-apiserver.yaml
sudo python3 -c "import yaml; yaml.safe_load(open('/etc/kubernetes/manifests/kube-apiserver.yaml'))"
```

Common Error: Unauthorized

```
$ kubectl get pods
Error from server (Forbidden): pods is forbidden: User "jane" cannot list resource "pods" in API group "
" in the namespace "default"

# This is an RBAC issue, not authentication
# Check what jane can do:
kubectl auth can-i list pods --as=jane
kubectl auth can-i --list --as=jane | grep pods

# Grant access:
kubectl create rolebinding jane-pod-reader \
  --clusterrole=view \
  --user=jane \
  --namespace=default
```

Common Error: x509 certificate signed by unknown authority

```
$ kubectl get pods
Unable to connect to the server: x509: certificate signed by unknown authority

# The certificate in kubeconfig does not match the cluster CA
# Regenerate kubeconfig:
sudo kubeadm kubeconfig user --client-name=admin --org=system:masters > ~/.kube/config
# Or copy admin.conf from control plane:
sudo cat /etc/kubernetes/admin.conf > ~/.kube/config
chmod 600 ~/.kube/config
```

Common Error: context deadline exceeded

```
$ kubectl get nodes
Error from server: etcd cluster is unavailable

# etcd is down or unreachable
# Check etcd pod:
sudo crictl ps | grep etcd
sudo crictl logs $(sudo crictl ps -q --name=etcd) 2>&1 | tail -20

# Check etcd data directory disk space (etcd often fails when disk is full)
df -h /var/lib/etcd
# Fix: free disk space, then restart etcd
sudo systemctl restart kubelet
```

1.18 Troubleshooting the Scheduler

Symptom: Pods stuck in Pending forever

```
$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
my-app-abc    0/1     Pending   0          15m

# Always describe first
kubectl describe pod my-app-abc

# Common Pending reasons in Events section:

# 1. Insufficient resources
# Events: 0/3 nodes are available: 3 Insufficient cpu, 3 Insufficient memory
# Fix: reduce requests, add nodes, or use a node with more capacity
kubectl get nodes -o custom-columns='NAME:.metadata.name,CPU:.status.allocatable.cpu,MEM:.status.alloca
table.memory'

# 2. No nodes match nodeSelector
# Events: 0/3 nodes are available: 3 node(s) didn't match Pod's node affinity/selector
kubectl get nodes --show-labels | grep disktype
kubectl label node worker-1 disktype=ssd  # Add missing label

# 3. Taint not tolerated
# Events: 0/3 nodes are available: 3 node(s) had untolerated taint {node-role.kubernetes.io/control-
plane: }
# Fix: Add toleration or use worker nodes

# 4. PVC not bound
# Events: persistentvolumeclaim "my-pvc" not found
# Events: waiting for a volume to be created, either by external provisioner
kubectl get pvc  # Check PVC status
kubectl describe pvc my-pvc  # Find why it's pending

# 5. Scheduler not running
kubectl -n kube-system get pod | grep scheduler
kubectl -n kube-system logs kube-scheduler-controlplane --tail=30
```

1.19 Understanding Events and their Lifecycle

Events are the most powerful debugging tool and are often ignored. They expire after 1 hour by default.

```
# Show all Warning events across entire cluster (sorted newest first)
kubectl get events --all-namespaces --field-selector type=Warning \
  --sort-by='.lastTimestamp'

# Show Normal events too
kubectl get events -n production --sort-by='.lastTimestamp' | tail -30

# Watch events in real time during a deployment
kubectl get events -n production -w &
kubectl apply -f deployment.yaml

# Events are also stored in the object's describe output
kubectl describe pod crashing-pod | grep -A 20 "Events:"
# Events:
#   Type     Reason     Age                From               Message
#   ----     ------     ----               ----               -------
#   Normal   Scheduled  2m                 default-scheduler  Successfully assigned default/crashing-pod
to worker-1
#   Normal   Pulling    2m                 kubelet            Pulling image "myapp:bad-tag"
#   Warning  Failed     2m                 kubelet            Failed to pull image: rpc error: ...
#   Warning  BackOff    1m (x5 over 2m)    kubelet            Back-off pulling image "myapp:bad-tag"

# Get events for a specific pod
kubectl get events --field-selector involvedObject.name=my-pod,involvedObject.kind=Pod

# Count events by reason
kubectl get events -A -o json | python3 -c "
from collections import Counter
import sys, json
data = json.load(sys.stdin)
reasons = [e['reason'] for e in data['items']]
for r, c in Counter(reasons).most_common(15):
    print(f'{c:4d}  {r}')
"
```

1.20 Common kubectl Errors and Fixes

```
# Error: resource not found
$ kubectl get deployments
Error from server (NotFound): deployments.apps "my-app" not found
# Check namespace: kubectl get deployments -n staging
# Check spelling: kubectl get deployments --all-namespaces | grep my

# Error: invalid API version
$ kubectl apply -f old-manifest.yaml
error: unable to recognize "old-manifest.yaml": no kind is registered for the version "extensions/
v1beta1"
# Fix: update apiVersion. Common migrations:
# extensions/v1beta1 Deployment → apps/v1
# extensions/v1beta1 Ingress → networking.k8s.io/v1
# batch/v1beta1 CronJob → batch/v1

# Error: field is immutable
$ kubectl apply -f service.yaml
The Service "my-svc" is invalid: spec.clusterIP: Invalid value: "": field is immutable
# Fix: delete and recreate the service
kubectl delete svc my-svc
kubectl apply -f service.yaml

# Error: must specify resource version on update
$ kubectl replace -f pod.yaml
error: Replace failed: Operation cannot be fulfilled on pods "my-pod": the object has been modified
# Fix: get fresh copy and retry
kubectl get pod my-pod -o yaml > fresh-pod.yaml
# Edit fresh-pod.yaml
kubectl replace -f fresh-pod.yaml

# Error: no matches for kind
$ kubectl apply -f crd-resource.yaml
error: no matches for kind "MyCustomResource" in version "mygroup.io/v1"
# The CRD hasn't been installed yet
kubectl get crd | grep mygroup
kubectl apply -f crd-definition.yaml  # Install CRD first
```

Chapter 2 — Cluster Installation from Scratch with kubeadm

2.1 Overview and Prerequisites

`kubeadm` is the official Kubernetes tool for bootstrapping a cluster. It handles generating TLS certificates, configuring the control plane components, and setting up the Kubernetes API server. Understanding this process end-to-end is **required** for the CKA exam.

A minimum kubeadm cluster requires:

- **1 control-plane node** (at least 2 CPUs, 2 GB RAM)
- **1+ worker nodes** (at least 1 CPU, 1 GB RAM)
- All nodes on the same network, able to reach each other
- Unique hostnames, MAC addresses, and product UUIDs on each node
- Required ports open (API server 6443, etcd 2379-2380, kubelet 10250, etc.)
- Swap disabled on all nodes

2.2 Step 1 — Prepare Every Node (Control Plane and Workers)

These steps must be run on **all nodes** before running kubeadm.

Disable swap permanently (Kubernetes requires swap off):

```
# Disable swap immediately
sudo swapoff -a

# Disable swap permanently across reboots
sudo sed -i '/ swap / s/^\(.*\)$/#\1/g' /etc/fstab
```

What this does: Kubernetes' memory management and the kubelet both require swap to be disabled. The sed command comments out the swap line in `/etc/fstab` so it does not re-enable after reboot. Verify with `free -h` — the Swap row should show 0.

Set required kernel modules:

```
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF

sudo modprobe overlay
sudo modprobe br_netfilter
```

What this does: `overlay` is the filesystem driver used by the container runtime. `br_netfilter` enables bridge network filtering required by Kubernetes networking. Loading them with `modprobe` applies them immediately; the file ensures they load on every reboot.

Set required kernel networking parameters (sysctl):

```
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF

# Apply immediately without rebooting
sudo sysctl --system
```

What this does: These three parameters are required by Kubernetes networking:

- `net.bridge.bridge-nf-call-iptables`: Allows iptables to see bridged traffic (needed for NetworkPolicy)
- `net.ipv4.ip_forward`: Allows the node to forward packets between network interfaces (required for Pod routing)

2.3 Step 2 — Install a Container Runtime (containerd)

Kubernetes uses the **Container Runtime Interface (CRI)** to talk to a container runtime. `containerd` is the standard runtime used in most production clusters.

```
# Install containerd
sudo apt-get update
sudo apt-get install -y containerd

# Generate default config
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml

# Configure containerd to use systemd cgroup driver (CRITICAL)
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml

# Restart and enable containerd
sudo systemctl restart containerd
sudo systemctl enable containerd

# Verify it is running
sudo systemctl status containerd
```

Why SystemdCgroup = true matters: Kubernetes uses the systemd cgroup driver by default since v1.22. If containerd uses a different cgroup driver (`cgroupfs`), there is a mismatch that causes kubelet failures and node instability. Always set `SystemdCgroup = true`.

Verify containerd is working with crictl:

```
sudo crictl --runtime-endpoint unix:///run/containerd/containerd.sock ps
```

2.4 Step 3 — Install kubeadm, kubelet, and kubectl

```
# Install dependencies
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg

# Add Kubernetes apt repository (for Kubernetes 1.31)
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.31/deb/Release.key | \
  sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] \
  https://pkgs.k8s.io/core:/stable:/v1.31/deb/ /' | \
  sudo tee /etc/apt/sources.list.d/kubernetes.list

# Install the three components
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl

# Pin versions to prevent accidental upgrades
sudo apt-mark hold kubelet kubeadm kubectl

# Enable kubelet service (kubeadm will start it)
sudo systemctl enable kubelet
```

What each component does:

- `kubelet` — the agent that runs on every node and manages containers. It receives instructions from the API server and reports node/pod status back.
- `kubeadm` — the bootstrap tool. Used only during cluster setup and upgrades.
- `kubectl` — the CLI tool for interacting with the cluster after setup.

2.5 Step 4 — Initialize the Control Plane Node

Run **only on the control-plane node**:

```
sudo kubeadm init \
  --pod-network-cidr=192.168.0.0/16 \
  --kubernetes-version=1.31.0 \
  --apiserver-advertise-address=<CONTROL_PLANE_IP>
```

Flag explanations:

Flag	Purpose
`--pod-network-cidr`	CIDR range for the Pod network. **Must match** what your CNI plugin expects (192.168.0.0/16 for Calico, 10.244.0.0/16 for Flannel)
`--kubernetes-version`	Pin the exact Kubernetes version
`--apiserver-advertise-address`	The IP address the API server will advertise — must be reachable by worker nodes

After `kubeadm init` completes successfully, it prints three critical pieces of output:

```
Your Kubernetes control-plane has initialized successfully!

To start using your cluster, run as a regular user:
  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

You should now deploy a pod network to the cluster...

Then you can join any number of worker nodes by running:
kubeadm join 192.168.1.100:6443 --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

Configure kubectl access immediately:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

What this does: Copies the admin kubeconfig to your home directory so `kubectl` can authenticate with the cluster. Without this step, `kubectl` will fail with authentication errors.

Verify control plane is up:

```
kubectl get nodes
# Output: control-plane node in NotReady state (normal — CNI not yet installed)

kubectl get pods -n kube-system
# All control-plane pods should be Running except CoreDNS (waits for CNI)
```

2.6 Step 5 — Install a CNI Plugin (Calico)

Without a CNI plugin, all Pods are in `Pending` state and CoreDNS will not start.

```
# Install Calico CNI (matches --pod-network-cidr=192.168.0.0/16)
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml

# Wait for Calico pods to come up
kubectl get pods -n kube-system -w

# Once CalicoPods are Running, the node becomes Ready
kubectl get nodes
# NAME             STATUS   ROLES           AGE   VERSION
# control-plane    Ready    control-plane   3m    v1.31.0
```

What a CNI plugin does: The Container Network Interface plugin sets up the Pod network. It assigns
IP addresses to Pods, enables Pod-to-Pod communication across nodes, and enforces NetworkPolicies.
Calico uses BGP routing between nodes; Flannel uses VXLAN tunneling.

2.7 Step 6 — Join Worker Nodes

Run **on each worker node** (using the join command printed by `kubeadm init`):

```
sudo kubeadm join 192.168.1.100:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

If the join token has expired (tokens expire after 24 hours):

```
# On the control-plane node, generate a new token
kubeadm token create --print-join-command
```

What this does: Generates a fresh join token and prints the complete `kubeadm join` command. Copy and run it on the worker node.

Verify the worker joined:

```
kubectl get nodes
# NAME             STATUS   ROLES           AGE   VERSION
# control-plane    Ready    control-plane   5m    v1.31.0
# worker-1         Ready    <none>          1m    v1.31.0
```

Label a worker node with its role:

```
kubectl label node worker-1 node-role.kubernetes.io/worker=worker
```

2.8 kubeadm Certificate Management

Kubernetes TLS certificates (used by all control plane components) expire after **1 year** by default.

Check certificate expiry dates:

```
sudo kubeadm certs check-expiration
```

Output shows each certificate, its expiry date, and the CA that signed it. Run this routinely in production clusters.

Renew all certificates manually:

```
sudo kubeadm certs renew all
```

Renew a specific certificate:

```
sudo kubeadm certs renew apiserver
sudo kubeadm certs renew etcd-server
```

After renewing certificates, restart the static pod manifests by moving them out and back:

```
cd /etc/kubernetes/manifests
sudo mv kube-apiserver.yaml /tmp/
sleep 5
sudo mv /tmp/kube-apiserver.yaml .
```

2.9 Resetting a Node with kubeadm

To cleanly remove Kubernetes from a node (for reimaging or troubleshooting):

```
# On the node to reset
sudo kubeadm reset

# Then clean up CNI and iptables rules
sudo rm -rf /etc/cni/net.d
sudo iptables -F && sudo iptables -t nat -F && sudo iptables -t mangle -F && sudo iptables -X
sudo ipvsadm --clear  # if using ipvs

# Remove the node from the cluster (run on control-plane)
kubectl delete node <node-name>
```

2.10 Troubleshooting kubeadm init Failures

Error: [preflight] running preflight checks

```
$ sudo kubeadm init
[preflight] Running pre-flight checks
error execution phase preflight: [preflight] Some fatal errors occurred:
        [ERROR NumCPU]: the number of available CPUs 1 is less than the required 2
        [ERROR Mem]: the system RAM (967 MB) is less than the minimum 1700 MB
        [ERROR Swap]: running with swap on is not supported. Please disable swap

# Fix NumCPU/RAM: Use a larger VM
# Fix Swap:
sudo swapoff -a
sudo sed -i '/ swap / s/^/#/' /etc/fstab

# Force init despite preflight failures (development only, never production):
sudo kubeadm init --ignore-preflight-errors=NumCPU,Mem
```

Error: Port already in use

```
$ sudo kubeadm init
[preflight] Some fatal errors occurred:
        [ERROR Port-6443]: Port 6443 is in use
        [ERROR Port-10259]: Port 10259 is in use

# A previous failed kubeadm run left processes running
# Or a duplicate control-plane component is running
sudo netstat -tlnp | grep -E '6443|10259|10257|2379|2380'
sudo fuser -k 6443/tcp 10259/tcp  # Kill processes on those ports
# Or reset completely:
sudo kubeadm reset -f
sudo rm -rf /etc/kubernetes /var/lib/etcd /var/lib/kubelet ~/.kube
```

Error: node not found after joining

```
$ kubectl get nodes
NAME            STATUS     ROLES           AGE   VERSION
control-plane   Ready      control-plane   10m   v1.30.0
# Worker node doesn't appear

# On the worker node, check kubelet:
sudo systemctl status kubelet
sudo journalctl -u kubelet -n 30

# Common cause: join token expired (tokens expire after 24h)
# Fix: generate a new token on the control plane
sudo kubeadm token create --print-join-command

# Common cause: worker can't reach API server (firewall)
# Test from worker:
curl -k https://CONTROL_PLANE_IP:6443/healthz
# Should return "ok"

# Common cause: wrong CA hash
# Regenerate the correct hash:
openssl x509 -pubkey -in /etc/kubernetes/pki/ca.crt | \
  openssl rsa -pubin -outform der 2>/dev/null | \
  openssl dgst -sha256 -hex | sed 's/^.* //'
```

Error: CrashLoopBackOff for coredns Pods after init

```
$ kubectl get pods -n kube-system
NAME                     READY   STATUS             RESTARTS   AGE
coredns-xxx              0/1     CrashLoopBackOff   5          5m
coredns-yyy              0/1     CrashLoopBackOff   5          5m

# Cause: No CNI plugin installed yet
# CoreDNS needs the cluster network to be operational
# Install your CNI plugin first:
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.27.0/manifests/calico.yaml

# Then wait for CoreDNS to recover automatically
kubectl get pods -n kube-system -w
```

Error: kubelet not starting after kubeadm join

```
$ sudo systemctl status kubelet
● kubelet.service - kubelet: The Kubernetes Node Agent
   Loaded: loaded (/lib/systemd/system/kubelet.service; enabled)
   Active: activating (auto-restart) (Result: exit-code)

$ sudo journalctl -u kubelet --since "5 minutes ago" | tail -20
"Failed to create kubelet: misconfiguration: kubelet cgroup driver: "cgroupfs" is different from docker
cgroup driver: "systemd""

# Fix: align cgroup drivers
# containerd uses systemd by default in modern K8s
# Check current driver:
sudo containerd config dump | grep -A 5 "SystemdCgroup"

# Fix kubelet to use systemd:
cat /var/lib/kubelet/config.yaml | grep cgroupDriver
# Edit /var/lib/kubelet/config.yaml:
#   cgroupDriver: systemd
sudo systemctl restart kubelet
```

2.11 Post-Installation Verification Checklist

```
# Complete health check after kubeadm install:

echo "=== 1. Node Status ==="
kubectl get nodes -o wide

echo "=== 2. System Pods ==="
kubectl get pods -n kube-system

echo "=== 3. Component Status ==="
kubectl get componentstatuses 2>/dev/null || echo "Note: deprecated, use pod checks"

echo "=== 4. API Server Health ==="
kubectl get --raw /healthz
kubectl get --raw /readyz
kubectl get --raw /livez

echo "=== 5. etcd Health ==="
kubectl exec -it etcd-$(hostname) -n kube-system -- \
  etcdctl endpoint health \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

echo "=== 6. DNS Functional Test ==="
kubectl run dns-test --image=busybox:1.35 --restart=Never --rm -it -- \
  nslookup kubernetes.default.svc.cluster.local

echo "=== 7. Pod-to-Pod Connectivity Test ==="
kubectl run pod1 --image=nginx --restart=Never
kubectl run pod2 --image=busybox:1.35 --restart=Never -- sleep 3600
sleep 5
POD1_IP=$(kubectl get pod pod1 -o jsonpath='{.status.podIP}')
kubectl exec pod2 -- wget -qO- http://$POD1_IP
kubectl delete pod pod1 pod2

echo "=== 8. External Connectivity from Pod ==="
kubectl run ext-test --image=busybox:1.35 --restart=Never --rm -it -- \
  wget -qO- http://httpbin.org/ip
```

2.12 kubeadm Upgrade Workflow

```
# Check available versions
sudo apt-cache madison kubeadm | head -5

# Step 1: Upgrade kubeadm on control plane first
sudo apt-mark unhold kubeadm
sudo apt-get install -y kubeadm=1.31.0-1.1
sudo apt-mark hold kubeadm

# Step 2: Verify the upgrade plan
sudo kubeadm upgrade plan
# Shows: Component → Current → Available
# Gives you the exact command to run

# Step 3: Apply the upgrade
sudo kubeadm upgrade apply v1.31.0
# This upgrades: kube-apiserver, kube-controller-manager, kube-scheduler
# It does NOT upgrade: kubelet, kubectl (done separately below)

# Step 4: Upgrade kubelet and kubectl on control plane
sudo apt-mark unhold kubelet kubectl
sudo apt-get install -y kubelet=1.31.0-1.1 kubectl=1.31.0-1.1
sudo apt-mark hold kubelet kubectl
sudo systemctl daemon-reload && sudo systemctl restart kubelet

# Step 5: Verify control plane
kubectl get nodes
# NAME            STATUS   ROLES           VERSION
# control-plane   Ready    control-plane   v1.31.0   ← Upgraded
# worker-1        Ready    <none>          v1.30.0   ← Still old

# Step 6: For each worker node:
# First drain the worker (from control plane):
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data

# On worker-1:
sudo apt-mark unhold kubeadm kubelet kubectl
sudo apt-get install -y kubeadm=1.31.0-1.1 kubelet=1.31.0-1.1 kubectl=1.31.0-1.1
sudo apt-mark hold kubeadm kubelet kubectl
sudo kubeadm upgrade node
sudo systemctl daemon-reload && sudo systemctl restart kubelet

# Back on control plane - uncordon the node:
kubectl uncordon worker-1

# Verify
kubectl get nodes
# All nodes should show v1.31.0
```

Chapter 3 — Highly-Available Control Plane Architecture

3.1 Why High Availability Matters

A single-node control plane means a single point of failure. If the control-plane node goes down, no new Pods can be scheduled, no workloads can be updated, and no administrative actions work — even though existing running Pods continue to operate. For production clusters, a highly-available (HA) control plane uses **multiple control-plane nodes** to eliminate this risk.

3.2 HA Topology Options

Kubernetes supports two HA topologies:

Stacked etcd (most common)

Each control-plane node runs etcd **alongside** the API server, scheduler, and controller-manager. etcd is co-located on the same node as the other control-plane components.

```
[Load Balancer]  ← kubectl and worker nodes connect here
       │
  ┌────┴──────────────────────────────────────────────────┐
  │                            │                          │
[CP Node 1]              [CP Node 2]              [CP Node 3]
 API Server               API Server               API Server
 Scheduler                Scheduler                Scheduler
 Ctrl Manager             Ctrl Manager             Ctrl Manager
 etcd                     etcd                     etcd
  └──────────────────────── etcd cluster ──────────────────┘
```

Advantages: Simpler setup, fewer nodes required.
Disadvantages: etcd and control-plane share resources; a node failure loses both.

External etcd

etcd runs on separate dedicated nodes. Control-plane nodes only run API server, scheduler, and controller-manager.

Advantages: Failure domains are separated; etcd performance is isolated.
Disadvantages: Requires more machines (minimum 6: 3 control-plane + 3 etcd).

3.3 Setting Up an HA Cluster with kubeadm

Step 1: Set up a load balancer for the API server

The load balancer receives all kubectl and node traffic on port **6443** and distributes it across the control-plane nodes. Options: HAProxy, AWS NLB, GCP TCP Load Balancer, kube-vip.

A minimal HAProxy configuration:

```
frontend k8s-api
  bind *:6443
  default_backend k8s-control-planes

backend k8s-control-planes
  balance roundrobin
  server cp1 192.168.1.10:6443 check
  server cp2 192.168.1.11:6443 check
  server cp3 192.168.1.12:6443 check
```

Step 2: Initialize the first control-plane node

```
sudo kubeadm init \
  --control-plane-endpoint "LOAD_BALANCER_DNS:6443" \
  --upload-certs \
  --pod-network-cidr=192.168.0.0/16
```

The critical flags for HA:

- `--control-plane-endpoint` — Points to the load balancer VIP/DNS, **not** this node's IP. Every kubeconfig and node configuration will use this address.
- `--upload-certs` — Uploads the control-plane certificates to a Kubernetes Secret so they can be automatically distributed to other control-plane nodes during join.

Step 3: Save the two join commands from the output

`kubeadm init` prints two join commands:

```
# For additional control-plane nodes:
kubeadm join LOAD_BALANCER_DNS:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --control-plane \
  --certificate-key <cert-key>

# For worker nodes:
kubeadm join LOAD_BALANCER_DNS:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash>
```

Step 4: Join additional control-plane nodes

```
# On cp2 and cp3 — use the CONTROL-PLANE join command (with --control-plane flag)
sudo kubeadm join LOAD_BALANCER_DNS:6443 \
  --token <token> \
  --discovery-token-ca-cert-hash sha256:<hash> \
  --control-plane \
  --certificate-key <cert-key>
```

Verify the HA cluster:

```
kubectl get nodes
# NAME    STATUS   ROLES           AGE
# cp1     Ready    control-plane   10m
# cp2     Ready    control-plane   5m
# cp3     Ready    control-plane   2m

# Check etcd cluster health
kubectl exec -it etcd-cp1 -n kube-system -- sh -c \
  'ETCDCTL_API=3 etcdctl member list \
   --endpoints=https://127.0.0.1:2379 \
   --cacert=/etc/kubernetes/pki/etcd/ca.crt \
   --cert=/etc/kubernetes/pki/etcd/server.crt \
   --key=/etc/kubernetes/pki/etcd/server.key'
```

3.4 etcd Quorum and Fault Tolerance

etcd uses the **Raft consensus algorithm**, which requires a quorum (majority) of members to be available for the cluster to function.

Total Members	Quorum Required	Tolerated Failures
1	1	0
3	2	**1** ← minimum HA
5	3	**2** ← recommended production
7	4	3

Why odd numbers? With 4 members, quorum is 3 — tolerating only 1 failure, same as 3 members but with more cost. Always use odd member counts.

```
# List etcd members and their status
ETCDCTL_API=3 etcdctl member list \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  -w table

# Output:
# +------------------+---------+---------+----------------------------+----------------------------+
# |        ID        | STATUS  |  NAME   |         PEER ADDRS         |        CLIENT ADDRS        |
# +------------------+---------+---------+----------------------------+----------------------------+
# | 1a6f9aa6b47e62b1 | started | cp1     | https://192.168.1.10:2380  | https://192.168.1.10:2379  |
# | 2b7e0bb7c58f73c2 | started | cp2     | https://192.168.1.11:2380  | https://192.168.1.11:2379  |
# | 3c8f1cc8d69084d3 | started | cp3     | https://192.168.1.12:2380  | https://192.168.1.12:2379  |
# +------------------+---------+---------+----------------------------+----------------------------+

# Check endpoint health
ETCDCTL_API=3 etcdctl endpoint health \
  --endpoints=https://192.168.1.10:2379,https://192.168.1.11:2379,https://192.168.1.12:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Check which node is leader
ETCDCTL_API=3 etcdctl endpoint status \
  --endpoints=https://192.168.1.10:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  -w table
```

3.5 etcd Backup and Restore (Critical for CKA)

Backing up etcd is one of the most tested CKA exam skills. The backup is a **snapshot** of all cluster state — all Pods, Deployments, Secrets, ConfigMaps, RBAC rules, etc.

Taking a Snapshot Backup

```
# Set environment variable for convenience
export ETCDCTL_API=3

# Take a snapshot
etcdctl snapshot save /opt/etcd-backup-$(date +%Y%m%d%H%M%S).db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Verify the snapshot
etcdctl snapshot status /opt/etcd-backup-20240101120000.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  -w table

# Output:
# +----------+----------+------------+------------+
# |   HASH   | REVISION | TOTAL KEYS | TOTAL SIZE |
# +----------+----------+------------+------------+
# | 52750ab4 |     1234 |        847 |     3.7 MB |
# +----------+----------+------------+------------+
```

Restoring from a Snapshot

IMPORTANT: On the CKA exam, the restore process is a common task. Follow this exactly:

```
# Step 1: Stop the API server (if using static Pods, temporarily move the manifest)
sudo mv /etc/kubernetes/manifests/kube-apiserver.yaml /tmp/
# Wait for API server to stop (check: sudo crictl ps | grep apiserver)

# Step 2: Restore the snapshot to a new data directory
ETCDCTL_API=3 etcdctl snapshot restore /opt/etcd-backup.db \
  --data-dir=/var/lib/etcd-restored \
  --initial-cluster="cp1=https://192.168.1.10:2380" \
  --initial-advertise-peer-urls="https://192.168.1.10:2380" \
  --name=cp1

# Step 3: Update the etcd static Pod manifest to use the new data dir
# Edit /etc/kubernetes/manifests/etcd.yaml:
# Find: --data-dir=/var/lib/etcd
# Change to: --data-dir=/var/lib/etcd-restored
# Also update the hostPath volume for the data directory

sudo sed -i 's|/var/lib/etcd|/var/lib/etcd-restored|g' /etc/kubernetes/manifests/etcd.yaml

# Step 4: Restore the API server manifest
sudo mv /tmp/kube-apiserver.yaml /etc/kubernetes/manifests/

# Step 5: Wait for etcd and API server to restart
sudo crictl ps | grep etcd
kubectl get nodes    # Should work once API server is back
```

3.6 kube-vip: A Software Load Balancer Alternative

For on-premise or bare-metal clusters without an external load balancer, **kube-vip** provides a virtual IP (VIP) that floats between control-plane nodes:

```
# Install kube-vip as a static Pod on the first control-plane node
# First, get the kube-vip manifest
export VIP=192.168.1.100   # The virtual IP for the control plane
export INTERFACE=eth0        # Network interface

docker run --rm ghcr.io/kube-vip/kube-vip:v0.7.0 \
  manifest pod \
  --interface $INTERFACE \
  --address $VIP \
  --controlplane \
  --arp \
  --leaderElection | sudo tee /etc/kubernetes/manifests/kube-vip.yaml
```

With kube-vip running, use the VIP as the `--control-plane-endpoint` in `kubeadm init`.

3.7 Testing HA Failover

After setting up an HA cluster, verify that it actually tolerates node failure:

```
# Step 1: Identify the current etcd leader
ETCDCTL_API=3 etcdctl endpoint status \
  --endpoints=https://192.168.1.10:2379,https://192.168.1.11:2379,https://192.168.1.12:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  -w table | grep true    # IS LEADER column

# Step 2: Simulate failure by stopping the leader's kubelet
# On the leader node:
sudo systemctl stop kubelet

# Step 3: Watch leader election happen (from another control-plane node)
ETCDCTL_API=3 etcdctl endpoint status \
  --endpoints=https://192.168.1.11:2379,https://192.168.1.12:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  -w table

# Step 4: Verify cluster still works
kubectl get pods -A               # Should still work via load balancer
kubectl create deployment test --image=nginx

# Step 5: Restore the failed node
# On the original leader node:
sudo systemctl start kubelet

# The node rejoins the cluster automatically
kubectl get nodes
```

3.8 Certificate Management in HA Clusters

In HA clusters, all control-plane nodes share the **same Certificate Authority (CA)**. kubeadm handles distributing certificates when using `--upload-certs`.

```
# Check certificate expiry (run on any control-plane node)
sudo kubeadm certs check-expiration

# Output shows expiry for each cert:
# CERTIFICATE                EXPIRES                RESIDUAL TIME   ...
# admin.conf                 Jan 15, 2026 10:00 UTC 364d            OK
# apiserver                  Jan 15, 2026 10:00 UTC 364d            OK
# etcd-ca                    Jan 12, 2034 10:00 UTC 9y              OK

# Renew all certificates (do before they expire!)
sudo kubeadm certs renew all

# After renewing, restart the static Pod components
sudo crictl rm $(sudo crictl ps -q)   # kills and restarts all static Pods

# Update the admin kubeconfig after renewal
sudo cp /etc/kubernetes/admin.conf ~/.kube/config
```

Chapter 4 — Extension Interfaces: CNI, CSI, and CRI

4.1 The Kubernetes Extension Architecture

Kubernetes is deliberately designed to be pluggable. Rather than hardcoding one container runtime, one network solution, or one storage system, Kubernetes defines **standard interfaces** that any conforming implementation can fulfill. This architecture is why Kubernetes runs on every cloud and supports hundreds of storage and networking solutions.

The three critical interfaces for the CKA exam are:

Interface	Full Name	Purpose
CRI	Container Runtime Interface	How Kubernetes talks to the container runtime
CNI	Container Network Interface	How Kubernetes sets up Pod networking
CSI	Container Storage Interface	How Kubernetes provisions and mounts storage volumes

---

4.2 CRI — Container Runtime Interface

The CRI is a gRPC API that the kubelet uses to manage container lifecycles. Any container runtime that implements the CRI can run containers in Kubernetes.

Common CRI-compliant runtimes:

Runtime	Description	Use Case
`containerd`	Industry standard, ships with Docker	Most production clusters
`CRI-O`	Lightweight, purpose-built for Kubernetes	OpenShift, security-focused deployments
`Docker Engine`	Via `cri-dockerd` shim (deprecated from k8s 1.24)	Legacy only

Checking which runtime a node uses:

```
kubectl get node worker-1 -o jsonpath='{.status.nodeInfo.containerRuntimeVersion}'
# Output: containerd://1.7.2
```

Using crictl to interact with the container runtime directly:

`crictl` is a CLI tool that speaks the CRI protocol directly — bypassing kubectl entirely. It is available on every Kubernetes node and is essential for troubleshooting when the API server is down.

```
# List all running containers (equivalent to docker ps)
sudo crictl ps

# List all pods managed by the runtime
sudo crictl pods

# Pull an image
sudo crictl pull nginx:1.25

# Inspect a container
sudo crictl inspect <container-id>

# Get logs from a container (when kubectl logs is unavailable)
sudo crictl logs <container-id>

# Remove a stopped container
sudo crictl rm <container-id>
```

Configure crictl to find the runtime socket:

```
sudo crictl config --set runtime-endpoint=unix:///run/containerd/containerd.sock
```

4.3 CNI — Container Network Interface

The CNI is a specification and set of libraries for configuring network interfaces in Linux containers. When Kubernetes creates a Pod, the kubelet calls the CNI plugin to:

1. Create a network interface (veth pair) for the Pod
2. Assign an IP address from the Pod CIDR
3. Set up routing so other Pods can reach this Pod

CNI plugin configuration location:

```
# CNI plugin binaries live here on each node
ls /opt/cni/bin/
# calico  calico-ipam  flannel  bridge  host-local  loopback  portmap ...

# CNI network configuration lives here
cat /etc/cni/net.d/10-calico.conflist
```

Common CNI plugins and their characteristics:

CNI Plugin	Network Model	NetworkPolicy Support	Notes
Calico	BGP or VXLAN	✅ Full support	Most feature-rich, production standard
Flannel	VXLAN	❌ No native support	Simple, low overhead
Weave Net	Mesh	✅ Full support	Easy setup
Cilium	eBPF-based	✅ Full support	High performance, modern

Diagnosing CNI issues:

```
# Check if CNI pods are running
kubectl get pods -n kube-system | grep -E "calico|flannel|cilium|weave"

# Check CNI plugin logs
kubectl logs -n kube-system -l k8s-app=calico-node --tail=50

# Check if a node has a PodCIDR assigned
kubectl get node worker-1 -o jsonpath='{.spec.podCIDR}'

# Test pod-to-pod connectivity
kubectl run test1 --image=busybox --rm -it --restart=Never -- \
  ping -c 3 <pod-ip-of-another-pod>
```

4.4 CSI — Container Storage Interface

The CSI is a standard for exposing arbitrary block and file storage systems to containerized workloads. Before CSI, storage drivers were compiled directly into the Kubernetes codebase (called "in-tree" drivers). CSI allows storage vendors to develop and ship their drivers independently as containers, called **CSI drivers**.

How CSI works end-to-end:

```
[PVC created] → [StorageClass selects CSI driver] → [CSI driver provisions volume in storage backend]
             → [Volume is bound to PVC] → [CSI driver mounts volume into Pod filesystem]
```

Common CSI drivers:

Driver	Storage Backend
`ebs.csi.aws.com`	AWS EBS volumes
`disk.csi.azure.com`	Azure Managed Disks
`pd.csi.storage.gke.io`	GCE Persistent Disks
`nfs.csi.k8s.io`	NFS shared storage
`rook-ceph.rbd.csi.ceph.com`	Ceph block storage

Check installed CSI drivers:

```
kubectl get csidriver
# NAME                    ATTACHREQUIRED   PODINFOONMOUNT   STORAGECAPACITY
# ebs.csi.aws.com        true             false            false

kubectl get csistoragecapacities -A
```

Check CSI node information:

```
kubectl get csinode
# Shows which CSI drivers are registered on each node
```

CSI driver components in the cluster:

```
# CSI drivers run as Pods — usually in kube-system
kubectl get pods -n kube-system | grep csi
# aws-ebs-csi-controller-...   Running
# aws-ebs-csi-node-...         Running  (DaemonSet — one per node)
```

The controller component handles volume provisioning and attachment at the cloud/storage level. The node component (DaemonSet) handles mounting the volume into the Pod's filesystem on each node.

4.5 Troubleshooting CNI Issues

Symptom: Pods in ContainerCreating with network errors

```
$ kubectl describe pod my-pod
Events:
  Warning  Failed    2m   kubelet  Failed to create pod sandbox:
    rpc error: code = Unknown desc = failed to setup network for sandbox:
    plugin type="calico" failed (add): 500 Internal Server Error

# Step 1: Check if CNI plugin pods are running
kubectl get pods -n kube-system | grep -E "calico|flannel|weave|cilium"

# Step 2: Check CNI binary exists on the node
ls /opt/cni/bin/

# Step 3: Check CNI config
ls /etc/cni/net.d/
cat /etc/cni/net.d/10-calico.conflist

# Step 4: Check CNI pod logs
kubectl logs -n kube-system -l k8s-app=calico-node --tail=50

# Step 5: If CNI pod CrashLooping, check node-specific issues
kubectl describe pod -n kube-system calico-node-xxxxx | grep -A 10 "Events:"
```

Symptom: Pod CIDR overlap with host network

```
# Error in calico-node logs:
# Failed to start Calico: 192.168.0.0/16 overlaps with host network

# The pod CIDR overlaps with your existing network
# Fix: choose a non-overlapping CIDR in kubeadm init:
sudo kubeadm init --pod-network-cidr=10.244.0.0/16

# Or reconfigure Calico to use a different CIDR
kubectl -n kube-system edit configmap calico-config
# Change CALICO_IPV4POOL_CIDR value
```

Verify CNI functionality:

```
# Test pod networking end-to-end
kubectl create deployment nginx --image=nginx --replicas=3
kubectl expose deployment nginx --port=80

# Get pod IPs
kubectl get pods -l app=nginx -o wide

# Test pod-to-pod:
kubectl run test --image=busybox --rm -it --restart=Never -- \
  wget -qO- http://nginx.default.svc.cluster.local

# Test cross-node communication
kubectl get pods -l app=nginx -o wide | awk '{print $6, $7}'  # IP and Node
# If you have 3 replicas across 2 nodes, all should be reachable
```

4.6 Troubleshooting CSI and Storage Issues

Common Error: PVC stuck in Pending

```
$ kubectl get pvc
NAME       STATUS    VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   AGE
data-pvc   Pending                                      fast-ssd       5m

kubectl describe pvc data-pvc
Events:
  Warning  ProvisioningFailed  2m  rancher.io/local-path
    failed to provision volume:
    node "worker-1" selected by affinity rule "Affinity" not found in NodeList

# Cause 1: StorageClass doesn't exist
kubectl get storageclass
kubectl describe storageclass fast-ssd    # Check provisioner

# Cause 2: No storage provisioner available
kubectl get pods -n kube-system | grep -i csi
kubectl get pods -n kube-system | grep -i provisioner

# Cause 3: Node affinity issue with local storage
kubectl describe pvc data-pvc | grep -A 10 "Node Affinity:"
```

Diagnose CSI driver issues:

```
# List installed CSI drivers
kubectl get csidrivers

# Check CSI node pods (run on every node)
kubectl get pods -n kube-system | grep csi-node

# Check CSI controller (usually a Deployment or StatefulSet)
kubectl get pods -n kube-system | grep csi-controller

# Check CSI driver logs
kubectl logs -n kube-system csi-aws-ebs-controller-xxx -c ebs-plugin --tail=30

# Check volume attachment status
kubectl get volumeattachments
kubectl describe volumeattachment <name>
```

4.7 Troubleshooting CRI Issues

Error: failed to create sandbox

```
$ kubectl describe pod my-pod
Warning  Failed  kubelet  Failed to create pod sandbox:
  rpc error: code = Unknown desc =
  failed to create containerd task: failed to create shim:
  OCI runtime create failed: container_linux.go:380:
  starting container process caused: process_linux.go:545:
  container init caused: rootfs_linux.go:76:
  mounting "/etc/resolv.conf" to rootfs at "/etc/resolv.conf" caused:
  mount through procfd: not a directory: unknown

# Cause: /etc/resolv.conf on the node is a symlink to a directory (common in systemd-resolved setups)
ls -la /etc/resolv.conf
# Fix for Ubuntu 22+:
sudo ln -sf /run/systemd/resolve/resolv.conf /etc/resolv.conf
sudo systemctl restart containerd kubelet
```

Error: ImagePullBackOff

```
$ kubectl get pods
NAME       READY   STATUS             RESTARTS   AGE
my-pod     0/1     ImagePullBackOff   0          5m

kubectl describe pod my-pod
Events:
  Warning  Failed     2m   kubelet  Failed to pull image "myregistry.example.com/myapp:v1.2":
    rpc error: code = Unknown desc = failed to pull and unpack image:
    failed to resolve reference "myregistry.example.com/myapp:v1.2":
    unexpected status code 401 Unauthorized

# Fix 1: Create imagePullSecret
kubectl create secret docker-registry registry-credentials \
  --docker-server=myregistry.example.com \
  --docker-username=myuser \
  --docker-password=mypassword \
  --docker-email=admin@example.com

# Reference in Pod spec:
# spec:
#   imagePullSecrets:
#   - name: registry-credentials

# Fix 2: Check if image actually exists
sudo crictl pull myregistry.example.com/myapp:v1.2

# Fix 3: Wrong tag - check available tags
curl -u myuser:mypassword \
  https://myregistry.example.com/v2/myapp/tags/list
```

Using crictl for CRI debugging:

```
# List running containers (like docker ps)
sudo crictl ps

# List all containers including stopped
sudo crictl ps -a

# Inspect a container
sudo crictl inspect <container-id>

# Get container logs (bypasses kubectl)
sudo crictl logs <container-id>
sudo crictl logs --tail=50 <container-id>

# List pulled images
sudo crictl images

# Pull an image manually
sudo crictl pull nginx:1.25

# Remove an image
sudo crictl rmi nginx:1.25

# Pod sandbox info
sudo crictl pods
sudo crictl inspectp <pod-sandbox-id>

# Run a command in a container (like docker exec)
sudo crictl exec -it <container-id> sh

# Get runtime info
sudo crictl info
```

Chapter 5 — Understanding Pods and Containers

Before diving into commands, it is important to understand the relationship between Pods and containers
in Kubernetes, as this directly informs how interaction commands are structured.

5.1 What Is a Pod?

A **Pod** is the smallest deployable unit in Kubernetes. Think of it as a thin wrapper around one or more containers that share the same network namespace (same IP address and port space), the same
storage volumes, and the same lifecycle.

When you run `kubectl get pods`, you see Pods — not containers directly. To interact with a container, you always address the Pod first, and then optionally target a specific container within it.

```
[Kubernetes Node]
  └── [Pod: my-app-pod]
        ├── [Container: app]       ← main application
        ├── [Container: sidecar]   ← helper (e.g., log shipper)
        └── [Shared Volume: /data]
```

5.2 Single vs. Multi-Container Pods

A single-container Pod is the most common pattern. However, multi-container Pods (using the sidecar,
ambassador, or adapter patterns) are common in production. When a Pod has multiple containers, most
`kubectl` commands require you to specify which container you want to target using the `-c` (or `--container`) flag.

5.3 The kubectl Command Structure

All Kubernetes CLI interactions follow this general pattern:

```
kubectl <verb> <resource-type> <resource-name> [flags]
```

For container-specific commands, the general structure extends to:

```
kubectl exec <pod-name> -c <container-name> -n <namespace> -- <command>
```

The double dash (`--`) is a UNIX convention that separates `kubectl` flags from the command being passed to the container's shell. Without it, argument parsing can be ambiguous.

5.4 Pod Anatomy in Depth

A Pod is the atomic unit of Kubernetes. It is **not** a container — it is a shared execution environment that wraps one or more containers. Containers in a Pod share:

- **Network namespace** — they share the same IP address and loopback interface; communicate via `localhost`
- **IPC namespace** — can use POSIX message queues and shared memory
- **Volumes** — any volume mounted to the Pod is accessible to all containers in it

Each Pod has its own **UTS namespace** (hostname) and containers have separate **PID namespaces** by default (though they can be shared with `shareProcessNamespace: true`).

```
Pod (shared network: 10.244.3.7, shared volumes)
┌──────────────────────────────────────────────────────┐
│                                                      │
│  ┌──────────────────┐   ┌──────────────────────────┐ │
│  │  main container  │   │  sidecar container       │ │
│  │  nginx:1.25      │   │  log-collector:latest    │ │
│  │  port 80         │   │  reads /var/log/nginx/   │ │
│  └──────────────────┘   └──────────────────────────┘ │
│                                                      │
│  ┌──────────────────────────────────────────────┐    │
│  │  Shared Volume: /var/log/nginx (emptyDir)    │    │
│  └──────────────────────────────────────────────┘    │
│                                                      │
│  pause container (infrastructure container)          │
│  (holds the network namespace)                       │
└──────────────────────────────────────────────────────┘
```

The pause Container

Every Pod actually contains a hidden **pause container** (also called the infrastructure container or sandbox container). It:

- Is the first container started in the Pod
- Holds the Pod's network namespace (IP address, iptables rules)
- Acts as the parent process for all other containers' namespaces
- Never does any real work — it just sleeps

```
# See the pause container on a node
sudo crictl ps | grep pause
# Or directly with containerd
sudo ctr containers list | grep pause
```

5.5 Init Containers

Init containers run and complete **before** app containers start. They are perfect for:

- Waiting for a dependency (database, service) to be ready
- Pre-populating shared volumes with configuration or data
- Running database migrations
- Performing security setup (e.g., fetching TLS certificates)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-with-init
spec:
  initContainers:
  - name: wait-for-db
    image: busybox:1.35
    command: ['sh', '-c',
      'until nc -z postgres-service 5432; do echo waiting for database; sleep 2; done']
    # This container loops until port 5432 is open on postgres-service

  - name: run-migrations
    image: myapp:latest
    command: ['sh', '-c', 'python manage.py migrate']
    env:
    - name: DATABASE_URL
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: url

  containers:
  - name: app
    image: myapp:latest
    ports:
    - containerPort: 8080
```

Key init container behaviors:

- They run in sequence (not in parallel)
- Each must complete successfully before the next starts
- If an init container fails, Kubernetes restarts the Pod (subject to `restartPolicy`)
- `kubectl logs pod-name -c wait-for-db` shows init container logs
- `kubectl describe pod pod-name` shows init container status in the `Init Containers` section

```
# Watch init container progress
kubectl get pod app-with-init -w
# NAME             READY   STATUS     RESTARTS   AGE
# app-with-init    0/1     Init:0/2   0          5s
# app-with-init    0/1     Init:1/2   0          12s
# app-with-init    0/1     PodInitializing   0   18s
# app-with-init    1/1     Running    0          20s

# Get logs from a specific init container
kubectl logs app-with-init -c wait-for-db
kubectl logs app-with-init -c run-migrations
```

5.6 Sidecar Containers

In Kubernetes 1.29+, **sidecar containers** have first-class support as a distinct container type (previously they were just regular containers by convention). Sidecars run alongside the main container and support the application:

```
spec:
  initContainers:
  - name: log-collector             # Sidecar declared in initContainers with restartPolicy
    image: fluentd:v1.16
    restartPolicy: Always            # This makes it a "native sidecar" (K8s 1.29+)
    volumeMounts:
    - name: log-volume
      mountPath: /var/log/app

  containers:
  - name: main-app
    image: myapp:latest
    volumeMounts:
    - name: log-volume
      mountPath: /var/log/app

  volumes:
  - name: log-volume
    emptyDir: {}
```

Common sidecar patterns:

- **Log collector** — Reads log files from a shared volume and ships to Elasticsearch/Splunk
- **Service mesh proxy** — Envoy/Linkerd proxies all network traffic for mTLS and observability
- **Config reloader** — Watches for ConfigMap changes and reloads the app
- **Metrics exporter** — Exposes application metrics in Prometheus format

5.7 Multi-Container Pod Patterns

Ambassador Pattern

The ambassador container proxies network connections from the main container:

```
containers:
- name: app
  image: myapp
  env:
  - name: DATABASE_HOST
    value: localhost   # Talks to ambassador on localhost
  - name: DATABASE_PORT
    value: "6432"

- name: db-ambassador
  image: pgbouncer   # Proxies to real database
  env:
  - name: DATABASE_URL
    value: "postgres://prod-db.example.com:5432/mydb"
```

Adapter Pattern

The adapter transforms output from the main container into a standard format:

```
containers:
- name: app
  image: legacy-app   # Outputs custom log format

- name: log-adapter
  image: log-transformer   # Reads custom format, outputs JSON
  volumeMounts:
  - name: logs
    mountPath: /input
```

5.8 Static Pods

Static Pods are managed directly by the kubelet on a specific node, not through the Kubernetes API server. The kubelet watches a directory (`/etc/kubernetes/manifests/` by default) and creates Pods from any YAML files found there.

Key properties:

- The kubelet restarts them automatically if they crash
- They appear in `kubectl get pods -n kube-system` as **mirror Pods** (read-only reflections)
- The control-plane components (`kube-apiserver`, `etcd`, `kube-controller-manager`, `kube-scheduler`) are all static Pods on the control-plane node

```
# See static Pod manifests on a control-plane node
ls /etc/kubernetes/manifests/
# etcd.yaml  kube-apiserver.yaml  kube-controller-manager.yaml  kube-scheduler.yaml

# Create a static Pod by placing a manifest in the directory
cat << 'EOF' > /etc/kubernetes/manifests/my-static-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-static-pod
  namespace: kube-system
spec:
  containers:
  - name: nginx
    image: nginx:1.25
EOF

# The kubelet will create it immediately; see it as a mirror Pod
kubectl get pods -n kube-system | grep my-static-pod-<node-name>

# To delete: remove the file (kubectl delete will just recreate it)
sudo rm /etc/kubernetes/manifests/my-static-pod.yaml
```

Static Pod path configuration:

```
# Find where kubelet looks for static Pods
cat /var/lib/kubelet/config.yaml | grep staticPodPath
# staticPodPath: /etc/kubernetes/manifests
```

5.9 Pod Scheduling Controls

nodeName — Direct Assignment

```
spec:
  nodeName: worker-node-2    # Bypasses scheduler; runs on this node only
```

nodeSelector — Simple Label Matching

```
spec:
  nodeSelector:
    disktype: ssd             # Node must have this label
    kubernetes.io/arch: amd64
```

```
# Label a node
kubectl label node worker-1 disktype=ssd

# Verify label
kubectl get node worker-1 --show-labels
```

Taints and Tolerations

Taints repel Pods from nodes. **Tolerations** allow Pods to be scheduled on tainted nodes.

```
# Taint a node (prevent most Pods from running here)
kubectl taint node gpu-node-1 gpu=true:NoSchedule

# List taints on nodes
kubectl describe node gpu-node-1 | grep Taints

# Remove a taint
kubectl taint node gpu-node-1 gpu=true:NoSchedule-
```

Taint effects:

- `NoSchedule` — New Pods won't be scheduled here (existing Pods unaffected)
- `PreferNoSchedule` — Scheduler avoids this node but may use it if necessary
- `NoExecute` — New Pods won't schedule AND existing Pods without toleration are evicted

```
spec:
  tolerations:
  - key: "gpu"              # Tolerate the "gpu=true:NoSchedule" taint
    operator: "Equal"
    value: "true"
    effect: "NoSchedule"

  - key: "node.kubernetes.io/not-ready"   # Built-in taint for NotReady nodes
    operator: "Exists"
    effect: "NoExecute"
    tolerationSeconds: 300   # Tolerate for 5 minutes before eviction
```

5.10 Pod Security and ServiceAccounts

Every Pod runs with a **ServiceAccount** that provides its API credentials:

```
spec:
  serviceAccountName: my-service-account   # Default: "default" SA in the namespace
  automountServiceAccountToken: false       # Disable if Pod doesn't need API access

  securityContext:          # Pod-level security
    runAsNonRoot: true      # Ensure container doesn't run as root
    runAsUser: 1000         # Run as UID 1000
    runAsGroup: 3000        # Run with GID 3000
    fsGroup: 2000           # Files in volumes owned by GID 2000
    seccompProfile:
      type: RuntimeDefault  # Apply default seccomp profile

  containers:
  - name: app
    image: myapp:latest
    securityContext:        # Container-level security (overrides Pod-level)
      allowPrivilegeEscalation: false
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]       # Drop all Linux capabilities
        add: ["NET_BIND_SERVICE"]  # Re-add only what's needed
```

```
# Create a service account with limited permissions
kubectl create serviceaccount read-pods-sa -n staging

# Bind it to a Role
kubectl create rolebinding read-pods-binding \
  --role=pod-reader \
  --serviceaccount=staging:read-pods-sa \
  -n staging

# Test what the SA can do
kubectl auth can-i list pods --as=system:serviceaccount:staging:read-pods-sa -n staging

# Token is mounted inside the Pod at:
# /var/run/secrets/kubernetes.io/serviceaccount/token
```

5.11 Complete Pod Debugging Workflow

```
Pod Problem → kubectl get pods → identify status
          ↓
  Pending    → kubectl describe pod → check Events
                                      (resources/taints/PVC/selector)
          ↓
  ContainerCreating → kubectl describe pod → Events
                                              (imagePullError/CNI/volume)
          ↓
  CrashLoopBackOff  → kubectl logs --previous → see crash reason
                  → kubectl describe pod → see exit code
                                            (exit 1=app error, 137=OOM, 139=segfault)
          ↓
  Running 0/1       → kubectl describe pod → readiness probe failing
                  → kubectl logs → is app healthy?
          ↓
  Terminating stuck → kubectl get pod -o yaml → check finalizers
                  → kubectl patch pod <name> -p '{"metadata":{"finalizers":[]}}' \
                                               --type=merge
```

Complete diagnosis example:

```
# Find all non-Running, non-Completed pods
kubectl get pods --all-namespaces | grep -v -E "Running|Completed|Terminating"

# For each problem pod:
kubectl describe pod <n> -n <ns> 2>&1 | tee pod-diagnosis.txt
kubectl logs <n> -n <ns> --previous 2>&1 >> pod-diagnosis.txt
kubectl logs <n> -n <ns> 2>&1 >> pod-diagnosis.txt

# Check the exit code
kubectl get pod <n> -n <ns> -o jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}'
# Exit codes:
# 0 = success (clean exit)
# 1 = general error
# 2 = misuse of shell command
# 126 = command found but not executable
# 127 = command not found
# 128+N = killed by signal N (137=SIGKILL/OOM, 143=SIGTERM)
# 139 = segmentation fault (SIGSEGV)
```

5.12 Deployment Strategies Deep Dive

```
# RollingUpdate (default): gradually replaces old Pods
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1           # Max Pods above desired during update
      maxUnavailable: 0     # Max Pods below desired during update
      # maxSurge=1 maxUnavailable=0: Always have full capacity + 1 new pod
      # maxSurge=0 maxUnavailable=1: Update in-place, briefly -1 pod (saves resources)
      # maxSurge=25% maxUnavailable=25%: Default - update 25% at a time
---
# Recreate: kill all old Pods, then create new ones (brief downtime)
spec:
  strategy:
    type: Recreate
```

```
# Watch a rolling update in progress
kubectl rollout status deployment/web-app
kubectl rollout history deployment/web-app

# Deployment stuck? Check why
kubectl describe deployment web-app | grep -A 20 "Conditions:"
# ProgressDeadlineExceeded: 10 minutes elapsed waiting for deployment
# This means new pods aren't becoming available - check the pods!

# Pause a rolling update (for canary testing)
kubectl rollout pause deployment/web-app
# ... test the partial deployment ...
kubectl rollout resume deployment/web-app

# Get the change-cause annotation for audit trail
kubectl annotate deployment/web-app \
  kubernetes.io/change-cause="Upgraded nginx to 1.25.3 for CVE-2023-4966"
kubectl rollout history deployment/web-app
# Shows the change-cause column
```

Chapter 6 — ConfigMaps and Secrets: Complete Reference

6.1 What Are ConfigMaps?

A **ConfigMap** stores non-sensitive configuration data as key-value pairs. They decouple configuration from the container image, allowing the same image to be deployed in dev, staging, and production with different settings.

ConfigMaps can hold:

- Simple key-value pairs (`DATABASE_HOST=postgres.production.svc`)
- Full configuration file contents (`nginx.conf`, `application.properties`)
- JSON or YAML data

6.2 Creating ConfigMaps

Imperatively from literals:

```
kubectl create configmap app-config \
  --from-literal=DATABASE_HOST=postgres \
  --from-literal=DATABASE_PORT=5432 \
  --from-literal=LOG_LEVEL=info

kubectl get configmap app-config -o yaml
```

Imperatively from a file:

```
# From a single file — key is the filename, value is the file contents
kubectl create configmap nginx-config \
  --from-file=nginx.conf

# From a file with a custom key
kubectl create configmap nginx-config \
  --from-file=my-custom-key=./nginx.conf

# From a directory — all files in the directory become key-value pairs
kubectl create configmap app-configs \
  --from-file=./config-dir/
```

Declaratively with YAML:

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: production
data:
  # Simple key-value pairs
  DATABASE_HOST: "postgres.production.svc.cluster.local"
  DATABASE_PORT: "5432"
  LOG_LEVEL: "info"

  # Full file content as a value (note the | for multi-line)
  application.properties: |
    server.port=8080
    spring.datasource.url=jdbc:postgresql://postgres:5432/mydb
    spring.datasource.username=appuser
    logging.level.root=INFO

  nginx.conf: |
    server {
      listen 80;
      server_name _;
      location / {
        proxy_pass http://localhost:8080;
      }
    }
```

6.3 Using ConfigMaps as Environment Variables

Inject specific keys as environment variables:

```yaml
spec:
  containers:
  - name: app
    image: my-app:v1
    env:
    - name: DB_HOST                      # Name of the env var in the container
      valueFrom:
        configMapKeyRef:
          name: app-config               # Name of the ConfigMap
          key: DATABASE_HOST             # Key within the ConfigMap
    - name: LOG_LEVEL
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: LOG_LEVEL
          optional: true                 # Don't fail if ConfigMap or key doesn't exist
```

Inject ALL keys from a ConfigMap as environment variables (envFrom):

```yaml
spec:
  containers:
  - name: app
    image: my-app:v1
    envFrom:
    - configMapRef:
        name: app-config                 # ALL keys become env vars
      prefix: "APP_"                    # Optional: prefix all env var names
    - configMapRef:
        name: database-config
```

What `envFrom` does: Every key in `app-config` becomes an environment variable with the same name in the container. With `prefix: "APP_"`, the key `DATABASE_HOST` becomes `APP_DATABASE_HOST`. This is convenient but can pollute the environment space.

6.4 Using ConfigMaps as Mounted Files

This approach mounts each ConfigMap key as a file inside the container at the specified path. This is the correct approach for configuration files like nginx.conf, application.properties, etc.

```
spec:
  containers:
  - name: app
    image: my-app:v1
    volumeMounts:
    - name: config-volume
      mountPath: /etc/app/config       # Directory where files will appear
      readOnly: true
    - name: nginx-volume
      mountPath: /etc/nginx/conf.d     # Nginx reads from here
      readOnly: true
  volumes:
  - name: config-volume
    configMap:
      name: app-config                  # Mount ALL keys as files
  - name: nginx-volume
    configMap:
      name: app-config
      items:                            # Mount ONLY specific keys as files
      - key: nginx.conf
        path: default.conf              # The file will be named default.conf
```

What happens inside the container: Each specified key becomes a file. If `app-config` has a key `application.properties`, the file `/etc/app/config/application.properties` will exist and contain the value of that key. The container can read it as a regular file.

6.5 Updating ConfigMaps and Hot Reload

Update a ConfigMap:

```
# Edit directly
kubectl edit configmap app-config

# Patch a specific key
kubectl patch configmap app-config \
  --type merge \
  -p '{"data":{"LOG_LEVEL":"debug"}}'

# Replace entirely from a file
kubectl create configmap app-config \
  --from-file=application.properties \
  --dry-run=client -o yaml | kubectl apply -f -
```

Hot reload behavior:

When a ConfigMap is **mounted as a volume**, Kubernetes automatically propagates updates to the mounted files within approximately 60 seconds (controlled by `kubelet.syncFrequency`). The files are replaced atomically using symlinks.

However, when a ConfigMap is used as **environment variables** (`env:` or `envFrom:`), the container does NOT see updates. The Pod must be restarted to pick up new values.

```
# Force Pod restart to pick up env var changes
kubectl rollout restart deployment my-app
```

6.6 Immutable ConfigMaps

Immutable ConfigMaps cannot be modified once created. This prevents accidental configuration changes and improves performance at scale (the kubelet does not watch immutable resources).

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config-v2
immutable: true                # Cannot be modified, only deleted and recreated
data:
  DATABASE_HOST: "postgres"
  VERSION: "2.0.0"
```

6.7 Secrets

Secrets are similar to ConfigMaps but designed for sensitive data: passwords, API keys, TLS certificates, tokens. Secrets are base64-encoded in etcd (not encrypted by default — encryption at rest requires additional etcd configuration) and can be encrypted using EncryptionConfiguration.

> ⚠️ ***Important:*** *Base64 is* ***encoding****, not encryption. Anyone with read access to the Secret object can decode it instantly. Use RBAC to restrict Secret access.*

6.8 Creating Secrets

Imperatively from literals (Kubernetes base64-encodes automatically):

```
kubectl create secret generic db-credentials \
  --from-literal=username=admin \
  --from-literal=password='Sup3rS3cr3t!'

# Verify (shows base64-encoded values)
kubectl get secret db-credentials -o yaml
```

From files:

```
kubectl create secret generic tls-certs \
  --from-file=tls.crt=./server.crt \
  --from-file=tls.key=./server.key

# Create a TLS secret (special type for Ingress/Gateway TLS)
kubectl create secret tls my-tls-secret \
  --cert=./server.crt \
  --key=./server.key
```

Declaratively (you must base64-encode values yourself):

```
# Encode values first
echo -n 'admin' | base64          # Output: YWRtaW4=
echo -n 'Sup3rS3cr3t!' | base64   # Output: U3VwM3JTM2NyM3Qh
```

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
  namespace: production
type: Opaque                        # Generic secret type
data:
  username: YWRtaW4=               # base64("admin")
  password: U3VwM3JTM2NyM3Qh       # base64("Sup3rS3cr3t!")
```

Secret Types:

Type	Use case
`Opaque`	Generic arbitrary data (default)
`kubernetes.io/tls`	TLS certificate and key pairs
`kubernetes.io/dockerconfigjson`	Docker registry credentials
`kubernetes.io/service-account-token`	ServiceAccount tokens
`kubernetes.io/ssh-auth`	SSH credentials
`kubernetes.io/basic-auth`	HTTP basic authentication

6.9 Using Secrets as Environment Variables

```
spec:
  containers:
  - name: app
    env:
    - name: DB_USERNAME
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: username
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: password
    envFrom:
    - secretRef:
        name: db-credentials       # All keys become env vars (decoded automatically)
```

6.10 Using Secrets as Mounted Files

```
spec:
  containers:
  - name: app
    volumeMounts:
    - name: secret-volume
      mountPath: /etc/secrets      # Files appear here — decoded from base64
      readOnly: true
  volumes:
  - name: secret-volume
    secret:
      secretName: db-credentials
      defaultMode: 0400            # File permissions: read-only by owner
      items:
      - key: password
        path: db-password.txt      # Accessible as /etc/secrets/db-password.txt
```

6.11 Decoding a Secret for Inspection

```
# View raw base64 value
kubectl get secret db-credentials -o jsonpath='{.data.password}'
# Output: U3VwM3JTM2NyM3Qh

# Decode it
kubectl get secret db-credentials \
  -o jsonpath='{.data.password}' | base64 --decode
# Output: Sup3rS3cr3t!

# View all keys and their decoded values (useful for debugging)
kubectl get secret db-credentials -o json | \
  python3 -c "import json,sys,base64; \
  d=json.load(sys.stdin)['data']; \
  [print(f'{k}: {base64.b64decode(v).decode()}') for k,v in d.items()]"
```

6.12 Encryption at Rest for Secrets

By default, Secrets in etcd are only base64-encoded. To enable true encryption at rest, configure the API server with an `EncryptionConfiguration`:

```
# /etc/kubernetes/enc/encryption-config.yaml
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
- resources:
  - secrets
  providers:
  - aescbc:
      keys:
      - name: key1
        secret: <base64-encoded-32-byte-key>   # openssl rand -base64 32
  - identity: {}                               # Fallback for reading existing unencrypted secrets
```

Add to the API server manifest (`/etc/kubernetes/manifests/kube-apiserver.yaml`):

```
spec:
  containers:
  - command:
    - kube-apiserver
    - --encryption-provider-config=/etc/kubernetes/enc/encryption-config.yaml
    # ...
    volumeMounts:
    - name: enc-config
      mountPath: /etc/kubernetes/enc
      readOnly: true
  volumes:
  - name: enc-config
    hostPath:
      path: /etc/kubernetes/enc
```

Encrypt existing secrets after enabling:

```
# Re-write all existing secrets through the API to encrypt them
kubectl get secrets -A -o json | kubectl replace -f -
```

6.13 Troubleshooting ConfigMaps and Secrets

Error: ConfigMap key not found

```
$ kubectl logs my-pod
Error: env variable DATABASE_HOST is empty

# Check if ConfigMap exists
kubectl get configmap app-config -n production
# Error: configmaps "app-config" not found
# Fix: deploy the ConfigMap first, or check namespace

# Verify specific key exists in ConfigMap
kubectl get configmap app-config -o jsonpath='{.data.DATABASE_HOST}'
# If empty: the key name is wrong in your Pod spec or ConfigMap

# Check exact key names in ConfigMap
kubectl get configmap app-config -o yaml | grep -A 20 "^data:"
```

Error: Secret not mounting

```
$ kubectl describe pod my-pod
Events:
  Warning  Failed    1m   kubelet  Error: secret "db-credentials" not found

# Secret in wrong namespace
kubectl get secret db-credentials  # default namespace
kubectl get secret db-credentials -n production  # check production namespace
# Secrets are namespace-scoped! Must be in same namespace as Pod

# Decode and verify secret content
kubectl get secret db-credentials -o jsonpath='{.data.password}' | base64 -d
```

Error: Volume mount shows empty directory

```
# Pod mounts ConfigMap as volume but files are empty
# Verify the ConfigMap key names match the subPath or path expectations
kubectl describe pod my-pod | grep -A 20 "Volumes:"
kubectl get configmap nginx-config -o yaml

# Issue: key in ConfigMap is "nginx.conf" but subPath is "nginx.config"
# They must match exactly

# Debug by listing what's in the mounted directory
kubectl exec my-pod -- ls -la /etc/nginx/
kubectl exec my-pod -- cat /etc/nginx/nginx.conf
```

Error: Secret changes not reflected in Pod

```
# Secrets mounted as volumes update automatically (eventually - up to 2 minutes)
# But environment variables from secrets NEVER update (require Pod restart)

# Force update for env var secrets:
kubectl rollout restart deployment/my-app

# For volume-mounted secrets, check if the file changed:
kubectl exec my-pod -- cat /run/secrets/token | head -1

# If Secret was updated more than 2 minutes ago and volume still shows old value:
kubectl describe pod my-pod | grep "Volumes:" -A 20
# Check syncPeriod in kubelet config:
sudo cat /var/lib/kubelet/config.yaml | grep -i sync
```

Practical: Debug missing ConfigMap in a running Pod

```
# See what environment variables the pod actually has
kubectl exec my-pod -- env | sort | grep -i database

# See what files are mounted
kubectl exec my-pod -- find / -path /proc -prune -o -name "*.conf" -print 2>/dev/null

# Check which ConfigMaps and Secrets are referenced
kubectl get pod my-pod -o jsonpath='{.spec.volumes[*].configMap.name}'
kubectl get pod my-pod -o jsonpath='{.spec.containers[0].envFrom[*].configMapRef.name}'

# Full dump of all env sources
kubectl get pod my-pod -o json | python3 -c "
import sys, json
pod = json.load(sys.stdin)
for c in pod['spec']['containers']:
    print(f'Container: {c[\"name\"]}')
    for ef in c.get('envFrom', []):
        if 'configMapRef' in ef:
            print(f'  envFrom ConfigMap: {ef[\"configMapRef\"][\"name\"]}')
        if 'secretRef' in ef:
            print(f'  envFrom Secret: {ef[\"secretRef\"][\"name\"]}')
    for e in c.get('env', []):
        if 'valueFrom' in e:
            src = e['valueFrom']
            if 'configMapKeyRef' in src:
                print(f'  env {e[\"name\"]} → ConfigMap {src[\"configMapKeyRef\"][\"name\"]}:
{src[\"configMapKeyRef\"][\"key\"]}')
            if 'secretKeyRef' in src:
                print(f'  env {e[\"name\"]} → Secret {src[\"secretKeyRef\"][\"name\"]}:
{src[\"secretKeyRef\"][\"key\"]}')
"
```

6.14 Secret Security Best Practices

```
# Secret data is only base64-encoded, NOT encrypted by default
# Enable encryption at rest (configure kube-apiserver):
cat << 'EOF' > /etc/kubernetes/encryption-config.yaml
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
  - resources:
      - secrets
    providers:
      - aescbc:
          keys:
            - name: key1
              # Generate: head -c 32 /dev/urandom | base64
              secret: $(head -c 32 /dev/urandom | base64)
      - identity: {}
EOF

# Add to kube-apiserver static pod manifest:
# --encryption-provider-config=/etc/kubernetes/encryption-config.yaml

# After enabling encryption, re-encrypt all existing secrets:
kubectl get secrets --all-namespaces -o json | kubectl replace -f -

# Verify encryption (secret should not be readable in etcd):
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  get /registry/secrets/default/my-secret | hexdump -C | head
# If encrypted: should show binary data, not readable text
# If plaintext: starts with "k8s:" followed by readable JSON
```

6.15 External Secret Management (Production Patterns)

For production, use an external secret store:

```
# Using External Secrets Operator with AWS Secrets Manager
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-password
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: db-credentials      # Creates this Kubernetes Secret
    creationPolicy: Owner
  data:
  - secretKey: password       # Key in the K8s Secret
    remoteRef:
      key: prod/database/credentials  # Path in AWS Secrets Manager
      property: password              # JSON key in the secret
```

```
# Install External Secrets Operator
helm install external-secrets \
  external-secrets/external-secrets \
  -n external-secrets \
  --create-namespace

# Verify the ExternalSecret syncs successfully
kubectl get externalsecret db-password
# NAME          STORE         REFRESH INTERVAL   STATUS   READY
# db-password   aws-secrets   1h                 True     True
```

Chapter 7 — StorageClasses and Dynamic Volume Provisioning

7.1 The Storage Lifecycle in Kubernetes

Kubernetes storage follows a clear lifecycle:

```
[StorageClass]  — defines HOW to provision storage (which CSI driver, what parameters)
      ↓
[PersistentVolumeClaim (PVC)] — user requests storage of a certain size and access mode
      ↓
[Dynamic Provisioning] — StorageClass + CSI driver automatically creates the volume
      ↓
[PersistentVolume (PV)] — the actual storage object representing the provisioned volume
      ↓
[Pod mounts the PVC] — the volume appears as a filesystem path inside the container
```

Without a StorageClass, PVCs must be manually bound to pre-created PVs (static provisioning). Dynamic provisioning with StorageClasses is the production standard.

7.2 Creating a StorageClass

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-ssd
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"   # Make this the default SC
provisioner: ebs.csi.aws.com                                # The CSI driver to use
parameters:
  type: gp3                                                 # AWS EBS volume type
  iops: "3000"
  throughput: "125"
  encrypted: "true"
reclaimPolicy: Delete          # What happens to the PV when the PVC is deleted
volumeBindingMode: WaitForFirstConsumer   # Delay provisioning until a Pod is scheduled
allowVolumeExpansion: true     # Allow PVCs to be resized after creation
```

Reclaim Policies explained:

Policy	Behavior when PVC is deleted
`Delete`	PV and the underlying storage volume are deleted. **Default for dynamic provisioning.**
`Retain`	PV is kept but marked Released. Manual cleanup required. Data is preserved.
`Recycle`	Deprecated. Data is scrubbed and PV made available again.

Volume Binding Modes:

Mode	When volume is provisioned
`Immediate`	As soon as the PVC is created (default)
`WaitForFirst-Consumer`	Not until a Pod that uses this PVC is scheduled. Allows the CSI driver to provision storage in the same availability zone as the Pod's node.

```
kubectl apply -f storageclass.yaml

# List all StorageClasses
kubectl get storageclass
# NAME                    PROVISIONER             RECLAIMPOLICY   VOLUMEBINDINGMODE
# fast-ssd (default)      ebs.csi.aws.com         Delete          WaitForFirstConsumer
# standard                kubernetes.io/no-provisioner  Delete   Immediate

# Describe a StorageClass
kubectl describe storageclass fast-ssd
```

7.3 Static Provisioning — Creating PVs Manually

Use static provisioning when you have existing storage that you want to expose to Kubernetes.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: manual-pv-100gi
spec:
  capacity:
    storage: 100Gi
  volumeMode: Filesystem
  accessModes:
  - ReadWriteOnce              # Only one node can mount for read-write
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual     # Matches the PVC's storageClassName
  hostPath:
    path: /data/myapp          # Only for single-node or dev clusters!
```

Access Modes — which nodes and how many can mount the volume:

Mode	Abbreviation	Meaning
`ReadWriteOnce`	RWO	Read-write by a single node
`ReadOnlyMany`	ROX	Read-only by many nodes simultaneously
`ReadWriteMany`	RWX	Read-write by many nodes simultaneously (requires NFS/Ceph/etc.)
`ReadWriteOncePod`	RWOP	Read-write by a single Pod (most restrictive, Kubernetes 1.22+)

```
kubectl apply -f pv.yaml
kubectl get pv
# NAME                CAPACITY   ACCESS MODES   RECLAIM POLICY   STATUS      STORAGECLASS
# manual-pv-100gi     100Gi      RWO            Retain           Available   manual
```

7.4 Creating PersistentVolumeClaims

A PVC is a request for storage. The user specifies size, access mode, and optionally a StorageClass. Kubernetes binds the PVC to an appropriate PV.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: app-data-pvc
  namespace: production
spec:
  accessModes:
  - ReadWriteOnce
  storageClassName: fast-ssd    # Omit or use "" to use the default StorageClass
  resources:
    requests:
      storage: 20Gi
```

```
kubectl apply -f pvc.yaml

# Watch the PVC status — it goes Pending → Bound
kubectl get pvc -w
# NAME            STATUS    VOLUME                   CAPACITY   ACCESS MODES
# app-data-pvc    Pending                                                    ← WaitForFirstConsumer
# app-data-pvc    Bound     pvc-a1b2c3d4-...          20Gi       RWO            ← after Pod scheduled
```

7.5 Mounting a PVC in a Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: app-with-storage
spec:
  containers:
  - name: app
    image: nginx
    volumeMounts:
    - name: data-volume
      mountPath: /usr/share/nginx/html    # Where the volume appears inside the container
      readOnly: false
  volumes:
  - name: data-volume
    persistentVolumeClaim:
      claimName: app-data-pvc             # References the PVC by name
```

7.6 Resizing a PVC (Volume Expansion)

If the StorageClass has `allowVolumeExpansion: true`, you can expand a PVC online:

```
# Edit the PVC to request more storage
kubectl patch pvc app-data-pvc \
  -p '{"spec":{"resources":{"requests":{"storage":"50Gi"}}}}'

# Or edit directly
kubectl edit pvc app-data-pvc
# Change storage: 20Gi → storage: 50Gi

# Monitor expansion status
kubectl describe pvc app-data-pvc | grep -A 5 Conditions
# Type: FileSystemResizePending — the volume is being resized
# Type: Resizing → eventually disappears when done
```

> ⚠️ ***Important:*** *You can only* ***increase*** *PVC size. Shrinking a PVC is not supported and will be rejected by the API server.*

7.7 Troubleshooting Storage Issues

PVC stuck in Pending state:

```
kubectl describe pvc app-data-pvc
# Look at the Events section:
# "no persistent volumes available for this claim and no storage class is set"
#   → The StorageClass doesn't exist or is misspelled
# "waiting for a volume to be created, either by external provisioner..."
#   → WaitForFirstConsumer — normal until Pod is scheduled
# "storageclass.storage.k8s.io not found"
#   → The specified StorageClass does not exist

# List storage classes
kubectl get sc

# Check if the CSI driver is running
kubectl get pods -n kube-system | grep csi
```

PV and PVC in Released/Available but not Binding:

```
# A PV in "Released" state has claimRef from the previous PVC
# Clear the claimRef to make it available again
kubectl patch pv <pv-name> \
  -p '{"spec":{"claimRef": null}}'
```

7.8 Complete PV/PVC Troubleshooting Guide

Understanding the Binding Process

```
PVC Created → PV found matching storage class/capacity/access modes
           → PV.claimRef points to this PVC
           → Both enter Bound state
```

Symptom: PVC in Pending

```
kubectl describe pvc my-pvc
# Look at Events section

# Case 1: No matching PV exists (static provisioning)
# Error: no persistent volumes available for this claim
kubectl get pv  # List available PVs
kubectl describe pv my-pv | grep -E "Access|Capacity|StorageClass"
# Must match: AccessModes, capacity (PV >= PVC request), storageClassName

# Case 2: Dynamic provisioning not working
# Error: waiting for a volume to be created, either by external provisioner
#        "kubernetes.io/no-provisioner" or manually created
# The StorageClass provisioner is "no-provisioner" - create PV manually:
cat << 'EOF' | kubectl apply -f -
apiVersion: v1
kind: PersistentVolume
metadata:
  name: manual-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
  - ReadWriteOnce
  storageClassName: local-storage
  hostPath:
    path: /data/volumes/pv1
EOF

# Case 3: StorageClass doesn't exist
kubectl get storageclass  # Check available classes
# Error: storageclass.storage.k8s.io "fast-ssd" not found
kubectl get pvc -o yaml | grep storageClassName  # Check what PVC requests
```

Symptom: PVC bound but Pod can't mount

```
$ kubectl describe pod my-pod
Events:
  Warning  FailedMount  2m  kubelet
    MountVolume.SetUp failed for volume "data" :
    mount /var/lib/kubelet/pods/.../volumes/kubernetes.io~csi/... failed:
    rpc error: code = Internal desc = NodePublishVolume failed:
    error NodePublishVolume for volume failed: error mounting "":
    exit status 32

# Common causes:
# 1. PVC is in different namespace from Pod (PVCs are namespace-scoped)
kubectl get pvc -A | grep my-pvc  # Check which namespace

# 2. Node doesn't have the CSI driver
kubectl get pods -n kube-system | grep csi-node
kubectl describe node $(kubectl get pod my-pod -o jsonpath='{.spec.nodeName}') | grep csi

# 3. PV/PVC ReadWriteOnce already bound to another node
kubectl get pv data-pv -o jsonpath='{.spec.claimRef.name}'
kubectl describe pv data-pv | grep -E "Access|Node"
# RWO (ReadWriteOnce) can only be mounted on ONE node at a time

# Fix RWO conflict: scale down old deployment, wait for volume detach, then scale up new
kubectl scale deployment old-app --replicas=0
kubectl get volumeattachments  # Wait until attachment is gone
kubectl scale deployment new-app --replicas=1
```

Symptom: Data lost after Pod restart

```
# Pod was using emptyDir (lost on restart) instead of PVC (persistent)
kubectl get pod my-pod -o jsonpath='{.spec.volumes[*].emptyDir}'

# Check what type of volume each volume is:
kubectl get pod my-pod -o json | python3 -c "
import sys, json
pod = json.load(sys.stdin)
for v in pod['spec']['volumes']:
    vtype = list(v.keys() - {'name'})[0] if len(v) > 1 else 'unknown'
    print(f'{v[\"name\"]}: {vtype}')
"

# emptyDir data is lost when Pod is deleted or rescheduled
# PVC data persists even when Pod is deleted (as long as reclaimPolicy is Retain)
```

PersistentVolume Reclaim Policies

```
# Check reclaim policy
kubectl get pv -o custom-columns='NAME:.metadata.name,POLICY:.spec.persistentVolumeReclaimPolicy,STATUS:
.status.phase'

# ReclaimPolicy = Retain: After PVC is deleted, PV goes to "Released" state
# The data still exists but the PV cannot be rebound to a new PVC automatically
# To reuse a Released PV:
kubectl patch pv my-pv -p '{"spec":{"claimRef": null}}'  # Clear claimRef
kubectl get pv my-pv  # Now shows Available
# Create new PVC referencing this PV by storageClass/volumeName

# ReclaimPolicy = Delete: PV and underlying storage are deleted when PVC is deleted
# Be very careful with this in production!

# Change reclaim policy:
kubectl patch pv my-pv -p '{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}'
```

7.9 Volume Types Reference with Examples

```
# emptyDir: temporary, shared between containers, deleted with Pod
volumes:
- name: cache
  emptyDir: {}
  # Or with size limit:
  # emptyDir:
  #   sizeLimit: 500Mi
  #   medium: Memory  # RAM-backed tmpfs

# hostPath: mounts a node path (use carefully - ties Pod to specific node)
- name: host-logs
  hostPath:
    path: /var/log/app
    type: DirectoryOrCreate  # Creates directory if not exists
    # Other types: Directory, File, FileOrCreate, Socket, BlockDevice

# configMap volume
- name: app-config
  configMap:
    name: my-config
    defaultMode: 0644  # File permissions
    items:             # Optional: select specific keys
    - key: nginx.conf
      path: nginx.conf
      mode: 0600       # Per-file mode override

# secret volume
- name: tls-certs
  secret:
    secretName: tls-secret
    defaultMode: 0400  # Secrets should be read-only

# projected: combine multiple sources into one volume
- name: combined
  projected:
    sources:
    - configMap:
        name: app-config
    - secret:
        name: app-secret
    - serviceAccountToken:
        path: token
        expirationSeconds: 3600
    - downwardAPI:
        items:
        - path: "pod-name"
          fieldRef:
            fieldPath: metadata.name

# nfs: network file system (no CSI needed)
- name: nfs-data
  nfs:
    server: 10.0.0.100
    path: /exports/data
    readOnly: false
```

7.10 StatefulSet Storage Patterns

StatefulSets use `volumeClaimTemplates` — each Pod gets its own PVC:

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres
  replicas: 3
  selector:
    matchLabels:
      app: postgres
  template:
    spec:
      containers:
      - name: postgres
        image: postgres:15
        env:
        - name: PGDATA
          value: /var/lib/postgresql/data/pgdata
        volumeMounts:
        - name: postgres-data
          mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:           # Each pod gets its own PVC
  - metadata:
      name: postgres-data
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: fast-ssd
      resources:
        requests:
          storage: 10Gi
```

```
# StatefulSet PVC naming convention: <template-name>-<statefulset-name>-<ordinal>
kubectl get pvc -l app=postgres
# NAME                     STATUS   VOLUME
# postgres-data-postgres-0  Bound    pvc-abc123
# postgres-data-postgres-1  Bound    pvc-def456
# postgres-data-postgres-2  Bound    pvc-ghi789

# PVCs from StatefulSets are NOT deleted when StatefulSet is deleted!
# This protects your data. Delete manually when sure:
kubectl delete pvc postgres-data-postgres-0
```

Chapter 8 — Workload Autoscaling with HPA and VPA

8.1 Horizontal Pod Autoscaler (HPA)

The **Horizontal Pod Autoscaler** automatically scales the number of Pod replicas in a Deployment, StatefulSet, or ReplicaSet based on observed metrics (CPU, memory, or custom metrics). It is one of the key topics under the Workloads & Scheduling domain.

HPA requires the **Metrics Server** to be installed in the cluster to read CPU/memory data.

Install Metrics Server (if not present):

```
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/
components.yaml

# Verify it is running
kubectl get deployment metrics-server -n kube-system
kubectl top nodes    # Should work once metrics-server is ready
```

8.2 Creating an HPA Imperatively

The fastest way for the CKA exam:

```
# Autoscale a Deployment between 2 and 10 replicas, targeting 50% CPU utilization
kubectl autoscale deployment my-app \
  --min=2 \
  --max=10 \
  --cpu-percent=50
```

What this does: Creates an HPA resource that watches the `my-app` Deployment. If average CPU across all Pods exceeds 50% of their CPU requests, HPA adds more replicas (up to 10). If CPU drops well below 50%, HPA removes replicas (down to 2). Scale-down has a 5-minute stabilization window by default to prevent flapping.

Check the HPA status:

```
kubectl get hpa
# NAME     REFERENCE           TARGETS   MINPODS   MAXPODS   REPLICAS   AGE
# my-app   Deployment/my-app   23%/50%   2         10        3          5m

kubectl describe hpa my-app
# Shows events like: Successful rescaling from 3 to 5 replicas
```

8.3 Creating an HPA with YAML

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: my-app-hpa
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app
  minReplicas: 2
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50          # Target: 50% of CPU requests
  - type: Resource
    resource:
      name: memory
      target:
        type: AverageValue
        averageValue: 256Mi             # Target: 256Mi average memory per Pod
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 300   # Wait 5 min before scaling down
      policies:
      - type: Percent
        value: 10                       # Remove at most 10% of replicas per period
        periodSeconds: 60
    scaleUp:
      stabilizationWindowSeconds: 0     # Scale up immediately
      policies:
      - type: Percent
        value: 100                      # Double replicas per period if needed
        periodSeconds: 30
```

Key fields explained:

- `scaleTargetRef` — which Deployment/StatefulSet to scale
- `metrics` — what to measure. `Utilization` is percentage of requests; `AverageValue` is absolute
- `behavior` — controls the rate of scaling up and down, preventing sudden spikes and thrashing

Critical prerequisite — the Deployment MUST have resource requests defined:

```yaml
resources:
  requests:
    cpu: "200m"        # HPA calculates percentage against this value
    memory: "256Mi"
```

Without `resources.requests.cpu`, HPA cannot calculate utilization percentage and will report `<unknown>` targets, preventing scaling.

8.4 Simulating Load to Test HPA

```
# In one terminal: watch HPA status
kubectl get hpa my-app -w

# In another terminal: generate CPU load
kubectl run load-gen \
  --image=busybox \
  --rm -it \
  --restart=Never \
  -- sh -c "while true; do wget -q -O- http://my-app-service/; done"
```

What this does: Creates an infinite loop of HTTP requests, driving up CPU usage in the target Pods. Watch the HPA terminal — you should see TARGETS rise above 50% and REPLICAS increase. Press Ctrl+C to stop the load generator. After ~5 minutes, HPA scales back down.

8.5 Deleting an HPA

```
# Delete HPA (Deployment keeps its current replica count)
kubectl delete hpa my-app-hpa

# Check the Deployment still has the scaled-up replicas
kubectl get deployment my-app
# Scale it back manually if needed
kubectl scale deployment my-app --replicas=2
```

8.6 Advanced HPA Configuration

Resource-Based HPA

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: web-app-hpa
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  minReplicas: 2
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 70    # Target 70% average CPU utilization
  - type: Resource
    resource:
      name: memory
      target:
        type: AverageValue
        averageValue: 400Mi       # Target 400MiB average memory per Pod
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60     # Wait 60s before scale-up decision
      policies:
      - type: Percent
        value: 100                        # Can double replicas per period
        periodSeconds: 60
      - type: Pods
        value: 5                          # Or add 5 pods per period (whichever is higher)
        periodSeconds: 60
      selectPolicy: Max                   # Use the policy that adds the most Pods
    scaleDown:
      stabilizationWindowSeconds: 300    # Wait 5 minutes before scale-down
      policies:
      - type: Percent
        value: 10                         # Remove at most 10% of replicas per period
        periodSeconds: 60
```

External and Custom Metrics HPA

HPA can also scale on custom metrics from monitoring systems:

```
spec:
  metrics:
  # External metric: queue depth from an external system (e.g., RabbitMQ, SQS)
  - type: External
    external:
      metric:
        name: rabbitmq_queue_messages
        selector:
          matchLabels:
            queue: orders
      target:
        type: AverageValue
        averageValue: 100          # Scale when > 100 messages per Pod

  # Custom metric: requests per second from Prometheus via custom metrics API
  - type: Pods
    pods:
      metric:
        name: http_requests_per_second
      target:
        type: AverageValue
        averageValue: 1000         # Scale when > 1000 req/s per Pod

  # Object metric: tied to another Kubernetes object (like an Ingress)
  - type: Object
    object:
      metric:
        name: requests-per-second
      describedObject:
        apiVersion: networking.k8s.io/v1
        kind: Ingress
        name: main-ingress
      target:
        type: Value
        value: 10000               # Scale when total ingress req/s > 10000
```

Monitoring HPA Status

```
# Create HPA imperatively (quick exam method)
kubectl autoscale deployment web-app --cpu-percent=70 --min=2 --max=20

# List HPAs
kubectl get hpa -n production

# Detailed HPA status
kubectl describe hpa web-app -n production
# Output shows:
#   Metrics: cpu resource utilization (percentage of request): 65% / 70%
#   Min replicas: 2, Max replicas: 20
#   Current replicas: 4, Desired replicas: 4

# Watch HPA scaling decisions in real time
kubectl get hpa web-app -n production -w

# Watch pods scale up/down
kubectl get pods -l app=web-app -n production -w

# Events show what triggered scaling
kubectl describe hpa web-app | grep -A 10 Events
```

8.7 Vertical Pod Autoscaler (VPA)

While HPA scales the number of replicas horizontally, **VPA** automatically adjusts the CPU and memory requests/limits of individual containers.

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: web-app-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: web-app
  updatePolicy:
    updateMode: "Auto"    # Options: Off, Initial, Recreate, Auto
  resourcePolicy:
    containerPolicies:
    - containerName: web
      minAllowed:
        cpu: 100m
        memory: 128Mi
      maxAllowed:
        cpu: 4000m
        memory: 4Gi
      controlledResources: ["cpu", "memory"]
      controlledValues: RequestsAndLimits   # or RequestsOnly
```

VPA Update Modes:

Mode	Behavior
`Off`	VPA only computes recommendations, never applies them (view only)
`Initial`	Applies recommendations only to new Pods, not running ones
`Recreate`	Evicts and recreates Pods to apply new resource values
`Auto`	Currently same as Recreate; future may enable in-place updates

```
# Install VPA (not installed by default)
git clone https://github.com/kubernetes/autoscaler.git
./autoscaler/vertical-pod-autoscaler/hack/vpa-up.sh

# View VPA recommendations
kubectl get vpa web-app-vpa -o yaml | grep -A 20 "recommendation:"
# Shows lowerBound, target, upperBound for CPU and memory

# In Off mode, just see what VPA would recommend
kubectl describe vpa web-app-vpa | grep -A 10 "Recommendation:"
```

HPA + VPA Conflict Warning: You cannot use both HPA (on CPU/memory) and VPA (on CPU/memory)
for the same Deployment — they will fight each other. Use either:

- HPA on CPU + VPA on memory only (with `controlledResources: ["memory"]`)
- HPA on custom metrics + VPA on CPU/memory (VPA manages sizes, HPA manages count)

8.8 KEDA: Kubernetes Event-Driven Autoscaling

For more advanced scaling scenarios beyond CPU/memory, **KEDA** is the industry standard:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata:
  name: rabbitmq-scaledobject
spec:
  scaleTargetRef:
    name: order-processor
  pollingInterval: 15          # Check metrics every 15 seconds
  cooldownPeriod: 300          # Wait 5 minutes before scaling to zero
  minReplicaCount: 0           # Scale to zero when queue is empty
  maxReplicaCount: 50
  triggers:
  - type: rabbitmq
    metadata:
      protocol: amqp
      queueName: orders
      mode: QueueLength
      value: "10"              # 1 Pod per 10 messages in queue
```

KEDA supports 60+ trigger types: Kafka, RabbitMQ, AWS SQS, Azure Service Bus, Prometheus, Cron schedules, and more.

8.9 Troubleshooting HPA

Error: HPA unable to calculate replica count

```
$ kubectl describe hpa web-app
Events:
  Warning  FailedGetScale  1m  horizontal-pod-autoscaler
    failed to get cpu utilization: unable to get metrics for resource cpu:
    no metrics returned from resource metrics API

# Cause: Metrics Server not installed
kubectl get apiservice v1beta1.metrics.k8s.io
# NAME                        AVAILABLE   AGE
# v1beta1.metrics.k8s.io      False        5m  ← Not available!

# Install Metrics Server:
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/
components.yaml

# Verify:
kubectl top pods  # Should work within 2 minutes
```

Error: HPA targets show `<unknown>/70%`

```
$ kubectl get hpa
NAME      REFERENCE            TARGETS         MINPODS   MAXPODS   REPLICAS
web-app   Deployment/web-app   <unknown>/70%   2         20        2

# Cause: Target deployment doesn't have CPU requests defined
kubectl get deployment web-app -o jsonpath='{.spec.template.spec.containers[0].resources}'
# {} ← empty! HPA can't calculate utilization without a request baseline

# Fix: Add CPU requests to the deployment
kubectl set resources deployment web-app \
  --containers=app --requests=cpu=100m

# Wait ~60 seconds for HPA to recalculate
kubectl get hpa web-app -w
```

HPA not scaling up when expected

```
# HPA won't scale up due to stabilization window (default 0s for scale-up)
# Check current state:
kubectl describe hpa web-app | grep -A 20 "Conditions:"
# Conditions:
#   Type            Status  Reason            Message
#   AbleToScale     True    ReadyForNewScale  recommended size matches current size
#   ScalingActive   True    ValidMetricFound  the HPA was able to successfully calculate...
#   ScalingLimited  True    TooManyReplicas   the desired replica count is less than minimum

# AbleToScale=False with reason BackoffBoth:
# HPA is in backoff period after recent scale event
# Default scaleDown stabilizationWindowSeconds: 300 (5 min)

# Check if maxReplicas is too low:
kubectl get hpa web-app -o jsonpath='{.spec.maxReplicas}'
kubectl get hpa web-app -o jsonpath='{.status.currentReplicas}'
# If currentReplicas == maxReplicas → ScalingLimited:TooManyReplicas
# Fix: increase maxReplicas

# Manually force scale to verify HPA takes over:
kubectl scale deployment web-app --replicas=1
# HPA should scale it back up within 30s
```

Chapter 9 — Self-Healing Primitives: Probes and PodDisruptionBudgets

9.1 Container Probes

Kubernetes provides three types of **probes** — health-check mechanisms that the kubelet uses to determine the state of a container. Probes are one of the most important topics for the CKA exam under Workloads & Scheduling.

Probe Type	What It Does	Action on Failure
Liveness	Is the container healthy/alive?	Kill and restart the container
Readiness	Is the container ready to receive traffic?	Remove from Service endpoints (no restart)
Startup	Has the container finished starting?	Kill and restart if not ready within threshold

9.2 Probe Mechanisms

All three probe types support four mechanisms:

HTTP GET — most common for web services:

```
livenessProbe:
  httpGet:
    path: /healthz           # The HTTP endpoint to call
    port: 8080               # The container port to use
    httpHeaders:
    - name: Custom-Header
      value: Awesome
  initialDelaySeconds: 10    # Wait 10s after container starts before first probe
  periodSeconds: 15          # Check every 15 seconds
  failureThreshold: 3        # Fail 3 consecutive times before taking action
  successThreshold: 1        # 1 success to consider healthy (liveness only allows 1)
  timeoutSeconds: 5          # Each probe must respond within 5 seconds
```

TCP Socket — for non-HTTP services (databases, queues):

```
livenessProbe:
  tcpSocket:
    port: 5432               # Attempt TCP connection to this port
  initialDelaySeconds: 15
  periodSeconds: 20
```

Exec Command — runs a command inside the container:

```
livenessProbe:
  exec:
    command:
    - /bin/sh
    - -c
    - "pg_isready -U postgres"    # Returns 0 if healthy, non-zero if not
  initialDelaySeconds: 30
  periodSeconds: 10
```

gRPC — for gRPC services (Kubernetes 1.24+):

```
livenessProbe:
  grpc:
    port: 9090
    service: "my.grpc.HealthService"
```

9.3 Complete Pod Spec with All Three Probes

```
apiVersion: v1
kind: Pod
metadata:
  name: robust-app
spec:
  containers:
  - name: app
    image: my-app:v2
    ports:
    - containerPort: 8080
    resources:
      requests:
        cpu: "200m"
        memory: "256Mi"
      limits:
        cpu: "500m"
        memory: "512Mi"

    # Startup probe — gives slow-starting apps time to initialize
    # Checked first. Liveness/readiness probes are DISABLED until startup succeeds
    startupProbe:
      httpGet:
        path: /healthz
        port: 8080
      failureThreshold: 30          # 30 failures × 10s period = 300s (5 min) max startup time
      periodSeconds: 10

    # Liveness probe — if this fails, container is killed and restarted
    livenessProbe:
      httpGet:
        path: /healthz
        port: 8080
      initialDelaySeconds: 0        # Startup probe already handled the delay
      periodSeconds: 15
      failureThreshold: 3           # 3 failures = restart (45 seconds of unresponsiveness)
      timeoutSeconds: 5

    # Readiness probe — if this fails, Pod is removed from Service endpoints
    # The container keeps running; it just stops receiving traffic
    readinessProbe:
      httpGet:
        path: /ready                # Different endpoint — checks DB connections, caches, etc.
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 10
      failureThreshold: 3
      successThreshold: 1
```

Design insight: The `/healthz` and `/ready` endpoints serve different purposes. `/healthz` is a minimal liveness check — just "is the process running". `/ready` is richer — it might check database connectivity, downstream service availability, and cache warmth. Using the same endpoint

for both liveness and readiness is a common mistake: if your DB is temporarily unavailable, you want the Pod removed from Service rotation (readiness failure) but NOT restarted (liveness should still pass).

9.4 Debugging Probe Failures

```
# See probe failure events
kubectl describe pod robust-app | tail -30
# Events:
#   Warning  Unhealthy  5m   kubelet  Liveness probe failed: HTTP probe failed
#             with statuscode: 503
#   Warning  Killing    5m   kubelet  Container app failed liveness probe,
#             will be restarted

# Check the restart count (increments with each liveness-triggered restart)
kubectl get pod robust-app -o jsonpath='{.status.containerStatuses[0].restartCount}'

# See previous container's logs (what happened before the crash)
kubectl logs robust-app --previous

# Manually test the health endpoint from inside the cluster
kubectl run test --image=curlimages/curl --rm -it --restart=Never -- \
  curl -v http://<pod-ip>:8080/healthz
```

9.5 PodDisruptionBudget (PDB)

A **PodDisruptionBudget** limits how many Pods of a deployment can be simultaneously unavailable during voluntary disruptions — node drains, cluster upgrades, and manual evictions.

Without a PDB, draining a node with all Pods of a Deployment could take all replicas offline simultaneously. A PDB prevents this.

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-app-pdb
  namespace: production
spec:
  # Option 1: Minimum available (at least N pods must be running)
  minAvailable: 2

  # Option 2: Maximum unavailable (at most N pods may be down at once)
  # maxUnavailable: 1

  selector:
    matchLabels:
      app: my-app              # Targets pods with this label
```

```
kubectl apply -f pdb.yaml

kubectl get pdb
# NAME         MIN AVAILABLE   MAX UNAVAILABLE   ALLOWED DISRUPTIONS   AGE
# my-app-pdb   2               N/A               1                     1m

# ALLOWED DISRUPTIONS = current replicas - minAvailable
# If replicas=3 and minAvailable=2, then 1 disruption is allowed at a time
```

Testing the PDB during a node drain:

```
# Try to drain a node — PDB will pace the evictions
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data

# If only 2 replicas exist and minAvailable=2, drain BLOCKS with:
# "Cannot evict pod as it would violate the pod's disruption budget"
```

9.6 Troubleshooting Probe Failures

Symptom: Pod in CrashLoopBackOff due to liveness probe

```
kubectl describe pod my-pod
Events:
  Warning  Unhealthy  5m  kubelet
    Liveness probe failed: Get "http://10.244.1.5:8080/healthz":
    dial tcp 10.244.1.5:8080: connect: connection refused

# Cause 1: Wrong port
# Verify the container is actually listening on the probe port:
kubectl exec my-pod -- ss -tlnp
# or
kubectl exec my-pod -- netstat -tlnp

# Cause 2: App not ready yet - initialDelaySeconds too short
# Check how long app takes to start:
kubectl logs my-pod | head -20  # Look for "server started" message timestamp
# Fix: increase initialDelaySeconds or add a startupProbe

# Cause 3: Health endpoint returns wrong status code
kubectl exec my-pod -- wget -qO- http://localhost:8080/healthz
# If it returns non-2xx, the probe fails
# Check: does /healthz return 200? Some return 204 (also OK) but wrong ones return 500

# Cause 4: Probe timeout too short
# If app is busy, the probe might time out before responding
# Fix: increase timeoutSeconds from default 1s
kubectl describe pod my-pod | grep -A 10 "Liveness:"
```

Symptom: Pod is Running but not receiving traffic

```
$ kubectl get pods
NAME       READY   STATUS    RESTARTS   AGE
my-pod     0/1     Running   0          2m  # 0/1 means not ready!

kubectl describe pod my-pod
  Readiness:  http-get http://:8080/ready delay=0s timeout=1s period=10s #success=1 #failure=3

# Readiness probe is failing - pod is excluded from Service endpoints
kubectl get endpoints my-service
# NAME         ENDPOINTS         AGE
# my-service   <none>            2m  ← empty! Pod not ready

# Debug the readiness endpoint:
kubectl exec my-pod -- wget -qO- -S http://localhost:8080/ready
# HTTP/1.1 503 Service Unavailable  ← App says not ready yet

# Check why app isn't ready (e.g. still connecting to database)
kubectl logs my-pod | grep -i "ready\|database\|connect"

# If app doesn't have a /ready endpoint, use exec probe:
# readinessProbe:
#   exec:
#     command: ["/bin/sh", "-c", "test -f /tmp/ready"]
```

Symptom: Pod keeps restarting with OOMKilled

```
$ kubectl get pods
NAME       READY   STATUS      RESTARTS   AGE
my-pod     0/1     OOMKilled   3          10m

kubectl describe pod my-pod | grep -A 5 "Last State:"
# Last State: Terminated
#   Reason:   OOMKilled       ← Killed by kernel out-of-memory killer
#   Exit Code: 137

# Check memory usage vs limits
kubectl top pod my-pod
kubectl get pod my-pod -o jsonpath='{.spec.containers[0].resources}'

# Fix: increase memory limit
kubectl set resources deployment my-app --limits=memory=512Mi

# Or find memory leak: get metrics over time
kubectl top pod my-pod --no-headers | awk '{print $3}'  # Memory column

# Check if there's a memory trend in events:
kubectl describe pod my-pod | grep -i "oom\|memory\|kill"
```

Liveness vs Readiness vs Startup — Decision Guide

```
Use LIVENESS when:
├── App can deadlock (all threads stuck, but process running)
├── App cache grows unbounded and needs periodic restart
└── App enters error loop and needs restart to recover

Use READINESS when:
├── App needs time to load data/cache before serving requests
├── App depends on external service being available
└── App periodically becomes temporarily unavailable (maintenance mode)

Use STARTUP when:
├── Container needs >30s to start (would be killed by liveness)
├── Initialization time is variable (could be 10s or 10min)
└── You want different probe behavior during startup vs steady state

Example: Use all three together for a Java application:
startupProbe:               # Generous timeout for JVM startup
  httpGet: {path: /health, port: 8080}
  failureThreshold: 30      # 30 attempts × 10s = 5 minutes max startup time
  periodSeconds: 10

livenessProbe:              # Strict - restart if deadlocked
  httpGet: {path: /health, port: 8080}
  initialDelaySeconds: 0    # startupProbe already handled the delay
  periodSeconds: 10
  failureThreshold: 3       # 3 failures = 30s before restart

readinessProbe:             # Remove from endpoints if temporarily busy
  httpGet: {path: /ready, port: 8080}
  periodSeconds: 5
  failureThreshold: 2       # Removed from Service after 10s
  successThreshold: 1       # Back in Service after first success
```

9.7 PodDisruptionBudget Troubleshooting

Symptom: kubectl drain blocked by PDB

```
$ kubectl drain worker-1 --ignore-daemonsets
evicting pod production/api-7d8f9-abc12
error when evicting pods/"api-7d8f9-abc12" -n "production"
  (will retry after 5s):
  Cannot evict pod as it would violate the pod's disruption budget.

# Check the PDB:
kubectl get pdb -n production
kubectl describe pdb api-pdb -n production
# Disruptions Allowed: 0   ← This is the problem!
# Current: 2/2 pods healthy, PDB says minAvailable: 2 → 0 disruptions allowed

# Fix Option 1: Delete the PDB temporarily (if maintenance window)
kubectl delete pdb api-pdb -n production
kubectl drain worker-1 --ignore-daemonsets
# Then recreate PDB after maintenance

# Fix Option 2: Scale up the deployment temporarily
kubectl scale deployment api --replicas=3 -n production
# Now 3/2 > minAvailable(2), so 1 disruption allowed
kubectl drain worker-1 --ignore-daemonsets

# Fix Option 3: Force drain (evicts pods ignoring PDB - data loss risk!)
kubectl drain worker-1 --ignore-daemonsets --disable-eviction --force
# WARNING: Only for emergency/already-down situations
```

Chapter 10 — Custom Resource Definitions and Operators

10.1 Extending the Kubernetes API with CRDs

Kubernetes ships with a fixed set of resource types: Pod, Deployment, Service, ConfigMap, etc. **Custom Resource Definitions (CRDs)** allow you to extend the Kubernetes API with your own resource types, making them first-class citizens — they get their own API endpoints, work with kubectl, support labels/annotations, and can be watched by controllers.

A CRD defines the schema of a new resource type. Once installed, users can create instances of that resource (called **Custom Resources** or **CRs**) just like creating Pods.

Example: A CRD for a `Database` resource

```
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: databases.mycompany.com        # Must be <plural>.<group>
spec:
  group: mycompany.com                 # API group for this resource
  names:
    kind: Database                     # The resource kind (singular, PascalCase)
    listKind: DatabaseList
    plural: databases                  # URL path: /apis/mycompany.com/v1/databases
    singular: database
    shortNames:
    - db                               # kubectl get db works!
  scope: Namespaced                    # Or Cluster for cluster-scoped resources
  versions:
  - name: v1
    served: true                       # This version is active
    storage: true                      # This version is stored in etcd
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            properties:
              engine:
                type: string
                enum: ["postgres", "mysql", "redis"]
              version:
                type: string
              replicas:
                type: integer
                minimum: 1
                maximum: 5
              storageGB:
                type: integer
```

Install the CRD:

```
kubectl apply -f database-crd.yaml

# Verify it was registered
kubectl get crd databases.mycompany.com

# Check it is available in the API
kubectl api-resources | grep databases
# databases    db    mycompany.com/v1    true    Database
```

Create a Custom Resource (an instance of the CRD):

```
# my-database.yaml
apiVersion: mycompany.com/v1
kind: Database
metadata:
  name: production-postgres
  namespace: default
spec:
  engine: postgres
  version: "15.4"
  replicas: 3
  storageGB: 100
```

```
kubectl apply -f my-database.yaml

# Interact with it exactly like built-in resources
kubectl get databases
kubectl get db                          # short name works
kubectl describe database production-postgres
kubectl delete database production-postgres
```

List all CRDs in the cluster:

```
kubectl get crd
kubectl get crd -o wide
```

10.2 Operators: Controllers for Custom Resources

A CRD alone is just a data structure stored in etcd. It does nothing by itself. An **Operator** is a controller that watches for Custom Resources and takes action to make the real-world state match the desired state defined in the CR.

The **Operator Pattern** = CRD (desired state schema) + Controller (reconciliation loop).

Example: What a Database Operator does

When you create the `production-postgres` Database CR, the Database Operator:

1. Sees the new CR (via a Watch on the Kubernetes API)
2. Provisions a StatefulSet with 3 PostgreSQL Pods
3. Creates a PVC for each replica using the specified storageGB
4. Creates a Service for database access
5. Continuously monitors the CR and reconciles if anything drifts

Installing an Operator (example: Prometheus Operator):

```
# Most operators are distributed as Helm charts or YAML bundles
# Method 1: Via Helm
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  -n monitoring --create-namespace

# Method 2: Via raw YAML (common for CKA exam scenarios)
kubectl apply -f https://raw.githubusercontent.com/prometheus-operator/prometheus-operator/main/
bundle.yaml
```

Verify an Operator is running:

```
# Check the operator Deployment
kubectl get deployment -n monitoring prometheus-operator

# Check the CRDs the operator registered
kubectl get crd | grep monitoring.coreos.com
# alertmanagerconfigs.monitoring.coreos.com
# prometheuses.monitoring.coreos.com
# prometheusrules.monitoring.coreos.com
# servicemonitors.monitoring.coreos.com

# Create a ServiceMonitor (a CRD instance)
kubectl get servicemonitors -A
```

Key CRD commands for the CKA exam:

```
# List all CRDs
kubectl get crd

# Get a CRD definition
kubectl get crd databases.mycompany.com -o yaml

# Describe a CRD (shows schema, accepted versions)
kubectl describe crd databases.mycompany.com

# Delete a CRD (also deletes ALL instances of that resource)
kubectl delete crd databases.mycompany.com

# List all custom resource instances across all namespaces
kubectl get databases -A
```

> ⚠️ ***Warning:*** *Deleting a CRD is destructive. All Custom Resources of that type are permanently*
> *deleted from etcd immediately. There is no soft-delete or grace period.*

10.5 Troubleshooting CRDs and Operators

Error: CRD resource not found after applying

```
$ kubectl apply -f my-database.yaml
error: no matches for kind "Database" in version "databases.example.com/v1"

# The CRD hasn't been installed
kubectl get crd | grep databases.example.com
# Empty → CRD not installed

# Install the CRD first:
kubectl apply -f database-crd.yaml

# Verify CRD is ready:
kubectl get crd databases.example.com
# NAME                        CREATED AT
# databases.example.com       2024-01-15T10:00:00Z
kubectl wait --for=condition=Established crd/databases.example.com --timeout=30s
```

Error: CRD validation failure

```
$ kubectl apply -f my-database.yaml
The Database "mydb" is invalid:
  spec.version: Unsupported value: "5.6": supported values: "5.7", "8.0"

# The CRD has validation (via OpenAPI schema) and your value is not allowed
# Check what values are valid:
kubectl explain database.spec.version
# Or examine the CRD schema:
kubectl get crd databases.example.com -o jsonpath='{.spec.versions[0].schema}' | python3 -m json.tool |
grep -A 5 "version"
```

Operator not reconciling resources

```
# Check if operator is running
kubectl get pods -n operator-system
# NAME                      READY   STATUS    RESTARTS
# database-operator-abc12   0/1     Pending   0   ← not running!

kubectl describe pod database-operator-abc12 -n operator-system
# Events: 0/3 nodes available: Insufficient memory

# Operator is pending - increase resources or free up nodes
# Check operator memory requirements:
kubectl get deployment database-operator -n operator-system \
  -o jsonpath='{.spec.template.spec.containers[0].resources}'

# Common operator issue: RBAC permissions missing
kubectl logs database-operator-abc12 -n operator-system | grep -i "forbidden\|permission\|access"
# Error: failed to get database resource:
#   databases.example.com "mydb" is forbidden:
#   User "system:serviceaccount:operator-system:database-operator"
#   cannot get resource "databases" in API group "databases.example.com"
# Fix: Check and update the operator's ClusterRole
kubectl get clusterrole database-operator-role -o yaml
```

Debug a CRD-managed resource

```
# See all instances of a CRD
kubectl get databases --all-namespaces

# Get status of a CRD instance (operators write to .status)
kubectl describe database mydb

# Check conditions (operators should populate these)
kubectl get database mydb -o jsonpath='{.status.conditions}' | python3 -m json.tool

# Watch CRD resource events
kubectl get events --field-selector involvedObject.kind=Database,involvedObject.name=mydb

# Check operator logs for this specific resource
kubectl logs -n operator-system -l control-plane=controller-manager \
  --tail=50 | grep "mydb\|reconcil"
```

Chapter 11 — Gateway API: Modern Ingress Traffic Management

11.1 Why Gateway API Exists

The traditional Kubernetes `Ingress` resource has significant limitations — it supports only HTTP/S, has limited routing capabilities, and vendors had to use non-standard annotations to add features. The **Gateway API** is the next-generation replacement, offering richer expressiveness, role-based routing, and support for TCP, UDP, gRPC, and TLS passthrough — all with native Kubernetes resources.

The Gateway API is now part of the **official CKA 2025/2026 curriculum** under the Services and Networking domain.

11.2 Gateway API Core Resources

The Gateway API introduces three main resource types:

```
[GatewayClass]  →  defines the controller/implementation (e.g., nginx, istio, cilium)
      ↓
[Gateway]       →  defines a load balancer instance with listeners (ports, protocols)
      ↓
[HTTPRoute]     →  defines routing rules (which paths/hosts go to which Services)
```

This separation provides **role-based delegation**:

- Cluster operators create `GatewayClass` and `Gateway`
- Application teams create `HTTPRoute` (no cluster-level access needed)

11.3 Installing the Gateway API CRDs

The Gateway API is not installed by default. Install its CRDs first:

```
# Install Gateway API CRDs (standard channel)
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.1.0/standard-
install.yaml

# Verify the CRDs are installed
kubectl get crd | grep gateway
# gatewayclasses.gateway.networking.k8s.io
# gateways.gateway.networking.k8s.io
# httproutes.gateway.networking.k8s.io
# grpcroutes.gateway.networking.k8s.io
# referencegrants.gateway.networking.k8s.io
```

11.4 GatewayClass — Defining the Implementation

A `GatewayClass` is cluster-scoped and identifies which controller will implement Gateways.
It is analogous to `IngressClass` for the old Ingress API.

```
apiVersion: gateway.networking.k8s.io/v1
kind: GatewayClass
metadata:
  name: nginx-gateway
spec:
  controllerName: gateway.nginx.org/nginx-gateway-controller
```

```
kubectl apply -f gatewayclass.yaml
kubectl get gatewayclass
# NAME            CONTROLLER                                        ACCEPTED
# nginx-gateway   gateway.nginx.org/nginx-gateway-controller       True
```

11.5 Gateway — Defining the Load Balancer

A `Gateway` is namespace-scoped and defines the actual listener (port, protocol, TLS config).

```
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: my-gateway
  namespace: production
spec:
  gatewayClassName: nginx-gateway          # References the GatewayClass
  listeners:
  - name: http
    port: 80
    protocol: HTTP
    allowedRoutes:
      namespaces:
        from: Same                          # Only allow routes from this namespace
  - name: https
    port: 443
    protocol: HTTPS
    tls:
      mode: Terminate                       # TLS termination at the gateway
      certificateRefs:
      - name: my-tls-secret                # Kubernetes TLS Secret
    allowedRoutes:
      namespaces:
        from: All                           # Allow routes from any namespace
```

```
kubectl apply -f gateway.yaml

kubectl get gateway -n production
# NAME         CLASS           ADDRESS        PROGRAMMED   AGE
# my-gateway   nginx-gateway   203.0.113.5    True         2m
```

11.6 HTTPRoute — Defining Routing Rules

`HTTPRoute` defines how HTTP traffic from a Gateway is routed to Services.

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: my-app-route
  namespace: production
spec:
  parentRefs:
  - name: my-gateway                      # Attach to the Gateway above
    namespace: production
  hostnames:
  - "myapp.example.com"                   # Match this hostname
  rules:
  - matches:
    - path:
        type: PathPrefix
        value: /api                       # Match /api/* paths
    backendRefs:
    - name: api-service
      port: 8080
      weight: 100
  - matches:
    - path:
        type: PathPrefix
        value: /                          # Default: match all other paths
    backendRefs:
    - name: frontend-service
      port: 3000
```

```
kubectl apply -f httproute.yaml

kubectl get httproute -n production
kubectl describe httproute my-app-route -n production
```

11.7 Traffic Splitting with HTTPRoute (Canary Deployments)

```yaml
rules:
- matches:
  - path:
      type: PathPrefix
      value: /
  backendRefs:
  - name: app-stable      # 90% of traffic
    port: 8080
    weight: 90
  - name: app-canary      # 10% of traffic to the new version
    port: 8080
    weight: 10
```

What this does: Distributes traffic based on weights. With weight 90/10, approximately 90% of requests go to `app-stable` and 10% go to `app-canary`. This enables progressive rollouts without changing DNS or Service configurations.

11.8 Comparing Ingress vs Gateway API

Feature	Ingress	Gateway API
Protocols	HTTP/S only	HTTP, HTTPS, TCP, UDP, gRPC, TLS
Routing logic	Limited (path, host)	Path, host, headers, method, query params
Traffic splitting	Via annotations (non-standard)	Native weight-based splitting
Role separation	None — single resource	GatewayClass (cluster), Gateway (ops), HTTPRoute (dev)
Status reporting	Minimal	Rich, structured status conditions
Stability	GA since v1.1	GA (v1) since Kubernetes 1.28

11.6 Troubleshooting Gateway API

Error: Gateway API CRDs not installed

```
$ kubectl apply -f gateway.yaml
error: resource mapping not found for name: "my-gateway"
  namespace: "" from "gateway.yaml":
  no matches for kind "Gateway" in version "gateway.networking.k8s.io/v1"

# Install Gateway API CRDs:
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.0.0/standard-
install.yaml

# Verify:
kubectl get crd | grep gateway.networking.k8s.io
# gatewayclasses.gateway.networking.k8s.io
# gateways.gateway.networking.k8s.io
# httproutes.gateway.networking.k8s.io
```

Error: GatewayClass not accepted

```
$ kubectl get gatewayclass
NAME      CONTROLLER                 ACCEPTED   AGE
nginx     k8s.nginx.org/nginx-class  Unknown    2m

kubectl describe gatewayclass nginx
# Conditions:
#   Type: Accepted
#   Status: Unknown
#   Reason: Pending
#   Message: Waiting for controller to reconcile

# The GatewayClass controller (ingress controller) is not running
kubectl get pods -n nginx-gateway
# If empty: install the nginx gateway controller
helm install nginx-gateway oci://ghcr.io/nginx/charts/nginx-gateway-fabric \
  --namespace nginx-gateway --create-namespace
```

HTTPRoute not routing traffic

```
$ curl http://my-app.example.com/api
curl: (7) Failed to connect to my-app.example.com port 80: Connection refused

# Step 1: Check Gateway is ready
kubectl get gateway my-gateway
# NAME         CLASS   ADDRESS        PROGRAMMED
# my-gateway   nginx   10.0.0.100     True        ← Address assigned, Programmed=True

# Step 2: Check HTTPRoute is attached
kubectl get httproute my-route
kubectl describe httproute my-route | grep -A 10 "Status:"
# Parents:
#   Parent: Kind=Gateway, Name=my-gateway
#   Conditions:
#     Accepted: True
#     ResolvedRefs: True    ← False means backend Service not found

# Step 3: If ResolvedRefs=False - backend service problem
kubectl describe httproute my-route | grep -A 10 "ResolvedRefs"
# Message: BackendNotFound: Service "api-service" not found in namespace "production"

# Step 4: Verify backend service exists and has endpoints
kubectl get svc api-service -n production
kubectl get endpoints api-service -n production

# Step 5: Test backend directly bypassing Gateway
kubectl port-forward svc/api-service 8080:80 -n production &
curl http://localhost:8080/api
```

Chapter 12 — Kubernetes Services: ClusterIP, NodePort, LoadBalancer, and Beyond

12.1 What Is a Service and Why Does It Exist?

Pods are ephemeral — they get new IP addresses every time they restart or are rescheduled. If your frontend Pod hard-coded the IP of a backend Pod, it would break every time the backend Pod restarted. **Services** solve this problem by providing a **stable virtual IP (ClusterIP)** that is backed by a dynamic set of Pods, discovered via label selectors.

```
Client Pod (10.244.1.5)
    │
    ▼ curl http://10.96.45.23:80  (ClusterIP - stable)
[Service: backend-svc]
    │
    ├──▶ Pod-A (10.244.1.10:8080) - healthy
    ├──▶ Pod-B (10.244.2.11:8080) - healthy
    └──   Pod-C (10.244.1.12:8080) - dead (not in Endpoints)
```

kube-proxy on every node maintains iptables/IPVS rules that DNAT traffic from ClusterIP:port to one of the healthy Pod IPs. The Endpoints controller keeps the Pod IP list updated.

12.2 Service Types

Kubernetes has four Service types, each adding capabilities:

```
ClusterIP (default) → accessible only inside cluster
    ↑
NodePort            → exposes on every node's IP:port (30000-32767)
    ↑
LoadBalancer        → provisions a cloud load balancer in front of NodePort
    ↑
ExternalName        → CNAME alias to an external DNS name (no proxying)
```

12.3 ClusterIP Service — Internal Traffic

The default type. Creates a virtual IP accessible only within the cluster.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
  namespace: production
spec:
  type: ClusterIP          # default if not specified
  selector:
    app: backend           # Routes to Pods with this label
    tier: api
  ports:
  - name: http
    protocol: TCP
    port: 80               # Port the Service listens on
    targetPort: 8080       # Port on the Pod (can be name or number)
  - name: https
    protocol: TCP
    port: 443
    targetPort: 8443
  sessionAffinity: None    # or ClientIP (sticky sessions by client IP)
```

```bash
# Create imperatively
kubectl create service clusterip backend-svc --tcp=80:8080

# Or expose a Deployment
kubectl expose deployment backend --port=80 --target-port=8080 --name=backend-svc

# Verify
kubectl get svc backend-svc
# NAME          TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
# backend-svc   ClusterIP   10.96.45.23    <none>        80/TCP    1m

# Test from inside cluster
kubectl run test --image=curlimages/curl --rm -it --restart=Never -- \
  curl http://backend-svc.production.svc.cluster.local
```

12.4 NodePort Service — External Access via Node IPs

Exposes the Service on every node's IP at a specific port (30000–32767).

```yaml
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
spec:
  type: NodePort
  selector:
    app: frontend
  ports:
  - port: 80           # ClusterIP port (internal)
    targetPort: 3000   # Pod port
    nodePort: 31080    # Node port (optional: omit to auto-assign in 30000-32767)
    protocol: TCP
```

```
# Create imperatively
kubectl create service nodeport frontend-svc --tcp=80:3000 --node-port=31080

# Find the auto-assigned nodePort
kubectl get svc frontend-svc -o jsonpath='{.spec.ports[0].nodePort}'

# Access the service from outside cluster
curl http://<any-node-ip>:31080

# Get all node IPs
kubectl get nodes -o jsonpath='{.items[*].status.addresses[?(@.type=="ExternalIP")].address}'
# or InternalIP if no external:
kubectl get nodes -o jsonpath='{.items[*].status.addresses[?(@.type=="InternalIP")].address}'
```

12.5 LoadBalancer Service — Cloud-Provisioned Load Balancer

Builds on NodePort and additionally asks the cloud provider to provision a real load balancer.

```
apiVersion: v1
kind: Service
metadata:
  name: web-lb
  annotations:
    # Cloud-specific annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: "nlb"           # AWS: Network LB
    service.beta.kubernetes.io/aws-load-balancer-scheme: "internet-facing"
    service.beta.kubernetes.io/azure-load-balancer-internal: "false"   # Azure: external
    cloud.google.com/load-balancer-type: "External"                    # GKE
spec:
  type: LoadBalancer
  selector:
    app: web
  ports:
  - port: 80
    targetPort: 8080
  loadBalancerSourceRanges:     # Optional: restrict IPs allowed to reach LB
  - 10.0.0.0/8
  - 203.0.113.0/24
```

```
# Watch until external IP is assigned (cloud provisioning takes ~1-2 min)
kubectl get svc web-lb -w
# NAME     TYPE           CLUSTER-IP     EXTERNAL-IP      PORT(S)        AGE
# web-lb   LoadBalancer   10.96.100.1    <pending>        80:31234/TCP   10s
# web-lb   LoadBalancer   10.96.100.1    34.120.45.67     80:31234/TCP   75s

# For on-premise clusters (no cloud), use MetalLB to get LoadBalancer support:
kubectl apply -f https://raw.githubusercontent.com/metallb/metallb/v0.14.3/config/manifests/metallb-
native.yaml
```

12.6 ExternalName Service — DNS CNAME Alias

Used to integrate in-cluster services with external systems. Creates a CNAME, no proxying.

```
apiVersion: v1
kind: Service
metadata:
  name: external-db
  namespace: production
spec:
  type: ExternalName
  externalName: mydb.database.windows.net   # DNS name to alias
  # No selector needed
```

```
# Pods can now reach the external DB using internal DNS:
# kubectl exec my-pod -- nslookup external-db.production.svc.cluster.local
# → CNAME → mydb.database.windows.net

# Useful for: migrating from external to internal service (just change externalName to selector)
# or for giving a stable internal name to an external SaaS service
```

12.7 Headless Services — Direct Pod Discovery

Sometimes you don't want load balancing — you want to discover all Pod IPs directly. StatefulSets use headless services for stable per-Pod DNS names.

```
apiVersion: v1
kind: Service
metadata:
  name: postgres-headless
spec:
  clusterIP: None       # ← Makes it headless (no VIP, no load balancing)
  selector:
    app: postgres
  ports:
  - port: 5432
    targetPort: 5432
```

```
# With headless service, DNS returns ALL Pod IPs:
kubectl run dnstest --image=busybox:1.35 --rm -it --restart=Never -- \
  nslookup postgres-headless.default.svc.cluster.local
# Server: 10.96.0.10
# Address 1: 10.244.1.10 postgres-0.postgres-headless.default.svc.cluster.local
# Address 2: 10.244.2.15 postgres-1.postgres-headless.default.svc.cluster.local
# Address 3: 10.244.3.20 postgres-2.postgres-headless.default.svc.cluster.local

# For StatefulSets, each Pod gets a stable DNS name:
# <pod-name>.<service-name>.<namespace>.svc.cluster.local
# postgres-0.postgres-headless.default.svc.cluster.local
```

12.8 Endpoints and EndpointSlices

Services don't directly track Pods — they use Endpoints (and in modern K8s, EndpointSlices):

```
# Endpoints are auto-created/updated by the endpoints controller
kubectl get endpoints backend-svc
# NAME          ENDPOINTS                           AGE
# backend-svc   10.244.1.10:8080,10.244.2.11:8080   5m

# If Endpoints shows <none>, the selector matches no Pods:
kubectl get pods -l app=backend  # Should return pods
kubectl describe svc backend-svc | grep Selector  # Check selector

# EndpointSlices (K8s 1.21+ default, more scalable than Endpoints)
kubectl get endpointslices -l kubernetes.io/service-name=backend-svc

# Manually create an Endpoint to point a Service to an external IP:
apiVersion: v1
kind: Endpoints
metadata:
  name: external-legacy-db    # Must match Service name
subsets:
- addresses:
  - ip: 192.168.100.50        # External server IP
  ports:
  - port: 5432
---
apiVersion: v1
kind: Service
metadata:
  name: external-legacy-db
spec:
  ports:
  - port: 5432
  # No selector - uses the manual Endpoints above
```

12.9 Service Troubleshooting Guide

Problem: Service has no endpoints (empty ENDPOINTS)

```
kubectl get endpoints my-svc
# NAME     ENDPOINTS   AGE
# my-svc   <none>      5m

# Step 1: Check Service selector
kubectl describe svc my-svc | grep Selector
# Selector: app=backend,version=v2

# Step 2: Check if any Pods match that selector
kubectl get pods -l app=backend,version=v2
# No resources found ← The Pod labels don't include version=v2

# Step 3: Check Pod labels
kubectl get pods -l app=backend --show-labels
# NAME        READY  STATUS   LABELS
# backend-0   1/1    Running  app=backend,version=v1  ← version=v1, not v2!

# Fix options:
# A: Update Service selector to match Pod labels
kubectl patch svc my-svc -p '{"spec":{"selector":{"app":"backend"}}}'
# B: Add version label to Pod (if intentional)
kubectl label pod backend-0 version=v2
```

Problem: Service accessible by ClusterIP but not by DNS name

```
# Test by ClusterIP
kubectl run test --image=curlimages/curl --rm -it --restart=Never -- \
  curl http://10.96.45.23:80    # Works

# Test by DNS
kubectl run test --image=busybox:1.35 --rm -it --restart=Never -- \
  wget -qO- http://my-svc.default.svc.cluster.local    # Fails

# CoreDNS issue - check CoreDNS pods
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns | grep -i error

# Check Pod's DNS config
kubectl exec some-pod -- cat /etc/resolv.conf
# search default.svc.cluster.local svc.cluster.local cluster.local
# nameserver 10.96.0.10  ← should be CoreDNS ClusterIP
kubectl get svc kube-dns -n kube-system  # Verify ClusterIP matches
```

Problem: NodePort not accessible from outside

```
# Check if the NodePort is listening
ssh <node-ip>
sudo ss -tlnp | grep 31080

# Check firewall rules
sudo iptables -t nat -L | grep 31080
# Should show DNAT rule

# Cloud firewall may be blocking the port
# AWS: Check Security Group inbound rules for port 31080
# GCP: Check firewall rules: gcloud compute firewall-rules list
# Azure: Check NSG inbound rules

# Test from inside the cluster first
kubectl run test --image=curlimages/curl --rm -it --restart=Never -- \
  curl http://<node-ip>:31080
```

Problem: LoadBalancer stuck in `<pending>`

```
kubectl get svc my-lb
# NAME    TYPE           CLUSTER-IP   EXTERNAL-IP   PORT(S)
# my-lb   LoadBalancer   10.96.1.1    <pending>     80:30000/TCP

# No cloud provider to provision external IP
# Solutions:
# 1. Use MetalLB (on-premise)
# 2. Use NodePort instead
# 3. Use port-forward for testing

kubectl describe svc my-lb | grep -A 5 "Events:"
# No events if no cloud controller running
# Check cloud controller manager:
kubectl get pods -n kube-system | grep cloud-controller
```

12.10 Service DNS Reference

All Services get DNS entries in the format:

```
<service-name>.<namespace>.svc.<cluster-domain>

Examples:
  backend.production.svc.cluster.local
  postgres.database.svc.cluster.local
  kubernetes.default.svc.cluster.local  ← The API server itself

Short forms (from within same namespace):
  backend                        → works from 'production' namespace
  backend.production             → works from any namespace
  backend.production.svc         → works from any namespace
  backend.production.svc.cluster.local  → fully qualified

SRV records for named ports:
  _http._tcp.backend.production.svc.cluster.local
  → returns port number and Pod IPs
```

```
# From inside a pod, test all forms:
kubectl exec my-pod -n production -- sh -c "
nslookup backend                                  # same namespace short
nslookup backend.production                       # cross-namespace short
nslookup backend.production.svc.cluster.local     # FQDN
nslookup _http._tcp.backend.production.svc.cluster.local  # SRV
"
```

Chapter 13 — Ingress: HTTP Routing and TLS Termination

13.1 What Is Ingress?

An **Ingress** is a Kubernetes resource that exposes HTTP and HTTPS routes from outside the cluster to Services inside the cluster. It is an API object that defines routing rules; the actual routing is performed by an **Ingress Controller** — a Pod running in the cluster.

```
Internet
    │
    ▼  HTTPS (443)
[Ingress Controller Pod]  ← reads Ingress rules
    │
    ├── /api/*    → api-service:8080
    ├── /web/*    → web-service:80
    └── /static/* → cdn-service:3000
```

Without Ingress: each Service needs its own LoadBalancer (expensive in cloud).
With Ingress: one LoadBalancer routes all HTTP traffic based on host/path rules.

13.2 Installing an Ingress Controller

Kubernetes does not come with an ingress controller. You must install one:

```
# NGINX Ingress Controller (most common)
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.9.5/deploy/
static/provider/cloud/deploy.yaml

# Verify it's running
kubectl get pods -n ingress-nginx
# NAME                                        READY   STATUS    RESTARTS   AGE
# ingress-nginx-controller-6dd4c6d8f-xk2p9   1/1     Running   0          2m

# Get the external IP of the ingress controller's Service
kubectl get svc -n ingress-nginx ingress-nginx-controller
# NAME                       TYPE           CLUSTER-IP    EXTERNAL-IP     PORT(S)
# ingress-nginx-controller   LoadBalancer   10.96.50.1    34.120.55.99    80:32080/TCP,443:32443/TCP

# For minikube:
minikube addons enable ingress

# For bare-metal / on-premise (no cloud LB):
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.9.5/deploy/
static/provider/baremetal/deploy.yaml
# Then use NodePort instead of LoadBalancer external IP
```

13.3 Basic Ingress — Single Service

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: simple-ingress
  namespace: production
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /  # Strip path prefix before forwarding
spec:
  ingressClassName: nginx   # Which controller handles this (required in K8s 1.22+)
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /
        pathType: Prefix    # Prefix, Exact, or ImplementationSpecific
        backend:
          service:
            name: frontend-svc
            port:
              number: 80
```

```
kubectl apply -f ingress.yaml
kubectl get ingress simple-ingress
# NAME             CLASS   HOSTS               ADDRESS         PORTS   AGE
# simple-ingress   nginx   myapp.example.com   34.120.55.99    80      1m

# Point DNS (or /etc/hosts for testing):
echo "34.120.55.99 myapp.example.com" | sudo tee -a /etc/hosts
curl http://myapp.example.com
```

13.4 Path-Based Routing

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: path-based-ingress
  namespace: production
  annotations:
    nginx.ingress.kubernetes.io/use-regex: "true"
spec:
  ingressClassName: nginx
  rules:
  - host: api.example.com
    http:
      paths:
      - path: /api/v1
        pathType: Prefix
        backend:
          service:
            name: api-v1-svc
            port:
              number: 8080
      - path: /api/v2
        pathType: Prefix
        backend:
          service:
            name: api-v2-svc
            port:
              number: 8080
      - path: /static
        pathType: Prefix
        backend:
          service:
            name: static-assets-svc
            port:
              number: 80
      - path: /          # Default catch-all
        pathType: Prefix
        backend:
          service:
            name: frontend-svc
            port:
              number: 80
```

13.5 Host-Based Routing (Virtual Hosting)

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: multi-host-ingress
spec:
  ingressClassName: nginx
  rules:
  - host: www.example.com       # Production frontend
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: prod-frontend
            port:
              number: 80
  - host: api.example.com       # API service
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 8080
  - host: staging.example.com   # Staging environment
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: staging-frontend
            port:
              number: 80
```

13.6 TLS Termination

```
# Step 1: Create a TLS Secret
kubectl create secret tls tls-secret \
  --cert=tls.crt \
  --key=tls.key \
  --namespace=production

# Or generate a self-signed cert for testing:
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout tls.key -out tls.crt \
  -subj "/CN=myapp.example.com/O=myapp"
kubectl create secret tls tls-secret --cert=tls.crt --key=tls.key
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: tls-ingress
  annotations:
    nginx.ingress.kubernetes.io/ssl-redirect: "true"  # HTTP → HTTPS redirect
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - myapp.example.com
    - api.example.com
    secretName: tls-secret    # Contains tls.crt and tls.key
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-svc
            port:
              number: 80
  - host: api.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: api-svc
            port:
              number: 8080
```

cert-manager: Automatic TLS with Let's Encrypt

```
# Install cert-manager
kubectl apply -f https://github.com/cert-manager/cert-manager/releases/download/v1.13.0/cert-
manager.yaml

# Create a ClusterIssuer for Let's Encrypt
cat << 'YAML' | kubectl apply -f -
apiVersion: cert-manager.io/v1
kind: ClusterIssuer
metadata:
  name: letsencrypt-prod
spec:
  acme:
    server: https://acme-v02.api.letsencrypt.org/directory
    email: admin@example.com
    privateKeySecretRef:
      name: letsencrypt-prod-key
    solvers:
    - http01:
        ingress:
          class: nginx
YAML
```

```yaml
# Ingress with auto-TLS via cert-manager:
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: auto-tls-ingress
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod   # Auto-provision TLS
    nginx.ingress.kubernetes.io/ssl-redirect: "true"
spec:
  ingressClassName: nginx
  tls:
  - hosts:
    - myapp.example.com
    secretName: myapp-tls       # cert-manager will create this Secret
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-svc
            port:
              number: 80
```

13.7 Common Ingress Annotations (NGINX)

```
metadata:
  annotations:
    # Rewrite / URL manipulation
    nginx.ingress.kubernetes.io/rewrite-target: /$2           # Strip path prefix
    nginx.ingress.kubernetes.io/use-regex: "true"             # Enable regex in paths

    # Rate limiting
    nginx.ingress.kubernetes.io/limit-rps: "10"               # 10 req/s per IP
    nginx.ingress.kubernetes.io/limit-connections: "5"        # 5 concurrent connections

    # Timeouts
    nginx.ingress.kubernetes.io/proxy-connect-timeout: "5"    # 5s connection timeout
    nginx.ingress.kubernetes.io/proxy-send-timeout: "60"      # 60s send timeout
    nginx.ingress.kubernetes.io/proxy-read-timeout: "60"      # 60s read timeout

    # CORS
    nginx.ingress.kubernetes.io/enable-cors: "true"
    nginx.ingress.kubernetes.io/cors-allow-origin: "https://myapp.example.com"

    # Authentication
    nginx.ingress.kubernetes.io/auth-type: basic
    nginx.ingress.kubernetes.io/auth-secret: basic-auth-secret
    nginx.ingress.kubernetes.io/auth-realm: "Restricted Area"

    # Custom headers
    nginx.ingress.kubernetes.io/configuration-snippet: |
      more_set_headers "X-Frame-Options: SAMEORIGIN";
      more_set_headers "X-Content-Type-Options: nosniff";

    # Backend protocol
    nginx.ingress.kubernetes.io/backend-protocol: "GRPC"      # For gRPC backends
    nginx.ingress.kubernetes.io/backend-protocol: "HTTPS"     # For mTLS backends

    # Session affinity
    nginx.ingress.kubernetes.io/affinity: "cookie"
    nginx.ingress.kubernetes.io/session-cookie-name: "SERVERID"
```

13.8 Troubleshooting Ingress

Problem: 404 Not Found from Ingress

```
# Step 1: Check Ingress is created and has an address
kubectl get ingress my-ingress
# NAME         CLASS   HOSTS             ADDRESS         PORTS   AGE
# my-ingress   nginx   myapp.example.com   <none>          80      5m
# ADDRESS is empty → ingress controller didn't pick it up

# Step 2: Check ingress controller logs
kubectl logs -n ingress-nginx \
  $(kubectl get pods -n ingress-nginx -l app.kubernetes.io/name=ingress-nginx -o jsonpath='{.items[0].m
etadata.name}') \
  --tail=50

# Step 3: Check ingressClassName
kubectl get ingress my-ingress -o jsonpath='{.spec.ingressClassName}'
# If empty and controller requires it:
kubectl patch ingress my-ingress -p '{"spec":{"ingressClassName":"nginx"}}'

# Step 4: Verify backend service exists and has endpoints
kubectl get svc frontend-svc
kubectl get endpoints frontend-svc
```

Problem: 502 Bad Gateway

```
# NGINX can reach backend but backend is returning errors
# Step 1: Test backend directly
kubectl port-forward svc/frontend-svc 8080:80
curl http://localhost:8080

# Step 2: Check if targetPort matches what the app listens on
kubectl get svc frontend-svc -o jsonpath='{.spec.ports[0].targetPort}'
kubectl exec frontend-pod -- ss -tlnp  # What port is the app on?

# Step 3: Check ingress controller config
kubectl get ingress my-ingress -o yaml
# Ensure path and backend match
```

Problem: SSL certificate errors

```
# Check certificate is valid
kubectl get secret tls-secret -o jsonpath='{.data.tls\.crt}' | base64 -d | \
  openssl x509 -text -noout | grep -E "Subject:|DNS:|Not After"

# Check cert-manager issued a certificate
kubectl get certificates -n production
kubectl describe certificate myapp-tls -n production

# cert-manager challenge stuck?
kubectl get challenges -A
kubectl describe challenge <challenge-name> -n production
# Common issue: HTTP01 challenge fails if Ingress isn't publicly accessible
# Use DNS01 challenge instead for private clusters
```

Chapter 14 — NetworkPolicy: Traffic Control and Microsegmentation

14.1 The Default: No Network Isolation

By default, Kubernetes allows all Pod-to-Pod traffic. Any Pod can reach any other Pod in any namespace. NetworkPolicies let you restrict this — but only if your CNI plugin supports them (Calico, Cilium, Weave do; Flannel does NOT).

```
# Check if your CNI supports NetworkPolicy
# Test by creating a deny-all and seeing if traffic is blocked
# If traffic still flows after a deny-all, your CNI doesn't support it
kubectl get pods -n kube-system | grep -E "calico|cilium|weave|canal"
```

NetworkPolicy is **additive** — Pods with no NetworkPolicy applied have unrestricted traffic. Once any NetworkPolicy selects a Pod, all traffic not explicitly allowed is denied.

14.2 Deny-All: The Security Baseline

Always start with a deny-all default, then add explicit allow rules:

```yaml
# Deny all ingress to a namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: production
spec:
  podSelector: {}         # {} = select ALL pods in namespace
  policyTypes:
  - Ingress               # Restrict incoming traffic
  # No ingress rules = deny all ingress
---
# Deny all egress from a namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-egress
  namespace: production
spec:
  podSelector: {}
  policyTypes:
  - Egress
  # No egress rules = deny all egress
---
# Deny all in both directions (most restrictive)
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: production
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  - Egress
```

14.3 Allow-Specific Patterns

Allow traffic from specific Pods

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: backend          # This policy applies to backend Pods
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend     # Only frontend Pods can send ingress to backend
    ports:
    - protocol: TCP
      port: 8080
```

Allow traffic from specific Namespace

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-monitoring
  namespace: production
spec:
  podSelector: {}           # All pods in production
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: monitoring    # Only from monitoring namespace
    ports:
    - port: 9090            # Prometheus scrape port
    - port: 9091
```

Allow traffic from Namespace AND specific Pod (AND logic)

```
# IMPORTANT: from[] with both selectors in the SAME list item = AND (both must match)
# from[] with selectors in SEPARATE list items = OR (either can match)

# AND: must be from monitoring namespace AND have label role=prometheus
ingress:
- from:
  - namespaceSelector:         # One item with BOTH selectors = AND
      matchLabels:
        name: monitoring
    podSelector:
      matchLabels:
        role: prometheus

# OR: from monitoring namespace OR any pod with label role=prometheus
ingress:
- from:
  - namespaceSelector:         # Separate items = OR
      matchLabels:
        name: monitoring
  - podSelector:
      matchLabels:
        role: prometheus
```

Allow traffic from external IPs

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-external-cidr
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: public-api
  policyTypes:
  - Ingress
  ingress:
  - from:
    - ipBlock:
        cidr: 0.0.0.0/0        # All external IPs
        except:
        - 10.0.0.0/8           # Except internal network
        - 172.16.0.0/12
        - 192.168.0.0/16
    ports:
    - port: 443
```

14.4 Egress Policies

```
# Allow pods to reach DNS + specific services only
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-egress-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
  - Egress
  egress:
  # Allow DNS (required for name resolution)
  - to:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: kube-system
      podSelector:
        matchLabels:
          k8s-app: kube-dns
    ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53
  # Allow reaching the database
  - to:
    - podSelector:
        matchLabels:
          app: postgres
    ports:
    - port: 5432
  # Allow reaching external API
  - to:
    - ipBlock:
        cidr: 142.250.0.0/15    # Google APIs IP range
    ports:
    - port: 443
```

14.5 Real-World Policy Patterns

Three-Tier Application (Frontend → Backend → Database)

```
# Tier 1: Frontend can receive from anywhere, can only send to backend
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      tier: frontend
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - {}                      # Allow all ingress (public-facing)
  egress:
  - to:
    - podSelector:
        matchLabels:
          tier: backend
    ports:
    - port: 8080
  - to:                     # Always allow DNS
    - namespaceSelector: {}
      podSelector:
        matchLabels:
          k8s-app: kube-dns
    ports:
    - port: 53
      protocol: UDP
---
# Tier 2: Backend only from frontend, only to database
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: backend-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      tier: backend
  policyTypes:
  - Ingress
  - Egress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          tier: frontend
    ports:
    - port: 8080
  egress:
  - to:
    - podSelector:
        matchLabels:
          tier: database
    ports:
    - port: 5432
  - to:                     # DNS
    - namespaceSelector: {}
      podSelector:
        matchLabels:
          k8s-app: kube-dns
    ports:
```

```
    - port: 53
      protocol: UDP
---
# Tier 3: Database only from backend
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: database-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      tier: database
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          tier: backend
    ports:
    - port: 5432
```

14.6 Testing NetworkPolicies

```
# Setup test environment
kubectl create namespace np-test
kubectl label namespace np-test kubernetes.io/metadata.name=np-test

# Deploy test pods
kubectl run server --image=nginx --labels="app=server" -n np-test
kubectl run client --image=curlimages/curl --labels="app=client" -n np-test \
  --command -- sleep 3600

# Get server IP
SERVER_IP=$(kubectl get pod server -n np-test -o jsonpath='{.status.podIP}')

# Test BEFORE any policy (should work)
kubectl exec client -n np-test -- curl -s --max-time 3 http://$SERVER_IP
# Should return nginx HTML

# Apply deny-all
kubectl apply -f - << 'YAML'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all
  namespace: np-test
spec:
  podSelector: {}
  policyTypes:
  - Ingress
YAML

# Test AFTER deny-all (should fail)
kubectl exec client -n np-test -- curl -s --max-time 3 http://$SERVER_IP
# curl: (28) Connection timed out ← blocked!

# Apply allow rule
kubectl apply -f - << 'YAML'
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-client-to-server
  namespace: np-test
spec:
  podSelector:
    matchLabels:
      app: server
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: client
    ports:
    - port: 80
YAML

# Test after allow rule (should work again)
kubectl exec client -n np-test -- curl -s --max-time 3 http://$SERVER_IP
# Returns nginx HTML ← allowed!

# Cleanup
kubectl delete namespace np-test
```

14.7 Troubleshooting NetworkPolicies

Problem: Traffic blocked when it should be allowed

```
# List all NetworkPolicies in a namespace
kubectl get networkpolicies -n production -o yaml

# Check if a policy is selecting the Pod you think it is
kubectl get networkpolicies -n production -o json | python3 -c "
import sys, json
data = json.load(sys.stdin)
for pol in data['items']:
    sel = pol['spec'].get('podSelector', {}).get('matchLabels', {})
    print(f\"Policy: {pol['metadata']['name']}, selector: {sel}\")
"

# Check Pod labels match policy selector
kubectl get pod my-pod -n production --show-labels

# Verify namespace labels (for namespaceSelector)
kubectl get namespace production --show-labels
# Look for: kubernetes.io/metadata.name=production (auto-set in K8s 1.21+)
# If missing (older K8s):
kubectl label namespace production kubernetes.io/metadata.name=production

# Common mistake: AND vs OR confusion
# These are NOT the same:
# from:                     # OR: either selector matches
# - podSelector: {app: frontend}
# - namespaceSelector: {name: monitoring}
#
# from:                     # AND: both must match
# - podSelector: {app: frontend}
#   namespaceSelector: {name: monitoring}
```

Problem: DNS stops working after applying egress policy

```
# This is the #1 most common NetworkPolicy mistake!
# If you restrict egress, you MUST explicitly allow DNS on UDP/TCP port 53

# Symptom: Pod can reach IPs directly but DNS fails
kubectl exec my-pod -- nslookup google.com     # Fails (DNS blocked)
kubectl exec my-pod -- curl http://8.8.8.8     # Might work if IP egress allowed

# Fix: add DNS egress rule to every egress policy
egress:
- to:
  - namespaceSelector:
      matchLabels:
        kubernetes.io/metadata.name: kube-system
    podSelector:
      matchLabels:
        k8s-app: kube-dns
  ports:
  - protocol: UDP
    port: 53
  - protocol: TCP
    port: 53
```

Chapter 15 — RBAC: Role-Based Access Control

15.1 The RBAC Model

Kubernetes RBAC answers one question: **"Can subject X do action Y on resource Z?"**

```
Subject          Verb           Resource
(who)            (what)         (which object)
_________        ______         ______________

User             get            pods
Group            list           deployments
ServiceAccount   create         secrets
                 update         configmaps
                 patch          services
                 delete         namespaces
                 watch          nodes
                 *              *  (wildcard)
```

Four RBAC objects:

```
Role            → permissions in ONE namespace
ClusterRole     → permissions cluster-wide (or reusable in any namespace)
RoleBinding     → grants a Role or ClusterRole to subjects IN ONE namespace
ClusterRoleBinding → grants a ClusterRole to subjects CLUSTER-WIDE
```

15.2 Roles and ClusterRoles

```
# Role: namespace-scoped permissions
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-manager
  namespace: staging
rules:
- apiGroups: [""]              # "" = core API group (pods, services, configmaps, etc.)
  resources: ["pods", "pods/log", "pods/exec"]
  verbs: ["get", "list", "watch", "create", "delete"]
- apiGroups: ["apps"]          # apps group (deployments, replicasets, etc.)
  resources: ["deployments"]
  verbs: ["get", "list", "update", "patch"]
- apiGroups: [""]
  resources: ["configmaps"]
  verbs: ["get", "list"]
  resourceNames: ["app-config", "feature-flags"]  # Only these specific ConfigMaps
---
# ClusterRole: cluster-wide permissions (used for node info, PVs, CRDs, etc.)
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-reader
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["persistentvolumes"]
  verbs: ["get", "list"]
- apiGroups: ["storage.k8s.io"]
  resources: ["storageclasses"]
  verbs: ["get", "list"]
```

```
# Create imperatively
kubectl create role pod-manager \
  --verb=get,list,watch,create,delete \
  --resource=pods \
  --namespace=staging

kubectl create clusterrole node-reader \
  --verb=get,list,watch \
  --resource=nodes,persistentvolumes

# List all roles/clusterroles
kubectl get roles -n staging
kubectl get clusterroles | grep -v system:   # Exclude system ClusterRoles

# Describe a role (shows rules in readable format)
kubectl describe role pod-manager -n staging
```

15.3 RoleBindings and ClusterRoleBindings

```yaml
# RoleBinding: gives Role permissions to subjects in ONE namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: alice-pod-manager
  namespace: staging
subjects:
- kind: User
  name: alice                          # User by name (from certificate CN)
  apiGroup: rbac.authorization.k8s.io
- kind: Group
  name: dev-team                       # Group by name
  apiGroup: rbac.authorization.k8s.io
- kind: ServiceAccount
  name: ci-runner                      # ServiceAccount
  namespace: staging                   # SA namespace required
roleRef:
  kind: Role
  name: pod-manager
  apiGroup: rbac.authorization.k8s.io
---
# RoleBinding can also reference a ClusterRole (limits scope to namespace)
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: alice-view-staging
  namespace: staging
subjects:
- kind: User
  name: alice
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole    # ClusterRole, but scoped to 'staging' namespace
  name: view           # Built-in ClusterRole
  apiGroup: rbac.authorization.k8s.io
---
# ClusterRoleBinding: gives ClusterRole permissions cluster-wide
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: ops-team-cluster-admin
subjects:
- kind: Group
  name: ops-team
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: cluster-admin   # Full superuser access
  apiGroup: rbac.authorization.k8s.io
```

```
# Create imperatively
kubectl create rolebinding alice-pod-manager \
  --role=pod-manager \
  --user=alice \
  --namespace=staging

kubectl create clusterrolebinding ops-admin \
  --clusterrole=cluster-admin \
  --group=ops-team

# Bind to a ServiceAccount
kubectl create rolebinding ci-runner-binding \
  --role=pod-manager \
  --serviceaccount=staging:ci-runner \
  --namespace=staging
```

15.4 Built-in ClusterRoles

Kubernetes ships with useful built-in ClusterRoles:

ClusterRole	What it allows
`cluster-admin`	Full superuser — everything
`admin`	Full access within a namespace (cannot modify namespace or quotas)
`edit`	Read/write access to most resources in a namespace
`view`	Read-only access to most resources in a namespace
`system:node`	Used by kubelets (read pods/services, write node/pod status)
`system:kube-scheduler`	Used by the scheduler
`system:controller:*`	Used by various controllers

```
# Assign built-in roles quickly
kubectl create rolebinding alice-edit \
  --clusterrole=edit \
  --user=alice \
  --namespace=production

kubectl create rolebinding bob-view \
  --clusterrole=view \
  --user=bob \
  --namespace=production

# View the permissions of a built-in role
kubectl describe clusterrole edit | head -40
```

15.5 ServiceAccount RBAC

ServiceAccounts are identities for Pods to access the Kubernetes API:

```
# Create a dedicated ServiceAccount for an application
kubectl create serviceaccount app-sa -n production

# Create Role with needed permissions
kubectl create role app-role \
  --verb=get,list \
  --resource=configmaps,secrets \
  --namespace=production

# Bind SA to Role
kubectl create rolebinding app-sa-binding \
  --role=app-role \
  --serviceaccount=production:app-sa \
  --namespace=production
```

```
# Reference the SA in a Pod/Deployment
spec:
  serviceAccountName: app-sa          # Use our custom SA
  automountServiceAccountToken: true  # Mount the token (default: true)
  containers:
  - name: app
    image: myapp:1.0
```

```
# Inside the Pod, the token is at:
# /var/run/secrets/kubernetes.io/serviceaccount/token
# The CA cert is at:
# /var/run/secrets/kubernetes.io/serviceaccount/ca.crt

# The app can call the API using this token:
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
curl -H "Authorization: Bearer $TOKEN" \
     --cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
     https://kubernetes.default.svc/api/v1/namespaces/production/configmaps
```

15.6 Testing and Auditing RBAC

```
# Test what the current user can do
kubectl auth can-i create pods
kubectl auth can-i delete secrets --namespace=production
kubectl auth can-i '*' '*'                    # Are you a superuser?

# Impersonate a user to test their permissions
kubectl auth can-i list pods --as=alice
kubectl auth can-i list pods --as=alice --namespace=staging
kubectl auth can-i delete nodes --as=alice   # Should be no

# Impersonate a ServiceAccount
kubectl auth can-i list configmaps \
  --as=system:serviceaccount:production:app-sa \
  --namespace=production

# List ALL permissions for a user (comprehensive audit)
kubectl auth can-i --list --as=alice --namespace=staging

# Check which ClusterRoles and Roles bind to a user
kubectl get rolebindings,clusterrolebindings --all-namespaces \
  -o json | python3 -c "
import sys, json
data = json.load(sys.stdin)
for item in data['items']:
    for subj in item.get('subjects', []):
        if subj.get('name') == 'alice':
            ns = item['metadata'].get('namespace', 'cluster-wide')
            role = item['roleRef']['name']
            print(f'{ns}: {item[\"metadata\"][\"name\"]} -> {role}')
"

# Find all bindings for a specific ServiceAccount
kubectl get rolebindings,clusterrolebindings --all-namespaces \
  -o custom-columns='NAME:.metadata.name,NAMESPACE:.metadata.namespace,SUBJECTS:.subjects[*].name' \
  | grep app-sa
```

15.7 RBAC Troubleshooting

Error: Forbidden — RBAC access denied

```
# Full error:
Error from server (Forbidden): deployments.apps is forbidden:
  User "alice" cannot list resource "deployments" in API group "apps"
  in the namespace "production"

# Diagnose:
# Step 1: Identify what's needed
#   resource: "deployments", apiGroup: "apps", verb: "list", namespace: "production"

# Step 2: Check if alice has a binding
kubectl get rolebindings,clusterrolebindings -A \
  -o wide | grep alice

# Step 3: Check if the binding's role has the permission
kubectl describe rolebinding alice-edit -n production
# Look for: deployments in apps group with list verb

# Step 4: Add permission if missing
kubectl create role deployment-reader \
  --verb=list,get,watch \
  --resource=deployments \
  --namespace=production

kubectl create rolebinding alice-deployment-reader \
  --role=deployment-reader \
  --user=alice \
  --namespace=production

# Step 5: Test immediately
kubectl auth can-i list deployments --as=alice -n production
```

Error: ServiceAccount can't access ConfigMap

```
# Pod logs show:
# Error: configmaps "app-config" is forbidden:
#   User "system:serviceaccount:production:default" cannot get resource "configmaps"

# The pod is using the DEFAULT service account (not a custom one with permissions)
# Fix: create SA with permissions and reference it in Pod

# Quick fix for testing (not recommended for production):
kubectl create rolebinding default-sa-configmap-reader \
  --clusterrole=view \
  --serviceaccount=production:default \
  --namespace=production

# Better: create a dedicated SA
kubectl create serviceaccount app-sa -n production
kubectl create rolebinding app-sa-configmap \
  --role=configmap-reader \
  --serviceaccount=production:app-sa \
  --namespace=production
# Then set serviceAccountName: app-sa in the Pod spec
```

RBAC Audit: Find over-privileged ServiceAccounts

```
# Find all SAs with cluster-admin or admin binding (should be minimal)
kubectl get clusterrolebindings -o json | python3 -c "
import sys, json
data = json.load(sys.stdin)
for b in data['items']:
    role = b['roleRef']['name']
    if role in ('cluster-admin', 'admin'):
        for subj in b.get('subjects', []):
            if subj['kind'] == 'ServiceAccount':
                print(f'SA {subj[\"namespace\"]}/{subj[\"name\"]} has {role}')
"

# Find all SAs with wildcard verbs (danger!)
kubectl get roles,clusterroles -A -o json | python3 -c "
import sys, json
data = json.load(sys.stdin)
for item in data['items']:
    for rule in item.get('rules', []):
        if '*' in rule.get('verbs', []):
            ns = item['metadata'].get('namespace', 'cluster-wide')
            print(f'{ns}/{item[\"metadata\"][\"name\"]}: wildcard verbs on {rule.get(\"resources\")}')
" | grep -v system:
```

Chapter 16 — Kubernetes Security: Authentication, Pod Security, and TLS

16.1 Authentication vs Authorization vs Admission

Requests to the API server go through three gates:

```
kubectl request
    │
    ▼
[1. Authentication]   Who are you?
    │                 → x509 certificates, bearer tokens, OIDC, webhook
    │
    ▼
[2. Authorization]    Are you allowed to do this?
    │                 → RBAC (covered in previous chapter)
    │                 → ABAC, Webhook (rare)
    │
    ▼
[3. Admission Control]  Is the request valid/allowed by policy?
    │                   → MutatingAdmissionWebhook (can modify request)
    │                   → ValidatingAdmissionWebhook (can reject request)
    │                   → Built-ins: ResourceQuota, LimitRanger, PodSecurity
    │
    ▼
[etcd / execution]
```

16.2 Authentication: Users and Certificates

Kubernetes has no built-in user database. Users are authenticated via:

```
# Method 1: x509 Client Certificates (most common for CKA exam)
# The CN (Common Name) becomes the username, O (Organization) becomes the group

# Create a private key for user 'alice'
openssl genrsa -out alice.key 2048

# Create a CSR
openssl req -new -key alice.key \
  -subj "/CN=alice/O=dev-team" \
  -out alice.csr

# Have the cluster CA sign it
sudo openssl x509 -req \
  -in alice.csr \
  -CA /etc/kubernetes/pki/ca.crt \
  -CAkey /etc/kubernetes/pki/ca.key \
  -CAcreateserial \
  -out alice.crt \
  -days 365

# Or use the CertificateSigningRequest API (modern approach):
cat << 'CSR' | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: alice
spec:
  request: $(cat alice.csr | base64 | tr -d '\n')
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 86400   # 24 hours
  usages:
  - client auth
CSR

kubectl certificate approve alice
kubectl get csr alice -o jsonpath='{.status.certificate}' | base64 -d > alice.crt

# Build a kubeconfig for alice
kubectl config set-credentials alice \
  --client-certificate=alice.crt \
  --client-key=alice.key \
  --embed-certs=true \
  --kubeconfig=alice.kubeconfig

kubectl config set-context alice-context \
  --cluster=kubernetes \
  --user=alice \
  --namespace=staging \
  --kubeconfig=alice.kubeconfig
```

16.3 Pod Security Standards (PSS)

Replaced the deprecated PodSecurityPolicy in K8s 1.25+. Three levels:

```
privileged  → unrestricted (no policy)
baseline    → minimal restrictions, prevents known escalations
restricted  → hardened, follows best practices
```

Enforcement via namespace labels:

```
# Apply 'restricted' policy to a namespace (audit mode first)
kubectl label namespace production \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/enforce-version=v1.30 \
  pod-security.kubernetes.io/audit=restricted \
  pod-security.kubernetes.io/warn=restricted

# Modes:
# enforce → reject Pods that violate policy
# audit   → log violations but allow
# warn    → user-facing warning but allow
```

Pod Security — Restricted policy requirements:

```
spec:
  securityContext:
    runAsNonRoot: true           # Required by restricted
    seccompProfile:
      type: RuntimeDefault       # Required by restricted
  containers:
  - name: app
    securityContext:
      allowPrivilegeEscalation: false   # Required
      capabilities:
        drop: ["ALL"]                    # Required
        add: ["NET_BIND_SERVICE"]        # Optional: only re-add what's needed
      runAsNonRoot: true                 # Required
      runAsUser: 1000                    # Good practice
      readOnlyRootFilesystem: true       # Good practice
```

16.4 Secrets: Encryption and Secure Handling

```
# Verify secrets are NOT encrypted at rest (default state):
ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  get /registry/secrets/default/my-secret | strings | head -10
# If you can read the data as text → not encrypted!

# Enable encryption at rest (covered in depth in Chapter 6)
# Key rotation procedure:
# 1. Add new key first in encryption config (as first provider)
# 2. Restart api-server to pick up new config
# 3. Re-encrypt all secrets: kubectl get secrets -A -o json | kubectl replace -f -
# 4. Remove old key from config
# 5. Restart api-server again
```

16.5 Audit Logging

Track every API server request for security auditing:

```
# /etc/kubernetes/audit-policy.yaml
apiVersion: audit.k8s.io/v1
kind: Policy
omitStages:
- RequestReceived             # Don't log the start of every request
rules:
# Log all activity in a sensitive namespace at RequestResponse level
- level: RequestResponse
  namespaces: ["production", "finance"]
  resources:
  - group: ""
    resources: ["secrets", "configmaps"]

# Log pod exec/attach at Metadata level (captures the action, not the data)
- level: Metadata
  resources:
  - group: ""
    resources: ["pods/exec", "pods/attach", "pods/portforward"]

# Log auth failures
- level: Request
  users: ["system:anonymous"]

# Minimal logging for other requests (reduce noise)
- level: Metadata
  resources:
  - group: ""
    resources: ["pods", "services", "deployments"]

# Don't log read-only system operations
- level: None
  users: ["system:kube-proxy", "system:node"]
  verbs: ["get", "list", "watch"]
```

```
# Enable in kube-apiserver static pod:
# --audit-policy-file=/etc/kubernetes/audit-policy.yaml
# --audit-log-path=/var/log/kubernetes/audit.log
# --audit-log-maxage=30
# --audit-log-maxbackup=3
# --audit-log-maxsize=100

# Query the audit log
sudo cat /var/log/kubernetes/audit.log | python3 -c "
import sys, json
for line in sys.stdin:
    try:
        e = json.loads(line)
        if e.get('level') != 'None':
            user = e.get('user', {}).get('username', 'unknown')
            verb = e.get('verb', '')
            resource = e.get('objectRef', {}).get('resource', '')
            ns = e.get('objectRef', {}).get('namespace', '')
            name = e.get('objectRef', {}).get('name', '')
            code = e.get('responseStatus', {}).get('code', '')
            print(f'{user} {verb} {resource}/{name} in {ns} → {code}')
    except:
        pass
" | head -30
```

16.6 TLS Certificates in Kubernetes

Kubernetes PKI has several certificate hierarchies:

```
Cluster CA (/etc/kubernetes/pki/ca.crt)
├── API Server cert (kube-apiserver.crt)
├── API Server kubelet client cert
├── Controller Manager cert
├── Scheduler cert
└── Admin (user) certs

etcd CA (/etc/kubernetes/pki/etcd/ca.crt)
├── etcd server cert (server.crt)
├── etcd peer cert (peer.crt)
└── API server etcd client cert (apiserver-etcd-client.crt)

Front Proxy CA (/etc/kubernetes/pki/front-proxy-ca.crt)
└── front-proxy-client.crt (for aggregated APIs)
```

```
# Check all certificate expiration dates
sudo kubeadm certs check-expiration
# CERTIFICATE                EXPIRES                  RESIDUAL TIME   CERTIFICATE AUTHORITY
# admin.conf                 Jan 15, 2025 10:00 UTC   29d             ca
# apiserver                  Jan 15, 2025 10:00 UTC   29d             ca
# etcd-peer                  Jan 15, 2025 10:00 UTC   29d             etcd-ca
# ...

# Renew ALL certificates (do this before they expire!)
sudo kubeadm certs renew all

# Or renew specific certificate:
sudo kubeadm certs renew apiserver
sudo kubeadm certs renew scheduler.conf
sudo kubeadm certs renew controller-manager.conf

# After renewal, restart control plane components:
sudo crictl ps | grep -E "api|scheduler|controller"
# Kill each to trigger restart:
sudo kill $(sudo crictl ps | grep kube-apiserver | awk '{print $1}')

# Update kubeconfig with renewed admin cert:
sudo kubeadm kubeconfig user --client-name=kubernetes-admin --org=system:masters > /tmp/admin.conf
sudo cp /tmp/admin.conf /etc/kubernetes/admin.conf
cp /etc/kubernetes/admin.conf ~/.kube/config

# Verify new expiry
sudo kubeadm certs check-expiration | grep admin
```

16.7 Security Scanning and Hardening

```
# Run kube-bench to check CIS Kubernetes Benchmark compliance
kubectl run kube-bench --image=aquasec/kube-bench:latest \
  --restart=Never \
  --overrides='{"spec": {"hostPID": true, "hostNetwork": true, "tolerations": [{"operator": "Exists"}],
"volumes": [{"name":"var-lib-etcd","hostPath":{"path":"/var/lib/etcd"}},{"name":"etc-
kubernetes","hostPath":{"path":"/etc/kubernetes"}}], "containers": [{"name":"kube-
bench","image":"aquasec/kube-bench:latest","command":["kube-bench"],"volumeMounts":[{"name":"var-lib-
etcd","mountPath":"/var/lib/etcd"},{"name":"etc-kubernetes","mountPath":"/etc/kubernetes"}]}]}}' \
  -- node

kubectl logs kube-bench | head -60
# [PASS] 4.1.1 Ensure that the kubelet service file permissions are set to 600
# [FAIL] 4.1.2 Ensure that the kubelet service file ownership is set to root:root
# [WARN] 4.1.3 If proxy kubeconfig file exists ensure permissions are set to 600
```

Chapter 17 — Executing Commands Inside Containers

17.1 The `kubectl exec` Command

`kubectl exec` is the primary tool for running commands inside a running container. It is equivalent to `docker exec` in the Docker world and operates by connecting to the container's runtime on the node.

Basic Syntax

```
kubectl exec <pod-name> -- <command>
```

17.2 Running a Single Non-Interactive Command

Use this when you want to run a command and get its output without opening a shell session. This is ideal for quick inspections, script automation, and health checks.

Example: List the contents of a directory inside the container

```
kubectl exec my-app-pod -- ls -la /app
```

What this does: Opens a connection to the container inside `my-app-pod`, runs `ls -la /app`, prints the output to your terminal, and immediately closes the connection.

Example: Print all environment variables

```
kubectl exec my-app-pod -- env
```

What this does: Outputs all environment variables set in the container. This is extremely useful for verifying that ConfigMaps and Secrets were correctly injected as environment variables.

Example: Check the process list inside the container

```
kubectl exec my-app-pod -- ps aux
```

What this does: Shows all running processes inside the container's PID namespace. If `ps` is not available (common in minimal images), you can try:

```
kubectl exec my-app-pod -- cat /proc/1/status
```

This reads the kernel's own process status file for PID 1 (the container's init process).

Example: Read the content of a config file

```
kubectl exec my-app-pod -- cat /etc/nginx/nginx.conf
```

What this does: Prints the contents of a configuration file directly to your terminal without needing to copy it out of the container.

17.3 Opening an Interactive Shell Session

When you need to explore a container interactively, navigate the filesystem, run multiple commands, or troubleshoot dynamically, you open an interactive shell.

The `-it` Flags Explained:

Flag	Long Form	Purpose
`-i`	`--stdin`	Keeps STDIN open so your keyboard input reaches the container
`-t`	`--tty`	Allocates a pseudo-TTY (terminal), enabling interactive features like prompts and colors

Both flags together (`-it`) give you a full interactive terminal session inside the container.

Example: Open a bash shell

```
kubectl exec -it my-app-pod -- /bin/bash
```

What this does: Opens a bash shell inside the container. You will see a prompt like `root@my-app-pod:/#` and can run commands interactively. Type `exit` or press `Ctrl+D` to leave.

Example: Open an sh shell (for minimal/Alpine-based images)

Many production images are built on Alpine Linux or distroless images that do not include bash. In these cases, use `sh`:

```
kubectl exec -it my-app-pod -- /bin/sh
```

Example: Try bash first, fall back to sh

```
kubectl exec -it my-app-pod -- bash 2>/dev/null || kubectl exec -it my-app-pod -- sh
```

What this does: Attempts to start bash; if it fails (exit code non-zero), falls back to sh.

17.4 Targeting a Specific Container in a Multi-Container Pod

When a Pod has more than one container, you must specify which container to target using `-c`.

Example: List containers in a Pod

```
kubectl get pod my-app-pod -o jsonpath='{.spec.containers[*].name}'
```

Output example: `app sidecar`

Example: Exec into the `sidecar` container specifically

```
kubectl exec -it my-app-pod -c sidecar -- /bin/sh
```

What this does: Opens a shell inside the `sidecar` container only. Without `-c`, kubectl defaults to the first container in the Pod spec, which may not be what you want.

Example: Run a command in a specific container non-interactively

```
kubectl exec my-app-pod -c app -- cat /var/log/app.log
```

17.5 Passing Arguments with Special Characters

When your command contains pipes, redirects, or shell operators, you need to invoke a shell explicitly
inside the container to interpret them, rather than letting kubectl try to parse them.

Example: Use grep inside the container (WRONG way)

```
# ❌ This will NOT work — kubectl does not interpret shell operators
kubectl exec my-app-pod -- cat /var/log/app.log | grep "ERROR"
```

The pipe `|` above is interpreted by YOUR local shell, not the container's shell. The grep runs locally against the output, but the intent was to filter inside the container.

Example: Use grep inside the container (CORRECT way)

```
# ✅ Wrap the command in sh -c so the container's shell interprets it
kubectl exec my-app-pod -- sh -c 'cat /var/log/app.log | grep "ERROR"'
```

What this does: Passes the entire string to `sh -c` inside the container. The container's shell then interprets the pipe. Always use single quotes to prevent your local shell from expanding variables or operators.

Example: Check disk usage inside the container

```
kubectl exec my-app-pod -- sh -c 'df -h && du -sh /app/*'
```

Example: Set a variable and use it in the container

```
kubectl exec my-app-pod -- sh -c 'TARGET=/app; ls -la $TARGET'
```

17.8 Troubleshooting kubectl exec Failures

Error: OCI runtime exec failed

```
$ kubectl exec -it my-pod -- /bin/bash
OCI runtime exec failed: exec failed: container_linux.go:380:
  starting container process caused:
  exec: "/bin/bash": stat /bin/bash: no such file or directory

# The container image doesn't have bash (common in Alpine/distroless images)
# Try sh instead:
kubectl exec -it my-pod -- /bin/sh

# Or use busybox sh for very minimal images:
kubectl exec -it my-pod -- /busybox/sh

# For distroless containers (no shell at all), use an ephemeral debug container:
kubectl debug -it my-pod --image=busybox:1.35 --target=my-container
```

Error: unable to upgrade connection: container not found

```
$ kubectl exec -it my-pod -c wrong-container -- sh
error: unable to upgrade connection: container not found ("wrong-container")

# List containers in the pod:
kubectl get pod my-pod -o jsonpath='{.spec.containers[*].name}'
# Or with more detail:
kubectl describe pod my-pod | grep "Container ID:" -B 3

# Use exact container name:
kubectl exec -it my-pod -c correct-container-name -- sh
```

Error: command terminated with exit code 126/127

```
$ kubectl exec my-pod -- mycommand
command terminated with exit code 127  # command not found
command terminated with exit code 126  # command not executable (permissions)

# 127: command not in PATH
# Check if it exists:
kubectl exec my-pod -- which mycommand
kubectl exec my-pod -- find / -name "mycommand" 2>/dev/null

# 126: file exists but not executable
kubectl exec my-pod -- ls -la /usr/local/bin/mycommand
kubectl exec my-pod -- chmod +x /usr/local/bin/mycommand

# Use full path explicitly:
kubectl exec my-pod -- /usr/local/bin/mycommand
```

Error: exec: error reading from connection

```
# Often caused by network interruption or VPN disconnect
# Usually safe to just re-run the exec command
# For interactive sessions, consider using tmux inside the container:
kubectl exec -it my-pod -- tmux new-session -d -s main
kubectl exec -it my-pod -- tmux attach -t main
```

17.9 Running One-Off Commands for Debugging

```
# Dump environment variables
kubectl exec my-pod -- env

# Check which process is listening on a port
kubectl exec my-pod -- ss -tlnp
kubectl exec my-pod -- netstat -tlnp  # if net-tools installed

# Check disk usage inside container
kubectl exec my-pod -- df -h
kubectl exec my-pod -- du -sh /var/log/*

# Check memory usage inside container
kubectl exec my-pod -- cat /proc/meminfo
kubectl exec my-pod -- free -h  # if available

# Inspect running processes
kubectl exec my-pod -- ps aux

# Check timezone (common config issue)
kubectl exec my-pod -- date
kubectl exec my-pod -- cat /etc/timezone

# Check /etc/hosts (DNS overrides)
kubectl exec my-pod -- cat /etc/hosts

# Check open file descriptors
kubectl exec my-pod -- ls -la /proc/1/fd | wc -l

# Run a quick Python script inside container
kubectl exec my-pod -- python3 -c "
import socket
print('Hostname:', socket.gethostname())
print('IP:', socket.gethostbyname(socket.gethostname()))
"

# Tail a log file inside container
kubectl exec -it my-pod -- tail -f /var/log/app/application.log

# Run a series of debug commands and save output
kubectl exec my-pod -- sh -c "
echo '=== Environment ==='; env | sort;
echo '=== Processes ==='; ps aux;
echo '=== Network ==='; ss -tlnp;
echo '=== Disk ==='; df -h;
" > pod-debug-$(date +%Y%m%d).txt
```

17.10 Multi-Container exec Patterns

```
# When a pod has multiple containers (e.g., app + sidecar + init)
kubectl get pod my-pod -o jsonpath='{.spec.containers[*].name}'
# Output: app log-collector metrics-exporter

# Run command in each container
for container in app log-collector metrics-exporter; do
  echo "=== $container ==="
  kubectl exec my-pod -c $container -- ps aux 2>/dev/null || echo "ps not available"
done

# Transfer data between containers via shared volume
kubectl exec my-pod -c app -- sh -c "echo 'test data' > /shared/data.txt"
kubectl exec my-pod -c log-collector -- cat /shared/data.txt
```

Chapter 18 — Viewing and Streaming Container Logs

18.1 The `kubectl logs` Command

Kubernetes captures the standard output (stdout) and standard error (stderr) streams of every container and stores them as logs. `kubectl logs` retrieves these logs. This is different from application-level log files written to disk inside the container.

13.2 Basic Log Retrieval

Example: View all logs from a Pod's default container

```
kubectl logs my-app-pod
```

What this does: Dumps the complete log history of the main container to your terminal. For busy applications, this can be thousands of lines.

Example: View logs from a specific container in a multi-container Pod

```
kubectl logs my-app-pod -c sidecar
```

13.3 Streaming Real-Time Logs

Example: Follow (tail) logs in real time

```
kubectl logs -f my-app-pod
```

What this does: The `-f` (follow) flag keeps the connection open and streams new log lines as they are produced. Equivalent to `tail -f` on a traditional log file. Press `Ctrl+C` to stop streaming.

Example: Follow logs for a specific container

```
kubectl logs -f my-app-pod -c app
```

13.4 Limiting Output Volume

Example: View only the last 50 lines of logs

```
kubectl logs my-app-pod --tail=50
```

What this does: Shows only the most recent 50 log lines. The `--tail` flag accepts any positive integer. Use `--tail=-1` to show all lines (same as no flag).

Example: Combine follow and tail for a live view of recent logs

```
kubectl logs -f my-app-pod --tail=100
```

What this does: Shows the last 100 lines immediately, then continues streaming new lines. This is the most common pattern for active debugging.

13.5 Time-Based Log Filtering

Example: View logs from the last 1 hour

```
kubectl logs my-app-pod --since=1h
```

What this does: Returns only log lines produced in the last hour. Valid units: `s` (seconds), `m` (minutes), `h` (hours).

Example: View logs since a specific timestamp (RFC3339 format)

```
kubectl logs my-app-pod --since-time="2024-06-01T09:00:00Z"
```

What this does: Returns all log lines produced after the specified UTC timestamp. Useful when debugging a specific incident whose start time you know.

13.6 Viewing Previous Container Logs (Crash Recovery)

When a container crashes and restarts, Kubernetes keeps the previous instance's logs briefly.

Example: View logs from the crashed/previous container instance

```
kubectl logs my-app-pod --previous
```

What this does: The `--previous` flag tells Kubernetes to return the logs of the last terminated container instance. This is often the first thing you do when a container is crashlooping, as it shows what was happening just before the crash.

Example: Combine previous with tail for the last moments before crash

```
kubectl logs my-app-pod --previous --tail=200
```

13.7 Viewing Logs from All Pods in a Deployment

Example: Get logs from all Pods matching a label selector

```
kubectl logs -l app=my-app --all-containers=true
```

What this does: Selects all Pods with the label `app=my-app` and retrieves logs from every container in each of those Pods. The `--all-containers=true` flag ensures containers with any name in multi-container Pods are all included.

18.2 Structured Logging and Log Parsing

Modern applications emit JSON-structured logs. `kubectl logs` can be combined with `jq` (or `python -m json.tool`) to parse them:

```
# Application that emits JSON logs
kubectl logs api-pod-abc123 | python3 -c "
import sys, json
for line in sys.stdin:
    try:
        obj = json.loads(line)
        print(f'{obj.get(\"timestamp\",\"\")} [{obj.get(\"level\",\"\")}] {obj.get(\"message\",\"\")}')
    except: print(line, end='')
"

# Filter only ERROR level JSON logs
kubectl logs api-pod | python3 -c "
import sys, json
for line in sys.stdin:
    try:
        obj = json.loads(line)
        if obj.get('level') == 'ERROR':
            print(json.dumps(obj, indent=2))
    except: pass
"
```

18.3 Aggregate Logs Across Pods

```
# Logs from ALL pods matching a label selector
kubectl logs -l app=frontend --all-containers --since=1h

# Log from the newest pod of a deployment
kubectl logs deployment/frontend --since=30m

# Follow logs from all pods in real time with pod name prefix
kubectl logs -l app=frontend -f --prefix

# Output includes pod name prefix:
# [pod/frontend-6d8b4d9-xk2p9] 2024-01-15T10:30:00Z GET /api/users 200 45ms
# [pod/frontend-6d8b4d9-m3n7r] 2024-01-15T10:30:01Z GET /api/health 200 2ms
```

18.4 Log Retention and Previous Container Logs

```
# Get logs from the PREVIOUS container instance (before crash)
kubectl logs my-pod --previous
kubectl logs my-pod -c main-container --previous

# This is critical for diagnosing CrashLoopBackOff — the current container
# has just started so it has no logs yet; --previous shows why it crashed

# Get only last N lines
kubectl logs my-pod --tail=50
kubectl logs my-pod --tail=100 --previous

# Get logs since a timestamp
kubectl logs my-pod --since-time="2024-01-15T10:00:00Z"

# Get logs emitted in the last 30 minutes
kubectl logs my-pod --since=30m

# Combine: last 100 lines from last 1 hour
kubectl logs my-pod --tail=100 --since=1h
```

18.5 Log Aggregation Architecture

For production, stream logs to a central system. The three common patterns:

Pattern 1: DaemonSet Log Collector (Node Agent)

```
# Fluent Bit or Fluentd runs on every node as a DaemonSet
# and reads container log files directly from /var/log/pods/
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluent-bit
  namespace: logging
spec:
  selector:
    matchLabels:
      name: fluent-bit
  template:
    spec:
      containers:
      - name: fluent-bit
        image: fluent/fluent-bit:2.1
        volumeMounts:
        - name: varlog
          mountPath: /var/log
        - name: varlibdockercontainers
          mountPath: /var/lib/docker/containers
          readOnly: true
      volumes:
      - name: varlog
        hostPath:
          path: /var/log
      - name: varlibdockercontainers
        hostPath:
          path: /var/lib/docker/containers
```

Pattern 2: Sidecar Log Forwarder

Each Pod has a sidecar that reads app logs from a shared volume and forwards to a central service.

Pattern 3: Direct Application Logging

App writes directly to stdout/stderr; node agent collects and ships. This is the recommended 12-factor app approach — containers should not manage their own log files.

```
# Where Kubernetes stores container log files on the node
ls /var/log/pods/<namespace>_<podname>_<uid>/<container-name>/
# 0.log     ← current log
# 0.log.gz  ← rotated logs
```

Chapter 19 — Copying Files To and From Containers

19.1 The `kubectl cp` Command

`kubectl cp` allows you to transfer files and directories between your local filesystem and a container's filesystem. Under the hood, it uses `tar` inside the container, so the `tar` binary must be present in the container image.

Basic Syntax

```
kubectl cp <source> <destination>
```

For container paths, use the format: `<pod-name>:<path>` or `<pod-name>/<container-name>:<path>`

19.2 Copying FROM a Container to Local

Example: Copy a log file from the container to your current directory

```
kubectl cp my-app-pod:/var/log/app/error.log ./error.log
```

What this does: Downloads `/var/log/app/error.log` from inside the container and saves it as `error.log` in your current working directory.

Example: Copy an entire directory from the container

```
kubectl cp my-app-pod:/etc/nginx ./nginx-config-backup/
```

What this does: Recursively copies the entire `/etc/nginx` directory from the container to a local folder called `nginx-config-backup`.

Example: Copy from a specific container in a multi-container Pod

```
kubectl cp my-app-pod:/var/log/app.log -c sidecar ./app.log
```

19.3 Copying FROM Local TO a Container

Example: Upload a configuration file into a running container

```
kubectl cp ./updated-config.yaml my-app-pod:/etc/app/config.yaml
```

What this does: Copies the local file `updated-config.yaml` into the container at the specified path. This is useful for hot-patching configurations during debugging without rebuilding an image.

> ⚠️ ***Important Warning:*** *Changes made this way are* ***not persistent****. When the Pod restarts, the container reverts to its original image state. For persistent changes, update ConfigMaps, Secrets, or the container image itself.*

Example: Upload a script for testing purposes

```
kubectl cp ./test-script.sh my-app-pod:/tmp/test-script.sh
kubectl exec my-app-pod -- chmod +x /tmp/test-script.sh
kubectl exec my-app-pod -- /tmp/test-script.sh
```

What this does: Uploads a shell script, makes it executable, and runs it inside the container. This is a common debugging technique when you need to run a complex multi-step test.

Example: Copy entire directory into container

```
kubectl cp ./local-test-data/ my-app-pod:/tmp/test-data/
```

19.3 Troubleshooting kubectl cp

Error: tar: removing leading '/'

```
$ kubectl cp my-pod:/etc/nginx/nginx.conf ./nginx.conf
tar: removing leading '/' from member names

# This is just a WARNING, not an error - the copy succeeds
# The leading / is stripped for safety (relative paths in tar)
# The file will be at ./nginx.conf (without the leading /)
```

Error: container not found

```
$ kubectl cp ./myfile.txt my-pod:/tmp/myfile.txt
error: container not found for pod my-pod

# Pod has multiple containers - specify which one:
kubectl cp ./myfile.txt my-pod:/tmp/myfile.txt -c main-container

# Find container names:
kubectl get pod my-pod -o jsonpath='{.spec.containers[*].name}'
```

Error: no such file or directory (source)

```
$ kubectl cp my-pod:/nonexistent/path ./output
tar: /nonexistent/path: Cannot stat: No such file or directory
error: exit status 1

# Verify the path exists first:
kubectl exec my-pod -- ls -la /nonexistent/
kubectl exec my-pod -- find / -name "filename" 2>/dev/null
```

Copying large files efficiently

```
# kubectl cp is not ideal for large files (streams through API server)
# For large file transfers, use port-forward + direct connection:

# Method 1: Port forward to a transfer server
kubectl run transfer-pod --image=python:3.11 --restart=Never --command -- \
  python3 -m http.server 8080
kubectl port-forward pod/transfer-pod 8080:8080 &
# Upload via curl
curl -X PUT http://localhost:8080/ --data-binary @largefile.tar.gz

# Method 2: Use tar + exec for directory archives
kubectl exec my-pod -- tar czf - /var/data | tar xzf - -C ./local-backup/

# Method 3: Copy via shared volume (for same namespace pods)
# Mount the same PVC in a debug pod and copy data there
```

19.4 Common File Operations via kubectl exec + cp

```
# Pattern: modify a config file in a running container
# Step 1: Copy out current config
kubectl cp my-pod:/etc/app/config.json ./config.json

# Step 2: Edit locally
vim config.json

# Step 3: Copy back
kubectl cp ./config.json my-pod:/etc/app/config.json

# Step 4: Reload app (method depends on app)
kubectl exec my-pod -- kill -HUP 1
# or
kubectl rollout restart deployment/my-app

# Backup entire directory from pod
kubectl exec my-pod -- tar czf - /var/lib/app/data/ > app-data-backup.tar.gz
ls -lh app-data-backup.tar.gz

# Restore backup to pod
cat app-data-backup.tar.gz | kubectl exec -i my-pod -- tar xzf - -C /

# Copy from one pod to another (via localhost)
kubectl cp source-pod:/data/file.db ./temp-file.db
kubectl cp ./temp-file.db target-pod:/data/file.db
```

Chapter 20 — Inspecting and Describing Containers

20.1 Getting Pod and Container Details

`kubectl describe pod`

The `describe` command provides a human-readable summary of a Kubernetes resource. For Pods, it includes container images, resource requests and limits, environment variables, volume mounts, events, and current container states.

Example: Describe a Pod

```
kubectl describe pod my-app-pod
```

Key sections to look at in the output:

- **Containers section:** Shows image name, container ID, ports, environment variables, volume mounts,
  resource limits, and the current state (Running, Waiting, Terminated) with reason and exit code.
- **Events section:** Shows recent Kubernetes events for the Pod — scheduling decisions, image pulls,
  container starts, OOMKills, and errors. This is often the first place to look for clues.

Example: Describe all Pods in a namespace

```
kubectl describe pods -n my-namespace
```

15.2 Getting Raw YAML/JSON Output

Example: Get the full Pod spec as YAML

```
kubectl get pod my-app-pod -o yaml
```

What this does: Outputs the full Kubernetes object definition as YAML. This is useful for seeing every field, including those not shown by `describe`, such as the `podIP`, `hostIP`, `nodeName`, all annotations, and the full container spec.

Example: Get a specific field using JSONPath

```
# Get the container image name
kubectl get pod my-app-pod -o jsonpath='{.spec.containers[0].image}'
```

```
# Get all container names in the Pod
kubectl get pod my-app-pod -o jsonpath='{.spec.containers[*].name}'
```

```
# Get the Pod's IP address
kubectl get pod my-app-pod -o jsonpath='{.status.podIP}'
```

```
# Get the node the Pod is scheduled on
kubectl get pod my-app-pod -o jsonpath='{.spec.nodeName}'
```

What JSONPath does: Allows you to extract specific fields from the JSON representation of a resource. The `.spec.containers[*].name` path returns the name of every container in the Pod.

15.3 Checking Container State and Exit Codes

Example: Check why a container is in CrashLoopBackOff

```
kubectl get pod my-app-pod -o jsonpath='{.status.containerStatuses[0]}'
```

What this does: Returns the full `containerStatus` object which includes: `state` (current), `lastState` (previous termination), `exitCode`, `reason`, `message`, and `restartCount`.

Example: Get the exit code of the last terminated container

```
kubectl get pod my-app-pod \
  -o jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}'
```

Exit code meanings:

Exit Code	Meaning
`0`	Success
`1`	General error
`137`	OOMKilled (Out of Memory, signal 9)
`143`	Graceful termination (signal 15)
`255`	Exit from shell without a code

15.4 Listing All Containers Across All Pods

Example: List every Pod and its container images in the cluster

```
kubectl get pods -A -o=custom-columns=\
'NAMESPACE:.metadata.namespace,POD:.metadata.name,CONTAINERS:.spec.containers[*].name,IMAGES:.spec.cont
ainers[*].image'
```

What this does: Uses custom column output format to show namespace, pod name, all container names,
and all container images in a tabular format across all namespaces.

20.2 Comparing Resource State with diff

```
# Compare live resource with a local file
kubectl diff -f deployment.yaml

# This shows what WOULD change if you applied the file
# Green (+) = will be added, Red (-) = will be removed

# Compare two different resources
kubectl get deployment frontend -o yaml > frontend-live.yaml
kubectl get deployment backend -o yaml > backend-live.yaml
diff frontend-live.yaml backend-live.yaml
```

20.3 Checking Resource Health with Conditions

Every Kubernetes object has **Conditions** that give detailed health information:

```
# Pod conditions
kubectl get pod mypod -o jsonpath='{.status.conditions}' | python3 -m json.tool
# Types:
#   PodScheduled: True  → Pod has been assigned to a node
#   ContainersReady: True → All containers passing readiness
#   Initialized: True   → All init containers completed
#   Ready: True         → Pod is ready to serve traffic

# Deployment conditions
kubectl describe deployment myapp | grep -A 30 "Conditions:"
# Available: True      → minAvailable replicas are ready
# Progressing: True    → rolling update is in progress or completed
# ReplicaFailure: False→ no replica creation failures

# Node conditions
kubectl describe node worker-1 | grep -A 20 "Conditions:"
# MemoryPressure: False  → node has sufficient memory
# DiskPressure: False    → node has sufficient disk
# PIDPressure: False     → node has sufficient PIDs
# Ready: True            → node is healthy
```

20.4 Using jsonpath and jq for Advanced Inspection

```
# Get all Pod IPs in a namespace
kubectl get pods -n production -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.status.podIP}{"\n"}
{end}'

# Get all container images across all pods
kubectl get pods --all-namespaces \
  -o jsonpath='{range .items[*]}{.metadata.namespace}{"\t"}{.metadata.name}{"\t"}
{range .spec.containers[*]}{.image}{"\n"}{end}{end}'

# Find pods with no resource requests defined
kubectl get pods --all-namespaces -o json | python3 -c "
import sys, json
data = json.load(sys.stdin)
for pod in data['items']:
    for c in pod['spec'].get('containers', []):
        if not c.get('resources', {}).get('requests'):
            print(f'{pod[\"metadata\"][\"namespace\"]}/{pod[\"metadata\"][\"name\"]}: {c[\"name\"]} has
no requests')
"

# Get all services with their selectors and ports
kubectl get svc --all-namespaces \
  -o custom-columns='NS:.metadata.namespace,NAME:.metadata.name,SELECTOR:.spec.selector,PORTS:.spec.por
ts[*].port'

# Find Deployments not at desired replicas
kubectl get deployment --all-namespaces \
  -o custom-columns='NS:.metadata.namespace,NAME:.metadata.name,DESIRED:.spec.replicas,READY:.status.re
adyReplicas' \
  | awk '$3 != $4 {print}'
```

20.5 events: The Most Underused Diagnostic Tool

```
# Events are the first place to look when something is wrong
kubectl get events -n production --sort-by='.lastTimestamp'

# Events for a specific resource
kubectl get events -n production --field-selector involvedObject.name=my-pod

# Watch events in real time (excellent during deployments)
kubectl get events -n production -w

# Events with reason filter
kubectl get events --field-selector reason=Failed -A
kubectl get events --field-selector reason=BackOff -A
kubectl get events --field-selector type=Warning -A

# Events don't survive node restarts by default (TTL: 1 hour)
# For persistent events, install an events exporter to Elasticsearch or Prometheus
```

Chapter 21 — Port Forwarding to Containers

21.1 The `kubectl port-forward` Command

`kubectl port-forward` creates a secure tunnel between a port on your local machine and a port on a container inside the cluster. Traffic is proxied through the Kubernetes API server and does not require a Service or Ingress to be configured.

This is invaluable for:

- Debugging services that are not exposed externally
- Testing new services before exposing them
- Accessing admin UIs, databases, or metrics endpoints directly

Basic Syntax

```
kubectl port-forward <pod-name> <local-port>:<container-port>
```

16.2 Forward a Single Port

Example: Access an nginx container running on port 80

```
kubectl port-forward my-nginx-pod 8080:80
```

What this does: Forwards your local port `8080` to port `80` inside the container. You can then open `http://localhost:8080` in your browser or run `curl http://localhost:8080` and the request is tunneled into the container.

The terminal stays occupied with the tunnel. Press `Ctrl+C` to close it.

Example: Forward to a database container on port 5432

```
kubectl port-forward my-postgres-pod 5432:5432
```

What this does: Allows you to connect to PostgreSQL running in the cluster using your local `psql` client: `psql -h localhost -p 5432 -U myuser`

16.3 Forwarding Multiple Ports

Example: Forward two ports simultaneously

```
kubectl port-forward my-app-pod 8080:80 9090:9090
```

What this does: Forwards local port 8080 to container port 80 (HTTP), and local port 9090 to container port 9090 (e.g., a Prometheus metrics endpoint).

16.4 Port Forwarding to a Service or Deployment

You can also forward to a Service or Deployment, and Kubernetes selects one of the backing Pods.

Example: Forward to a Service

```
kubectl port-forward service/my-app-service 8080:80
```

Example: Forward to a Deployment

```
kubectl port-forward deployment/my-app-deployment 8080:80
```

16.5 Run Port Forward in Background

```
kubectl port-forward my-app-pod 8080:80 &
PF_PID=$!
# ... do your work ...
kill $PF_PID
```

What this does: Runs the port-forward in the background (`&`), captures its PID, and allows you to kill it when done.

21.2 Port Forwarding for Database Access

Port forwarding is especially useful for database access during debugging:

```
# Forward to a PostgreSQL pod
kubectl port-forward pod/postgres-0 5432:5432 -n database

# Forward via Service (better — survives pod restarts)
kubectl port-forward svc/postgres 5432:5432 -n database &

# Now connect from localhost with any PostgreSQL client:
psql -h localhost -U postgres -d mydb
pg_dump -h localhost -U postgres mydb > backup.sql

# Forward to Redis
kubectl port-forward svc/redis 6379:6379 -n cache &
redis-cli -h localhost ping    # PONG

# Forward to Kubernetes Dashboard
kubectl port-forward svc/kubernetes-dashboard -n kubernetes-dashboard 8443:443 &
# Now open: https://localhost:8443
```

21.3 Multiple Simultaneous Port Forwards

```
# Forward multiple ports of the same pod
kubectl port-forward pod/myapp 8080:8080 9090:9090 6060:6060 &
# 8080 → app port
# 9090 → Prometheus metrics
# 6060 → pprof debugging

# Port forward in the background with PID tracking
kubectl port-forward svc/elasticsearch 9200:9200 &
PF_PID=$!
echo "Port forward running with PID $PF_PID"

# Do your work...
curl localhost:9200/_cluster/health

# Cleanup when done
kill $PF_PID
```

21.4 Limitations of Port Forwarding

Port forwarding is a **debugging tool**, not a production traffic routing mechanism:

- Only one client can use the tunnel at a time (it's a single TCP connection)
- Stops when kubectl process exits
- No TLS for the local side (connection is plain TCP locally)
- Cannot forward UDP
- Cannot forward to headless Services (no ClusterIP)

For production external access, use:

- `Service.type=NodePort` — exposes on every node's IP
- `Service.type=LoadBalancer` — creates a cloud load balancer
- `Ingress` / Gateway API — HTTP routing with hostname/path rules

Chapter 22 — Resource Monitoring Inside Containers

22.1 Using `kubectl top`

`kubectl top` shows real-time CPU and memory consumption of Pods and their containers. It requires the **Metrics Server** to be installed in your cluster.

Example: View resource usage of all Pods in the current namespace

```
kubectl top pods
```

Example: View resource usage broken down by container

```
kubectl top pod my-app-pod --containers
```

Output example:

```
POD           NAME      CPU(cores)   MEMORY(bytes)
my-app-pod    app       15m          128Mi
my-app-pod    sidecar   2m           32Mi
```

What this tells you: The `app` container is using 15 millicores of CPU and 128 MiB of memory. This is useful for right-sizing resource requests and limits.

Example: Sort by memory usage

```
kubectl top pods --sort-by=memory
```

Example: Sort by CPU across all namespaces

```
kubectl top pods -A --sort-by=cpu
```

22.2 Checking Resource Requests and Limits

Example: View resource requests and limits via describe

```
kubectl describe pod my-app-pod | grep -A 6 "Limits\|Requests"
```

Example: Get resource requests via JSONPath

```
kubectl get pod my-app-pod \
  -o jsonpath='{.spec.containers[0].resources}'
```

22.3 Metrics Server Architecture

Metrics Server is the in-tree solution for core metrics (CPU/memory). It is a prerequisite for `kubectl top` and HPA.

```
# Install Metrics Server
kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/
components.yaml

# For lab environments where nodes don't have valid TLS certificates:
kubectl patch deployment metrics-server -n kube-system \
  --type=json \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"}]'

# Verify Metrics Server is running and healthy
kubectl get deployment metrics-server -n kube-system
kubectl top nodes
kubectl top pods --all-namespaces
```

22.4 Resource Quotas and LimitRanges

ResourceQuota — Limit Total Namespace Consumption

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: production-quota
  namespace: production
spec:
  hard:
    # Compute resources
    requests.cpu: "20"           # Total CPU requests in namespace
    requests.memory: 40Gi        # Total memory requests
    limits.cpu: "40"             # Total CPU limits
    limits.memory: 80Gi          # Total memory limits

    # Object count limits
    pods: "100"                  # Max Pods
    services: "20"               # Max Services
    secrets: "50"                # Max Secrets
    configmaps: "50"             # Max ConfigMaps
    persistentvolumeclaims: "20" # Max PVCs

    # Service type restrictions
    services.loadbalancers: "2"  # Only 2 LoadBalancer Services
    services.nodeports: "0"      # No NodePort Services allowed
```

```
# Create ResourceQuota
kubectl apply -f quota.yaml

# Check quota usage
kubectl describe resourcequota production-quota -n production
# Output:
# Resource               Used   Hard
# --------               ----   ----
# limits.cpu             8      40
# limits.memory          16Gi   80Gi
# pods                   23     100
# requests.cpu           4      20
# requests.memory        8Gi    40Gi

# When quota is hit, Pods will fail to create:
# Error: pods "new-pod" is forbidden: exceeded quota: production-quota,
#        requested: pods=1, used: pods=100, limited: pods=100
```

LimitRange — Set Default Requests and Limits

Without LimitRange, users can create Pods with no resource requests, which makes scheduling unpredictable. LimitRange sets defaults automatically:

```
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: production
spec:
  limits:
  - type: Container
    default:             # Applied as limit if not specified
      cpu: 500m
      memory: 512Mi
    defaultRequest:      # Applied as request if not specified
      cpu: 250m
      memory: 256Mi
    max:                 # Container cannot exceed these
      cpu: "4"
      memory: 4Gi
    min:                 # Container must request at least this
      cpu: 100m
      memory: 128Mi

  - type: Pod
    max:                 # Total Pod limits cannot exceed
      cpu: "8"
      memory: 8Gi

  - type: PersistentVolumeClaim
    max:
      storage: 50Gi      # PVCs cannot request more than 50Gi
    min:
      storage: 1Gi       # PVCs must request at least 1Gi
```

```
# View limit ranges in a namespace
kubectl describe limitrange default-limits -n production

# A new Pod without resources gets the default from LimitRange:
kubectl run test --image=nginx -n production
kubectl describe pod test -n production | grep -A 5 "Limits:"
# Limits:
#   cpu:     500m         ← Set by LimitRange default
#   memory:  512Mi
# Requests:
#   cpu:     250m         ← Set by LimitRange defaultRequest
#   memory:  256Mi
```

22.5 Node and Cluster Capacity Planning

```
# Total allocatable resources on each node
kubectl describe nodes | grep -A 5 "Allocatable:"

# Total requested resources vs capacity across all nodes
kubectl describe nodes | grep -A 10 "Allocated resources:"

# Find nodes under memory pressure
kubectl get nodes -o custom-columns=\
'NAME:.metadata.name,CONDITIONS:.status.conditions[?(@.type=="MemoryPressure")].status'

# Check for resource pressure events on nodes
kubectl describe node <node-name> | grep -A 20 "Events:"

# List all Pods sorted by CPU request
kubectl get pods -A -o json | python3 -c "
import sys, json
data = json.load(sys.stdin)
pods = []
for pod in data['items']:
    for c in pod['spec'].get('containers', []):
        cpu = c.get('resources', {}).get('requests', {}).get('cpu', '0')
        pods.append((pod['metadata']['namespace'], pod['metadata']['name'], c['name'], cpu))
for p in sorted(pods, key=lambda x: x[3], reverse=True)[:20]:
    print(f'{p[0]}/{p[1]} container={p[2]} cpu={p[3]}')
"
```

Chapter 23 — Removing and Modifying Containers in Pods

23.1 The Immutability of Running Pod Specs

One of the most important concepts to understand before attempting to remove or modify containers in a
Pod is that **Kubernetes Pods are largely immutable once created**. The core container list of a running
Pod — the `spec.containers[]` array — cannot be edited in-place the way you might edit a text file. This is a deliberate design decision: the Pod spec defines the desired state that Kubernetes enforced at creation time, and modifying it arbitrarily would make the actual state unpredictable.

However, this does not mean you are stuck. Kubernetes gives you several mechanisms to achieve the
practical outcome of removing or changing containers, depending on the resource type managing the Pod.

There are four main scenarios this chapter addresses:

1. Removing a container from a **bare (unmanaged) Pod** — requires delete and recreate.
2. Removing a container from a Pod managed by a **Deployment, StatefulSet, or DaemonSet** — update the
   spec and Kubernetes rolls out the change.
3. **Killing a container's process** inside the Pod to force a restart without recreating the Pod.
4. Using **strategic merge patches** and **JSON patches** to surgically modify Pod specs.

23.2 Why You Cannot Simply Remove a Container from a Running Pod

Kubernetes enforces the following fields as immutable on running Pods:

- `spec.containers` (the full list — you cannot add, remove, or rename containers)
- `spec.initContainers`
- `spec.volumes` (mostly immutable, some fields can change)
- `spec.nodeName`
- `spec.serviceAccountName`

If you attempt to patch a running Pod to remove a container entry, the API server will reject it:

```
# ❌ This will be rejected by the API server
kubectl patch pod my-app-pod --type=json \
  -p='[{"op": "remove", "path": "/spec/containers/1"}]'
```

Expected error:

```
The Pod "my-app-pod" is invalid:
spec.containers: Forbidden: pod updates may not add or remove containers
```

This is not a bug — it is protection. The container runtime on the node manages container lifecycle tightly coupled to the Pod spec that was used at creation. Changing it mid-flight would leave the runtime in an inconsistent state.

23.3 Removing a Container from a Bare (Unmanaged) Pod

If your Pod is not managed by a higher-level controller (no Deployment, no StatefulSet), the only way to remove a container from it is to delete the Pod and recreate it with the updated spec.

Step 1: Export the current Pod spec to a YAML file

```
kubectl get pod my-app-pod -o yaml > my-app-pod.yaml
```

What this does: Exports the full live Pod specification to a YAML file. This includes all fields that Kubernetes has added (status, resourceVersion, etc.) as well as your original spec.

Step 2: Clean up the exported YAML

The exported YAML contains runtime-only fields that will cause errors if submitted back to the API. Remove these fields:

```
# Remove runtime-managed fields using kubectl to do a dry-run apply
# Or manually remove these sections from the YAML file:
```

Fields to remove from the exported YAML before resubmitting:

- `metadata.uid`
- `metadata.resourceVersion`
- `metadata.creationTimestamp`
- `metadata.selfLink` (if present)
- `status` (entire block)
- `spec.nodeName` (forces reschedule)

Step 3: Edit the YAML to remove the container

Open the YAML file and find the `spec.containers` section:

```
# BEFORE — two containers
spec:
  containers:
  - name: app
    image: my-app:v1
    ports:
    - containerPort: 8080
  - name: sidecar               # ← Remove this entire block
    image: log-shipper:latest
    volumeMounts:
    - name: logs
      mountPath: /var/log
```

```
# AFTER — sidecar removed
spec:
  containers:
  - name: app
    image: my-app:v1
    ports:
    - containerPort: 8080
```

Step 4: Delete the old Pod and recreate with the new spec

```
# Delete the running Pod (note: for bare pods, this causes downtime)
kubectl delete pod my-app-pod

# Recreate with updated spec
kubectl apply -f my-app-pod.yaml
```

All-in-one with kubectl replace --force (faster, but destructive)

```
# Edit the file first, then:
kubectl replace --force -f my-app-pod.yaml
```

What `--force` does: Deletes the existing resource and immediately recreates it from the file. This is equivalent to `delete` + `create` in one command. **Warning:** There is a brief period of downtime between deletion and recreation.

23.4 Removing a Container from a Deployment-Managed Pod

This is the correct production approach. When Pods are managed by a Deployment, you update the Deployment's Pod template spec — not the Pod directly. Kubernetes then performs a rolling update, gracefully replacing old Pods (with the sidecar) with new Pods (without the sidecar).

Method 1: kubectl edit (interactive)

```
kubectl edit deployment my-app-deployment
```

What this does: Opens the Deployment spec in your default editor (set by `$EDITOR` or `$KUBE_EDITOR`). Navigate to `spec.template.spec.containers`, find the container you want to remove, and delete its entire block. Save and close the file. Kubernetes immediately begins a rolling update.

Method 2: kubectl patch with JSON Patch (precise, scriptable)

JSON Patch operations allow you to surgically modify specific array elements. To remove the second container (index 1) from the containers list:

```
kubectl patch deployment my-app-deployment --type=json \
  -p='[{"op": "remove", "path": "/spec/template/spec/containers/1"}]'
```

JSON Patch operations explained:

Operation	`op` value	Effect
Remove element	`"remove"`	Deletes the element at the path
Replace value	`"replace"`	Replaces the value at the path
Add element	`"add"`	Adds at the path
Copy element	`"copy"`	Copies from one path to another

Method 3: kubectl patch with Strategic Merge Patch (by name)

Strategic merge patch is a Kubernetes-specific extension of JSON merge patch. For containers, you can instruct Kubernetes to remove a container by name using the `$patch: delete` directive:

```
kubectl patch deployment my-app-deployment --type=strategic \
  -p='{"spec":{"template":{"spec":{"containers":[{"name":"sidecar","$patch":"delete"}]}}}}'
```

What this does: Kubernetes uses the container `name` as the merge key. Setting `$patch: delete` on a named container tells the strategic merge to remove that container from the array, rather than trying to merge it. This is name-based, not index-based, making it safer than JSON Patch.

Method 4: Edit and apply from a file

```
# Get deployment spec
kubectl get deployment my-app-deployment -o yaml > deployment.yaml

# Edit deployment.yaml — remove the container from spec.template.spec.containers

# Apply the change
kubectl apply -f deployment.yaml
```

23.5 Watching the Rollout After Container Removal

After editing a Deployment to remove a container, Kubernetes performs a rolling update. You can monitor this in real time.

Example: Watch the rollout status

```
kubectl rollout status deployment/my-app-deployment
```

Output while rolling:

```
Waiting for deployment "my-app-deployment" rollout to finish: 1 out of 3 new replicas have been
updated...
Waiting for deployment "my-app-deployment" rollout to finish: 2 out of 3 new replicas have been
updated...
Waiting for deployment "my-app-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "my-app-deployment" successfully rolled out
```

Example: Watch Pods being replaced

```
kubectl get pods -w -l app=my-app
```

The `-w` flag (watch) streams live updates of Pod status changes to your terminal.

Example: Rollback if something went wrong

```
kubectl rollout undo deployment/my-app-deployment
```

What this does: Reverts the Deployment to the previous revision, restoring the removed container. Kubernetes keeps a configurable history of revisions (default: 10).

Example: View rollout history

```
kubectl rollout history deployment/my-app-deployment
```

```
# See what changed in a specific revision
kubectl rollout history deployment/my-app-deployment --revision=3
```

23.6 Removing Containers from StatefulSets and DaemonSets

The same `kubectl patch` and `kubectl edit` approach applies to StatefulSets and DaemonSets, with one important difference: StatefulSets perform rolling updates one Pod at a time (ordered), and DaemonSets update one node at a time.

Example: Remove a container from a StatefulSet

```
kubectl patch statefulset my-statefulset --type=json \
  -p='[{"op": "remove", "path": "/spec/template/spec/containers/1"}]'
```

Example: Remove a container from a DaemonSet

```
kubectl patch daemonset my-daemonset --type=strategic \
  -p='{"spec":{"template":{"spec":{"containers":[{"name":"log-agent","$patch":"delete"}]}}}}'
```

23.7 Killing a Container Process to Force a Restart

Sometimes you do not want to remove a container permanently — you just want to force it to restart (e.g., to pick up a new configuration, or to recover from a deadlock). You can kill the container's main process from inside the Pod. Kubernetes will detect the process exit and restart the container according to the Pod's `restartPolicy`.

Example: Send SIGTERM (graceful shutdown) to PID 1 in the container

```
kubectl exec my-app-pod -c sidecar -- kill -15 1
```

What this does: Sends signal 15 (SIGTERM) to PID 1 inside the `sidecar` container. A well-behaved application should shut down gracefully. Kubernetes sees the container exit and restarts it.

Example: Send SIGKILL (immediate, forceful kill) to PID 1

```
kubectl exec my-app-pod -c sidecar -- kill -9 1
```

What this does: Signal 9 (SIGKILL) cannot be caught or ignored by the process — the kernel kills it immediately. Use this when the container is frozen and not responding to SIGTERM.

Example: Find and kill a specific process (not PID 1)

```
# Step 1: Find the process ID
kubectl exec my-app-pod -- ps aux | grep "stuck-process"

# Step 2: Kill it by its PID
kubectl exec my-app-pod -- kill -9 <PID>
```

Example: Restart all containers in a Pod by deleting the Pod (for Deployment-managed Pods)

```
kubectl delete pod my-app-pod-xyz123
```

What this does: Kubernetes (via the Deployment's ReplicaSet controller) immediately creates a replacement Pod. This is the cleanest way to force a full Pod restart when the Pod is managed by a Deployment. The old Pod terminates gracefully and a new one is scheduled.

23.8 Removing Init Containers

Init containers run to completion before the main containers start. They can also be removed using the same strategic merge patch approach:

Example: Remove an init container by name

```
kubectl patch deployment my-app-deployment --type=json \
  -p='[{"op": "remove", "path": "/spec/template/spec/initContainers/0"}]'
```

Note: Init containers are at `/spec/template/spec/initContainers`, not `/spec/template/spec/containers`.

23.9 Scaling Down to Zero (Temporary Removal of All Containers)

If you need to effectively "stop" all containers managed by a Deployment without deleting the Deployment itself:

Example: Scale a Deployment to zero replicas

```
kubectl scale deployment my-app-deployment --replicas=0
```

What this does: Terminates all running Pods of the Deployment. No containers run. The Deployment object itself, its configuration, ConfigMaps, and Secrets all remain intact.

Example: Restore by scaling back up

```
kubectl scale deployment my-app-deployment --replicas=3
```

This pattern is useful for:

- Maintenance windows where you need all containers stopped temporarily.

- Debugging a Deployment without traffic hitting it.
- Cost-saving in non-production environments.

Example: Scale down all Deployments in a namespace to zero (maintenance mode)

```
kubectl get deployments -n my-namespace -o name | \
  xargs -I {} kubectl scale {} --replicas=0 -n my-namespace
```

23.10 Troubleshooting Pod Modification Errors

Error: cannot update pod spec field

```
$ kubectl edit pod my-pod
# After saving:
pods "my-pod" was not valid:
 * spec: Forbidden: pod updates may not change fields other than
   `spec.containers[*].image`, `spec.initContainers[*].image`,
   `spec.activeDeadlineSeconds`, `spec.tolerations` (only additions),
   `spec.terminationGracePeriodSeconds`

# You tried to change an immutable field (command, ports, volumes, etc.)
# Solution: export, edit, delete, recreate
kubectl get pod my-pod -o yaml > pod.yaml
# Edit pod.yaml - make your changes
kubectl delete pod my-pod
kubectl apply -f pod.yaml
```

Updating a Deployment's container image (zero-downtime)

```
# Recommended: set image command (triggers rolling update)
kubectl set image deployment/web app=nginx:1.25.3

# Watch the rolling update
kubectl rollout status deployment/web
# Waiting for deployment "web" rollout to finish:
# 1 out of 3 new replicas have been updated...
# 2 out of 3 new replicas have been updated...
# 3 out of 3 new replicas have been updated...
# deployment "web" successfully rolled out

# Watch pods during update
kubectl get pods -l app=web -w
# NAME            READY   STATUS              RESTARTS   AGE
# web-abc123      1/1     Running             0          2m
# web-def456      1/1     Running             0          2m
# web-ghi789      0/1     ContainerCreating   0          5s    ← new
# web-abc123      1/1     Terminating         0          2m    ← old
# web-jkl012      0/1     ContainerCreating   0          5s    ← new

# Undo if something goes wrong
kubectl rollout undo deployment/web

# Undo to specific revision
kubectl rollout history deployment/web
kubectl rollout undo deployment/web --to-revision=3
```

Forcing a Pod restart without changing its spec

```
# Method 1: Delete the Pod (Deployment recreates it)
kubectl delete pod -l app=my-app  # Deletes all matching pods (recreated by deployment)

# Method 2: Add an annotation to trigger rolling restart
kubectl rollout restart deployment/my-app
# This adds a restartedAt annotation to the pod template, triggering a rolling update

# Method 3: Scale to 0 and back up (brief downtime)
kubectl scale deployment my-app --replicas=0
kubectl scale deployment my-app --replicas=3

# Method 4: For StatefulSet
kubectl rollout restart statefulset/my-db
```

23.11 Advanced: Patching Running Deployments

```
# Add an environment variable to a running deployment
kubectl set env deployment/my-app NEW_VAR=new-value

# Remove an environment variable
kubectl set env deployment/my-app NEW_VAR-

# Add a resource limit
kubectl set resources deployment/my-app \
  --containers=app \
  --limits=cpu=500m,memory=256Mi \
  --requests=cpu=250m,memory=128Mi

# Update with a JSON patch (surgical change)
kubectl patch deployment my-app \
  --type=json \
  -p='[{"op":"replace","path":"/spec/template/spec/containers/0/image","value":"nginx:1.25"}]'

# Strategic merge patch (merge-patch, easier syntax)
kubectl patch deployment my-app --type=strategic -p='
spec:
  template:
    spec:
      containers:
      - name: app
        env:
        - name: LOG_LEVEL
          value: debug
'

# Add a volume mount to existing deployment
kubectl patch deployment my-app --type=strategic -p='
spec:
  template:
    spec:
      volumes:
      - name: tmp-cache
        emptyDir: {}
      containers:
      - name: app
        volumeMounts:
        - name: tmp-cache
          mountPath: /tmp/cache
'
```

Chapter 24 — On-the-Spot Test Container Creation and Deletion

24.1 The Concept of Ephemeral Test Containers

One of the most powerful patterns in Kubernetes debugging is spinning up a temporary, throw-away container inside the cluster to perform tests — send HTTP requests, ping services, resolve DNS names, trace routes — and then destroying it when done.

This approach is superior to running tests from your local machine because:

- **Same network namespace:** The container runs inside the cluster and has access to Services, ClusterIPs, and internal DNS that are not reachable from outside.
- **Same RBAC/network policies:** You test with the actual network policies in effect.
- **Clean environment:** The container is isolated and disposable.

24.2 The `kubectl run --rm -it` Pattern

The most common pattern uses `kubectl run` with the `--rm` flag (delete on exit) and `-it` (interactive terminal).

Basic Syntax

```
kubectl run <pod-name> \
  --image=<image> \
  --rm \
  -it \
  --restart=Never \
  -- <command>
```

Flag explanations:

Flag	Purpose
`--rm`	Automatically delete the Pod when the container exits
`-it`	Interactive terminal (stdin + TTY)
`--restart=Never`	Do not restart the container (run once and exit)
`--`	Separator: everything after this is the command to run in the container

24.3 Creating a Busybox Test Pod for Network Utilities

`busybox` is an extremely small image (around 1–5 MB) that packages dozens of UNIX utilities into a single binary. It includes `ping`, `nslookup`, `wget`, `telnet`, `nc` (netcat), and more.

Example: Spawn an interactive busybox shell

```
kubectl run test-shell \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- /bin/sh
```

What this does: Creates a Pod named `test-shell` running busybox, opens a shell inside it. When you type `exit`, Kubernetes automatically deletes the Pod. You now have a shell inside your cluster from which you can run any network test.

Example: Run a one-shot ping and destroy

```
kubectl run ping-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- ping -c 4 google.com
```

What this does: Creates a Pod, pings `google.com` 4 times, prints the results, then the Pod is automatically deleted. The `-c 4` flag limits ping to 4 packets.

Example: Ping an internal Kubernetes Service by its DNS name

```
kubectl run ping-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- ping -c 4 my-service.my-namespace.svc.cluster.local
```

What this does: Tests connectivity to a Kubernetes Service using its fully-qualified domain name (FQDN). This also indirectly tests that CoreDNS is resolving service names correctly.

24.4 Creating a curl-Capable Test Container

`busybox`'s `wget` is limited compared to `curl`. For full HTTP testing, use `curlimages/curl` or `nicolaka/netshoot`.

Example: Run a single curl request and destroy the Pod

```
kubectl run curl-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -s http://my-service.my-namespace.svc.cluster.local
```

What this does: Creates a Pod using the lightweight `curlimages/curl` image, runs `curl` against an internal Service endpoint, prints the HTTP response to your terminal, then deletes the Pod.

Example: Test an internal service with verbose output

```
kubectl run curl-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -v http://my-service:8080/health
```

What the `-v` *flag does:* Enables verbose output, showing the full request headers, response headers, TLS handshake details (if HTTPS), and response body. Essential for debugging HTTP issues.

Example: Test with custom headers and POST body

```
kubectl run curl-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -X POST \
     -H "Content-Type: application/json" \
     -H "Authorization: Bearer mytoken" \
     -d '{"key":"value"}' \
     http://my-api-service:8080/api/endpoint
```

What this does: Sends a POST request with a JSON body and authentication header to an internal API service. This is exactly how you would test a microservice without exposing it externally.

Example: Measure HTTP response time

```
kubectl run curl-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -o /dev/null -s -w \
     "Time to connect: %{time_connect}s\nTime total: %{time_total}s\nHTTP Status: %{http_code}\n" \
     http://my-service:8080
```

What this does: Suppresses the body output (`-o /dev/null`), runs silently (`-s`), and prints a formatted summary of timing metrics. Useful for baseline latency measurements.

24.5 Using netshoot — The Swiss Army Knife Image

`nicolaka/netshoot` is a container image purpose-built for network troubleshooting. It includes: `curl`, `wget`, `ping`, `traceroute`, `nmap`, `netstat`, `ss`, `dig`, `nslookup`, `iperf3`, `tcpdump`, `nftables`, `iptables`, `mtr`, `socat`, `jq`, and many more.

Example: Launch a netshoot interactive session

```
kubectl run netshoot \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- bash
```

Once inside, you have a full toolkit:

```
# Inside the netshoot container:

# Ping an internal service
ping -c 3 my-service.default.svc.cluster.local

# DNS lookup
dig my-service.default.svc.cluster.local

# Trace the network route
traceroute my-service.default.svc.cluster.local

# Scan open ports on a service
nmap -p 80,443,8080 my-service.default.svc.cluster.local

# Check if a TCP port is open (netcat)
nc -zv my-service 8080

# HTTP request with full details
curl -v http://my-service:8080/health

# Capture packets on the network interface
tcpdump -i eth0 -n host my-service
```

Example: Run a traceroute directly (non-interactive)

```
kubectl run trace-test \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- traceroute my-service.default.svc.cluster.local
```

Example: Check if a specific TCP port is open

```
kubectl run nc-test \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- nc -zv my-postgres-service 5432
```

What this does: Uses netcat (`nc`) in zero-I/O mode (`-z`) with verbose output (`-v`) to check if port 5432 (PostgreSQL) is accepting TCP connections on the target service. This is a pure connectivity test with no data transfer.

24.6 Running Test Pods in a Specific Namespace

By default, `kubectl run` creates the Pod in the current namespace. For service-to-service connectivity testing, you often need to run the test Pod in a specific namespace.

Example: Create a test Pod in a specific namespace

```
kubectl run curl-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -n production \
  -- curl -s http://api-service.production.svc.cluster.local:8080/health
```

What this does: Creates the test Pod in the `production` namespace, allowing you to test from that namespace's network perspective, respecting NetworkPolicies that apply to that namespace.

24.7 Attaching to a Running Test Pod

Sometimes you want to create the Pod without a command and attach to it separately.

Example: Create a long-running test Pod

```
kubectl run debug-pod \
  --image=nicolaka/netshoot \
  --restart=Never \
  -- sleep 3600
```

Example: Attach a shell to the running Pod

```
kubectl exec -it debug-pod -- bash
```

Example: Delete the Pod manually when done

```
kubectl delete pod debug-pod
```

24.8 Creating a Test Pod with Environment Variables

Example: Inject environment variables for testing

```
kubectl run test-pod \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  --env="API_URL=http://my-service:8080" \
  --env="API_TOKEN=mytoken" \
  -- sh -c 'curl -H "Authorization: Bearer $API_TOKEN" $API_URL/endpoint'
```

What this does: Injects environment variables into the test container and uses them in the command. This simulates how your application containers would consume these variables.

24.9 Verifying Pod Deletion After `--rm`

After the test Pod exits, confirm it was cleaned up:

```
kubectl get pods | grep test
```

If for some reason `--rm` failed (e.g., kubectl connection dropped), manually clean up:

```
# List and delete stuck test pods
kubectl delete pod curl-test ping-test netshoot debug-pod --ignore-not-found=true
```

The `--ignore-not-found=true` flag prevents an error if the Pod was already deleted.

Chapter 25 — Network Testing from Ephemeral Containers

25.1 Testing HTTP Connectivity

This section provides targeted, production-ready examples of network tests.

Example: Test HTTP and capture response code only

```
kubectl run http-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -o /dev/null -s -w "%{http_code}" http://my-service:8080/
```

Expected output: `200` — or whatever HTTP status code the service returned.

Example: Test HTTPS with certificate verification disabled (for internal self-signed certs)

```
kubectl run https-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -k -s -o /dev/null -w "%{http_code}" https://my-secure-service:8443/
```

What `-k` does: Disables SSL certificate verification. Useful when testing services with self-signed certificates in internal clusters. **Never use `-k` in production requests.**

Example: Follow HTTP redirects

```
kubectl run redirect-test \
  --image=curlimages/curl \
  --rm \
  -it \
  --restart=Never \
  -- curl -L -s -o /dev/null -w "%{http_code} final URL: %{url_effective}\n" \
     http://my-service:8080/old-path
```

What `-L` does: Follows HTTP redirects. The `%{url_effective}` variable shows the final URL after all redirects.

25.2 Testing TCP Connectivity

Example: TCP port check with netcat (nc)

```
kubectl run tcp-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- sh -c 'nc -zv my-db-service 5432 && echo "Port OPEN" || echo "Port CLOSED"'
```

What this does: Checks if TCP port 5432 is accepting connections. The `&&` and `||` logic prints a human-readable result.

Example: Test multiple ports in one run

```
kubectl run port-scanner \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- sh -c 'for port in 80 443 8080 9090; do
      nc -zv my-service $port 2>&1 && echo "  $port: OPEN" || echo "  $port: CLOSED"
    done'
```

What this does: Loops through a list of ports and tests each one. This is useful for quickly auditing which ports a service is actually listening on.

25.3 Testing Latency and Bandwidth

Example: Measure round-trip latency with ping

```
kubectl run latency-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- ping -c 10 my-service.default.svc.cluster.local
```

What to look for in the output: The min/avg/max/stddev line at the end. High stddev suggests network jitter. Packet loss percentage indicates connectivity issues.

Example: Bandwidth test with iperf3 (requires iperf3 server on target)

```
# First: Start iperf3 server in a target pod
kubectl exec my-target-pod -- iperf3 -s -D

# Then: Run iperf3 client test
kubectl run iperf-client \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- iperf3 -c my-target-service -p 5201 -t 10
```

25.4 Comprehensive Network Troubleshooting Matrix

```
Symptom                          → Check
──────────────────────────────────────────────────────────────

Pod can't reach Service          → Service selector matches Pod labels?
                                   kubectl get endpoints <svc>  (should not be empty)
                                   kube-proxy running on node?

Pod can't reach external URL     → DNS resolving? (nslookup from pod)
                                   NetworkPolicy blocking egress?
                                   Node has internet access? (curl from node)

Service accessible from pod      → Ingress rules correct?
but not from outside cluster       NodePort/LoadBalancer Service type?
                                   Cloud firewall rules?
                                   Ingress controller running?

Pod DNS not resolving            → CoreDNS running? (kubectl get pods -n kube-system)
                                   Pod's dnsPolicy correct?
                                   /etc/resolv.conf in pod correct?

Intermittent connection failures → Pod anti-affinity spreading?
                                    Resource limits causing restarts?
                                    HPA scaling creating new pods?
```

25.5 Advanced Network Testing Commands

```
# Complete connectivity test from a debug pod
kubectl run nettest --image=nicolaka/netshoot --rm -it --restart=Never -- bash

# Inside netshoot:
# Test DNS
nslookup kubernetes.default.svc.cluster.local
dig +short my-service.production.svc.cluster.local

# Test TCP port reachability
nc -zv my-service.production.svc.cluster.local 8080
# Connection to my-service.production.svc.cluster.local (10.96.45.23) 8080 port [tcp/*] succeeded!

# Traceroute (path through network)
traceroute my-service.production.svc.cluster.local

# HTTP deep inspection
curl -v http://my-service.production.svc.cluster.local/api/health
# Shows full request/response headers, TLS handshake, timing

# MTU/packet size test (detects fragmentation issues)
ping -M do -s 1450 my-service.production.svc.cluster.local

# Check node-to-node connectivity (run from a pod scheduled on each node)
for node_ip in 10.0.0.1 10.0.0.2 10.0.0.3; do
  echo -n "Node $node_ip: "
  nc -zv -w2 $node_ip 10250 2>&1 | grep -E "open|refused|timeout"
done

# Bandwidth test between pods
kubectl run iperf-server --image=networkstatic/iperf3 --restart=Never -- -s
kubectl run iperf-client --image=networkstatic/iperf3 --restart=Never -- \
  -c iperf-server.default.svc.cluster.local -t 10
```

25.6 Diagnosing NetworkPolicy Blocking

```
# Check if any NetworkPolicies exist in namespace
kubectl get networkpolicies -n my-namespace

# Simulate connection that's being blocked
kubectl run test-client --image=curlimages/curl --rm -it --restart=Never \
  -n source-namespace -- curl -v http://my-service.target-namespace.svc.cluster.local

# If blocked, examine NetworkPolicies in target namespace
kubectl get networkpolicies -n target-namespace -o yaml

# Check if the "deny-all" policy is in effect
kubectl get networkpolicies -n target-namespace | grep deny-all

# Trace policy matching (which policy matches this connection?)
# There's no built-in tool, but Cilium provides this:
# kubectl exec -n kube-system cilium-xxx -- cilium monitor --type drop
# For Calico:
# kubectl exec -n kube-system calico-node-xxx -- calicoctl policy match

# Temporarily disable NetworkPolicy for testing (never in production!)
kubectl delete networkpolicy deny-all -n target-namespace
curl -v http://my-service.target-namespace.svc.cluster.local
kubectl apply -f deny-all-policy.yaml  # Restore
```

Chapter 26 — CoreDNS: Testing, Diagnosing, and Verifying DNS Resolution

26.1 What is CoreDNS?

CoreDNS is the default DNS server in Kubernetes clusters (replacing kube-dns since Kubernetes 1.11). It runs as a Deployment in the `kube-system` namespace and every Pod in the cluster uses it as its DNS resolver.

CoreDNS is responsible for:

- Resolving Kubernetes Service names to ClusterIP addresses
- Resolving Pod DNS names
- Forwarding external DNS queries (e.g., `google.com`) to upstream resolvers
- Supporting custom DNS configurations via its `Corefile`

Without CoreDNS working correctly, Pods cannot communicate with each other using service names,
which breaks virtually all microservice architectures.

26.2 Kubernetes DNS Naming Conventions

Understanding the naming patterns is essential for DNS testing.

Service DNS names:

Pattern	Resolves to	Example
`<service>`	ClusterIP (same namespace)	`my-service`
`<service>.<namespace>`	ClusterIP	`my-service.production`
`<service>.<namespace>.svc`	ClusterIP	`my-service.production.svc`
`<service>.<namespace>.svc.cluster.local`	ClusterIP (FQDN)	`my-service.production.svc.cluster.local`

Pod DNS names (when `hostname` is set):

```
<pod-ip-dashes>.<namespace>.pod.cluster.local
# e.g.: 10-0-1-5.default.pod.cluster.local
```

26.3 Checking CoreDNS Status

Example: View the CoreDNS Deployment

```
kubectl get deployment coredns -n kube-system
```

What to look for: The `READY` column should show all replicas ready (e.g., `2/2`). If it shows `0/2` or `1/2`, CoreDNS has a problem and service discovery will be impaired.

Example: View CoreDNS Pods

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

Example: Check CoreDNS logs for errors

```
kubectl logs -n kube-system -l k8s-app=kube-dns --all-containers=true
```

What to look for: Lines containing `SERVFAIL`, `REFUSED`, `error`, or repeated queries without responses. These indicate CoreDNS is receiving queries but failing to resolve them.

Example: Follow CoreDNS logs in real time while testing DNS

```
kubectl logs -f -n kube-system -l k8s-app=kube-dns
```

Pro tip: Open this in a separate terminal, then run your DNS tests. You can see CoreDNS receiving and processing each query in real time.

26.4 Viewing the CoreDNS Configuration (Corefile)

CoreDNS is configured via a ConfigMap called `coredns` in the `kube-system` namespace.

Example: View the CoreDNS Corefile

```
kubectl get configmap coredns -n kube-system -o yaml
```

What to look for in the output:

```
data:
  Corefile: |
    .:53 {
        errors
        health {
           lameduck 5s
        }
        ready
        kubernetes cluster.local in-addr.arpa ip6.arpa {
           pods insecure
           fallthrough in-addr.arpa ip6.arpa
           ttl 30
        }
        prometheus :9153
        forward . /etc/resolv.conf {
           max_concurrent 1000
        }
        cache 30
        loop
        reload
        loadbalance
    }
```

Key configuration blocks:

- `kubernetes cluster.local`: Handles resolution for `.cluster.local` names (all Kubernetes services)
- `forward . /etc/resolv.conf`: Forwards external DNS queries to upstream (node's resolver)

- `cache 30`: Caches responses for 30 seconds
- `loop`: Detects and prevents DNS query loops

26.5 DNS Resolution Testing with nslookup

`nslookup` is included in `busybox` and is the quickest tool for DNS testing.

Example: Test resolution of a Kubernetes Service

```
kubectl run dns-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- nslookup kubernetes.default.svc.cluster.local
```

What this does: Resolves the `kubernetes` Service in the `default` namespace (the Kubernetes API server Service). This is a built-in Service that always exists and is the best sanity check for CoreDNS health.

Expected output:

```
Server:         10.96.0.10
Address:        10.96.0.10:53

Name:   kubernetes.default.svc.cluster.local
Address: 10.96.0.1
```

The `Server` line shows the DNS server being used (CoreDNS ClusterIP). The `Address` line shows the ClusterIP of the kubernetes Service. If this fails, CoreDNS is broken.

Example: Test resolution of your own service

```
kubectl run dns-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- nslookup my-service.my-namespace.svc.cluster.local
```

Example: Test short-name resolution (relies on search domains)

```
kubectl run dns-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -n my-namespace \
  -- nslookup my-service
```

What this does: Tests whether `my-service` resolves using the default search domains in the namespace. Kubernetes configures each Pod's `/etc/resolv.conf` with:

```
search <namespace>.svc.cluster.local svc.cluster.local cluster.local
```

This means `nslookup my-service` will try `my-service.my-namespace.svc.cluster.local` first.

26.6 DNS Resolution Testing with dig

`dig` (Domain Information Groper) provides much more detailed DNS query information than `nslookup`.
It is available in the `nicolaka/netshoot` image.

Example: Query a service with dig

```
kubectl run dig-test \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- dig my-service.default.svc.cluster.local
```

What the output shows:

- `QUESTION SECTION`: What was queried
- `ANSWER SECTION`: The resolved IP address(es) and TTL
- `Query time`: How long CoreDNS took to respond (milliseconds)
- `SERVER`: Which DNS server was used

Example: Query CoreDNS directly (bypass the cache)

```
kubectl run dig-test \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- dig @10.96.0.10 my-service.default.svc.cluster.local
```

What this does: The `@10.96.0.10` specifies querying the CoreDNS ClusterIP directly. Replace `10.96.0.10` with your CoreDNS ClusterIP (found with `kubectl get svc kube-dns -n kube-system`).
This bypasses any local resolver and tests CoreDNS directly.

Example: Perform a reverse DNS lookup (IP to hostname)

```
kubectl run dig-test \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- dig -x 10.96.0.1
```

What this does: Queries for the PTR record of a ClusterIP, resolving it back to its hostname.
This tests reverse DNS resolution, which is used by some logging and monitoring tools.

Example: Query only the answer section (clean output)

```
kubectl run dig-test \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- dig +short my-service.default.svc.cluster.local
```

What this does: The `+short` flag suppresses all output except the answer — just the IP address. Useful for scripted DNS checks.

26.7 Inspecting a Pod's DNS Configuration

Every Pod has `/etc/resolv.conf` configured by Kubernetes. Examining it tells you exactly how DNS resolution will work for that Pod.

Example: View the resolv.conf inside a running Pod

```
kubectl exec my-app-pod -- cat /etc/resolv.conf
```

Typical output:

```
nameserver 10.96.0.10
search default.svc.cluster.local svc.cluster.local cluster.local
options ndots:5
```

What each line means:

- `nameserver 10.96.0.10` — CoreDNS ClusterIP. All DNS queries go here.
- `search default.svc.cluster.local svc.cluster.local cluster.local` — Search domains appended to short names. When you query `my-service`, the resolver tries each search domain in order.
- `options ndots:5` — If a name has fewer than 5 dots, search domains are tried before treating it as a FQDN. This affects performance because external names like `api.external.com` will first be tried with search domains appended (generating failed lookups) before being resolved as-is.

Example: View resolv.conf in a test container

```
kubectl run dns-inspect \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- cat /etc/resolv.conf
```

26.8 Testing External DNS Resolution

Example: Verify external DNS works from inside the cluster

```
kubectl run dns-external-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- nslookup google.com
```

What this does: Tests that CoreDNS can forward external queries upstream. If internal DNS works but external DNS fails, the issue is in the `forward` block of the CoreDNS Corefile, or the node's upstream resolver is unreachable.

Example: Compare internal vs external resolution

```
kubectl run dns-compare \
  --image=nicolaka/netshoot \
  --rm \
  -it \
  --restart=Never \
  -- sh -c '
    echo "=== Internal Service ==="
    dig +short kubernetes.default.svc.cluster.local

    echo "=== External Domain ==="
    dig +short google.com

    echo "=== CoreDNS Info ==="
    cat /etc/resolv.conf
  '
```

What this does: Runs three checks in sequence in a single Pod, comparing internal and external resolution side by side.

26.9 Diagnosing CoreDNS with 5-second timeout test

Example: Test DNS with explicit timeout

```
kubectl run dns-timeout-test \
  --image=busybox \
  --rm \
  -it \
  --restart=Never \
  -- sh -c 'nslookup -timeout=5 my-service.default.svc.cluster.local && echo "DNS OK" || echo "DNS
FAILED"'
```

What this does: If CoreDNS does not respond within 5 seconds, the command fails with "DNS FAILED".
Under normal conditions, DNS resolution inside a cluster should take under 5 milliseconds.

26.10 Testing CoreDNS with dnsutils / dnsperf

Example: Launch a dnsutils container for full DNS toolkit

```
kubectl run dnsutils \
  --image=gcr.io/kubernetes-e2e-test-images/dnsutils:1.3 \
  --restart=Never \
  -- sleep 3600
```

```
kubectl exec -it dnsutils -- bash
```

Inside dnsutils, you have access to:

```
# Full dig with all details
dig kubernetes.default.svc.cluster.local

# nslookup with specific DNS server
nslookup my-service 10.96.0.10

# Check search domain expansion
host my-service

# Test TCP DNS (instead of UDP)
dig +tcp my-service.default.svc.cluster.local
```

Example: Clean up dnsutils pod when done

```
kubectl delete pod dnsutils
```

26.11 CoreDNS Metrics and Health Endpoints

Example: Port-forward to CoreDNS health endpoint

```
# Get a CoreDNS pod name
COREDNS_POD=$(kubectl get pods -n kube-system -l k8s-app=kube-dns -o jsonpath='{.items[0].metadata.name}
')

# Forward the health port
kubectl port-forward -n kube-system pod/$COREDNS_POD 8080:8080
```

Then in another terminal:

```
curl http://localhost:8080/health
# Expected: OK

curl http://localhost:8080/ready
# Expected: OK
```

Example: Access CoreDNS Prometheus metrics

```
kubectl port-forward -n kube-system pod/$COREDNS_POD 9153:9153
curl http://localhost:9153/metrics | grep coredns_dns_requests_total
```

What this shows: The total number of DNS queries CoreDNS has received and processed. Look for high `SERVFAIL` counts which indicate resolution failures.

26.12 Diagnosing CoreDNS Failures

Symptom: DNS resolution intermittently fails

```
# Check CoreDNS pods for restarts
kubectl get pods -n kube-system -l k8s-app=kube-dns
# NAME             READY   STATUS    RESTARTS   AGE
# coredns-abc123   1/1     Running   12         2d    ← 12 restarts is suspicious!

kubectl logs coredns-abc123 -n kube-system | tail -30 | grep -i "error\|fail\|panic"

# CoreDNS OOMKilled (memory exhausted by DNS cache)
kubectl describe pod coredns-abc123 -n kube-system | grep -A 5 "Last State"
# Last State: Terminated
#   Reason: OOMKilled
# Fix: increase CoreDNS memory limit:
kubectl -n kube-system get deployment coredns -o yaml | grep -A 5 resources
kubectl -n kube-system set resources deployment coredns \
  --containers=coredns --limits=memory=256Mi --requests=memory=128Mi
```

Symptom: NXDOMAIN for in-cluster services

```
# Test DNS from a pod
kubectl run dnstest --image=busybox:1.35 --rm -it --restart=Never -- \
  nslookup my-service.production.svc.cluster.local

# Server:    10.96.0.10
# Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local
# nslookup: can't resolve 'my-service.production.svc.cluster.local'

# Step 1: Verify the service actually exists
kubectl get svc my-service -n production
# Error: services "my-service" not found  ← That's the problem!

# Step 2: Check if DNS works for other services
kubectl run dnstest --image=busybox:1.35 --rm -it --restart=Never -- \
  nslookup kubernetes.default.svc.cluster.local
# This should always work. If it doesn't, CoreDNS is broken.

# Step 3: Check cluster domain
kubectl get configmap coredns -n kube-system -o yaml | grep "cluster.local"
# Ensure your pod's /etc/resolv.conf uses the same domain:
kubectl exec my-pod -- cat /etc/resolv.conf
# search default.svc.cluster.local svc.cluster.local cluster.local
```

Symptom: External DNS not resolving from pods

```
# External DNS (e.g., google.com) fails but internal works
kubectl run dnstest --image=busybox:1.35 --rm -it --restart=Never -- sh
> nslookup kubernetes.default  # Works
> nslookup google.com          # Fails

# Check CoreDNS forward configuration
kubectl get configmap coredns -n kube-system -o yaml
# Corefile should have:
# .:53 {
#     ...
#     forward . /etc/resolv.conf  ← this forwards to upstream DNS
# }

# If CoreDNS can't reach upstream DNS, check node's resolv.conf:
cat /etc/resolv.conf  # On the node
# Should have a valid nameserver

# Test from CoreDNS pod directly
kubectl exec -n kube-system coredns-abc123 -- nslookup google.com 8.8.8.8
```

Customizing CoreDNS for common patterns

```
# Add a custom DNS entry for a specific host
# Edit CoreDNS configmap:
apiVersion: v1
kind: ConfigMap
metadata:
  name: coredns
  namespace: kube-system
data:
  Corefile: |
    .:53 {
        errors
        health { lameduck 5s }
        ready
        kubernetes cluster.local in-addr.arpa ip6.arpa {
           pods insecure
           fallthrough in-addr.arpa ip6.arpa
        }
        hosts {                           # Custom host entries
          10.1.2.3   mylegacyserver
          10.1.2.4   another-host.local
          fallthrough
        }
        rewrite name exact \
          old-service.example.com \
          new-service.production.svc.cluster.local   # Rewrite DNS
        prometheus :9153
        forward . /etc/resolv.conf {
          max_concurrent 1000
        }
        cache 30
        loop
        reload
        loadbalance
    }
```

```
# After editing CoreDNS ConfigMap, it reloads automatically within 30s
# Force immediate reload:
kubectl rollout restart deployment/coredns -n kube-system

# Test your custom DNS entry
kubectl run dnstest --image=busybox:1.35 --rm -it --restart=Never -- \
  nslookup mylegacyserver
```

Chapter 27 — Advanced Debugging with Ephemeral Containers

27.1 What Are Ephemeral Containers?

Kubernetes 1.25+ introduced **Ephemeral Containers** as a stable (GA) feature. Unlike regular containers
in a Pod spec, ephemeral containers are temporary — they share the Pod's namespaces (network, PID,
etc.) but can be added to a running Pod without restarting it.

This solves a critical problem: distroless and minimal images have no shell and no debugging tools. An ephemeral container can be injected into the same Pod at runtime to debug it.

27.2 Adding an Ephemeral Container to a Running Pod

Example: Inject a debug container into a running Pod

```
kubectl debug -it my-app-pod \
  --image=nicolaka/netshoot \
  --target=app \
  -- bash
```

What this does: Adds a `netshoot` container as an ephemeral container to the running `my-app-pod`. The `--target=app` flag targets the process namespace of the `app` container (you can see its processes using `ps aux` from inside the debug container). When you exit, the Pod keeps running but the ephemeral container terminates.

Example: Debug a distroless container (no shell)

```
kubectl debug -it my-distroless-pod \
  --image=busybox \
  --target=app \
  -- sh
```

What this does: Since distroless images have no shell, this injects a busybox container into the same Pod namespace, giving you a shell that shares the network stack with the distroless container.

27.3 Creating a Debug Copy of a Pod

If you need to modify the pod spec for debugging (e.g., change the entrypoint, add capabilities), you can create a copy of the Pod with modifications.

Example: Create a debugging copy with a shell replacing the entrypoint

```
kubectl debug my-app-pod \
  -it \
  --copy-to=my-app-debug \
  --image=busybox \
  -- sh
```

What this does: Creates a new Pod named `my-app-debug` that is a copy of `my-app-pod` but with the container image replaced by busybox. Useful when you want to explore the same volume mounts and environment variables with a different image.

Example: Clean up the debug copy Pod

```
kubectl delete pod my-app-debug
```

27.4 Running a Debug Node Pod (Privileged)

To debug node-level networking or filesystem issues:

Example: Spawn a privileged Pod on a specific node

```
kubectl debug node/my-node-name \
  -it \
  --image=nicolaka/netshoot
```

What this does: Creates a privileged Pod on the specified node, giving you access to the node's host filesystem mounted at `/host`, host network, and host PID namespace. Use with care.

27.2 Systematic Node Troubleshooting

When a node shows `NotReady`, use this systematic approach:

```
# Step 1: Check node status and conditions
kubectl get nodes
kubectl describe node <problem-node> | grep -A 20 "Conditions:"

# Common NotReady conditions and their meanings:
# Condition: Ready=False
#   Reason: KubeletNotReady, KubeletHasInsufficientMemory, KubeletHasDiskPressure

# Step 2: Check kubelet status on the node (requires SSH or node access)
ssh worker-1
sudo systemctl status kubelet
sudo journalctl -u kubelet -n 50 --no-pager

# Step 3: Common kubelet failure causes
# - Certificate expired: "failed to load certificates" in logs
# - Container runtime not running: "Error getting node info... dial unix /run/containerd/
containerd.sock"
# - Disk full: "disk space below threshold"

# Step 4: Check container runtime
sudo systemctl status containerd
sudo crictl ps

# Step 5: Check system resources
free -h          # Memory
df -h            # Disk
top              # CPU
```

27.3 Troubleshooting Control Plane Components

```
# Check all control plane component status
kubectl get pods -n kube-system
kubectl get componentstatuses  # Legacy but useful: shows etcd, controller-manager, scheduler health

# API server is down — connect to control-plane node directly
ssh control-plane-node
sudo crictl logs $(sudo crictl ps -q --name=kube-apiserver)
cat /etc/kubernetes/manifests/kube-apiserver.yaml | grep -- '--'

# kube-controller-manager issues
kubectl -n kube-system logs kube-controller-manager-<node> --tail=50 | grep -i error

# kube-scheduler not scheduling Pods (Pods stuck in Pending)
kubectl -n kube-system logs kube-scheduler-<node> --tail=50
kubectl describe pod <pending-pod> | grep -A 10 "Events:"
# "0/3 nodes are available: 3 Insufficient cpu" → needs more resources
# "0/3 nodes are available: 3 node(s) had untolerated taint" → add toleration
```

27.4 Troubleshooting Services and Networking

```
# Pod can't reach a Service by DNS?
# Step 1: Test DNS resolution
kubectl run dnstest --image=busybox:1.35 --rm -it --restart=Never -- \
  nslookup my-service.staging.svc.cluster.local

# Step 2: Test Service directly by ClusterIP
kubectl get svc my-service -n staging -o jsonpath='{.spec.clusterIP}'
kubectl run curltest --image=curlimages/curl --rm -it --restart=Never -- \
  curl http://10.96.100.5:80

# Step 3: Check Endpoints — if empty, label selector is wrong
kubectl get endpoints my-service -n staging
# If "my-service   <none>" → no Pods match the Service selector

# Step 4: Verify Pod labels match Service selector
kubectl get svc my-service -n staging -o jsonpath='{.spec.selector}'
# {"app":"frontend"}
kubectl get pods -n staging -l app=frontend   # Should return Pods

# Step 5: Check if kube-proxy is running
kubectl -n kube-system get pods -l k8s-app=kube-proxy
kubectl -n kube-system logs kube-proxy-<name> --tail=20

# Step 6: Verify iptables rules exist (on a node)
sudo iptables -t nat -L KUBE-SERVICES | grep my-service
```

27.5 Cluster-Level Troubleshooting Checklist

A systematic checklist for the CKA exam troubleshooting scenarios:

```
Pod stuck in Pending:
□ Describe pod → check Events section
□ Check if node has enough CPU/memory: kubectl describe nodes | grep "Allocated"
□ Check for taints: kubectl describe node | grep Taints
□ Check PVC is bound (if using storage): kubectl get pvc
□ Check resource quotas: kubectl describe resourcequota

Pod in CrashLoopBackOff:
□ kubectl logs <pod> --previous
□ kubectl describe pod → check Last State exit code
□ Check if command/args are correct
□ Check if readiness probe is misconfigured
□ Check if env vars / ConfigMaps / Secrets are correct

Pod stuck in ContainerCreating:
□ kubectl describe pod → Events: "failed to pull image"
□ Check imagePullSecret if private registry
□ Check PVC is available
□ Check CNI plugin is running

Service not accessible:
□ kubectl get endpoints <service> → should not be <none>
□ kubectl get pods -l <service-selector> → pods must be Running
□ Test from within cluster with curl/wget
□ Check NetworkPolicy: kubectl get networkpolicy -n <namespace>
```

Chapter 28 — Namespace-Aware Commands and Context Management

28.1 Working with Namespaces

All `kubectl` commands operate in a namespace. If you don't specify one, kubectl uses the namespace configured in your current context (usually `default`).

Example: Run a command in a specific namespace

```
kubectl exec -it my-pod -n production -- bash
kubectl logs my-pod -n staging
kubectl run test -n development --image=busybox --rm -it --restart=Never -- sh
```

Example: List all namespaces

```
kubectl get namespaces
```

Example: View Pods in all namespaces

```
kubectl get pods -A
# -A is short for --all-namespaces
```

28.2 Switching Context and Namespace

Example: View current context

```
kubectl config current-context
```

Example: Set a default namespace for the current context

```
kubectl config set-context --current --namespace=production
```

What this does: Sets the default namespace for all subsequent commands without needing `-n` every time. This is a persistent setting stored in your kubeconfig.

Example: View all contexts

```
kubectl config get-contexts
```

Example: Switch to a different cluster context

```
kubectl config use-context my-production-cluster
```

28.3 Executing Across All Pods in a Deployment

Example: Execute a command in every Pod of a Deployment

```
for pod in $(kubectl get pods -l app=my-app -o jsonpath='{.items[*].metadata.name}'); do
  echo "=== $pod ==="
  kubectl exec $pod -- cat /etc/hostname
done
```

What this does: Loops through all Pods matching the label `app=my-app` and runs a command in each one. Useful for checking consistency across all replicas.

28.4 RBAC Scoped to Namespaces

RBAC lets you give different teams different permissions per namespace:

```
# Create a namespace per team
kubectl create namespace team-alpha
kubectl create namespace team-beta

# Create a Role in team-alpha that allows managing Pods and Deployments
kubectl create role developer-role \
  --verb=get,list,watch,create,update,patch,delete \
  --resource=pods,deployments,services,configmaps \
  -n team-alpha

# Bind a user to the role in that namespace
kubectl create rolebinding alice-binding \
  --role=developer-role \
  --user=alice \
  -n team-alpha

# Alice can work in team-alpha but has no access to team-beta
kubectl auth can-i create pods --as=alice -n team-alpha   # yes
kubectl auth can-i create pods --as=alice -n team-beta    # no
kubectl auth can-i get pods --as=alice -n kube-system     # no
```

28.5 NetworkPolicy for Namespace Isolation

By default, all Pods can communicate with all other Pods in the cluster (no network isolation). NetworkPolicies restrict this:

```yaml
# Deny all ingress to staging namespace by default
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-ingress
  namespace: staging
spec:
  podSelector: {}      # Applies to ALL pods in namespace
  policyTypes:
  - Ingress             # Restricts incoming traffic
  # No ingress rules = deny all
---
# Allow staging to receive traffic only from production namespace
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-production
  namespace: staging
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: production
---
# Allow specific pod to pod communication across namespaces
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-api-from-frontend
  namespace: backend
spec:
  podSelector:
    matchLabels:
      app: api
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          team: frontend
      podSelector:
        matchLabels:
          app: frontend
    ports:
    - port: 8080
```

```bash
# Label a namespace (required for namespaceSelector matching)
kubectl label namespace production kubernetes.io/metadata.name=production

# Test network policy
kubectl run test --image=busybox -n staging --rm -it --restart=Never -- \
  wget -qO- http://api-service.backend.svc.cluster.local:8080

# List all network policies
kubectl get networkpolicies --all-namespaces
```

28.6 Cross-Namespace Service Access

Services are DNS-addressable from other namespaces:

```
# DNS format: <service-name>.<namespace>.svc.cluster.local
# From any Pod in any namespace:
curl http://my-service.staging.svc.cluster.local:8080

# Within same namespace (short form):
curl http://my-service:8080

# Full FQDN (from outside cluster or debugging):
# my-service.staging.svc.cluster.local
```

You can also create a Service that points to a service in another namespace using ExternalName:

```
# In the "production" namespace, create a service that proxies to staging
apiVersion: v1
kind: Service
metadata:
  name: staging-api
  namespace: production
spec:
  type: ExternalName
  externalName: api.staging.svc.cluster.local
  # Now production Pods can use: curl http://staging-api:8080
  # and it resolves to api.staging.svc.cluster.local
```

28.7 Namespace Lifecycle and Cleanup

```
# Delete a namespace (deletes ALL resources in it)
kubectl delete namespace old-staging

# This is ASYNC — namespace goes into Terminating state
kubectl get namespace old-staging
# NAME          STATUS       AGE
# old-staging   Terminating  2m

# If stuck in Terminating, it may have finalizers
kubectl get namespace old-staging -o json | grep finalizers

# Force remove stuck namespace (use carefully)
kubectl proxy &
curl -k -H "Content-Type: application/json" -X PUT \
  --data-binary '{"kind":"Namespace","apiVersion":"v1","metadata":{"name":"old-staging"},"spec":
{"finalizers":[]}}' \
  http://127.0.0.1:8001/api/v1/namespaces/old-staging/finalize

# List all resources in a namespace before deleting
kubectl get all -n old-staging
kubectl api-resources --verbs=list --namespaced -o name | \
  xargs -I {} kubectl get {} -n old-staging --ignore-not-found
```

Chapter 29 — kubectl Kustomize: Configuration Management at Scale

29.1 What is Kustomize?

Kustomize is a Kubernetes-native configuration management tool that lets you customize raw YAML
manifests without templates, without modifying original files, and without a separate rendering engine. It was integrated directly into `kubectl` starting with version 1.14, meaning you need no additional tools installed — `kubectl kustomize` is available on any modern Kubernetes installation.

The philosophy of Kustomize is fundamentally different from Helm:

Aspect	Kustomize	Helm
Approach	Declarative overlay patching	Templating engine with Go templates
Learning curve	Lower (pure YAML)	Higher (template syntax)
Original files	Left unmodified	Must be written as templates
Multiple environments	Overlays on a base	Value files (`values.yaml`)
Kubernetes-native	Yes (built into kubectl)	Separate binary required

Kustomize works by:

1. You define a **base** — the canonical YAML for your resources.
2. You create **overlays** for each environment (dev, staging, production).
3. Overlays use **patches**, **transformers**, and **generators** to apply targeted changes on top of the base, without touching the base files.
4. `kubectl kustomize` renders the final merged output, which you pipe to `kubectl apply`.

29.2 The kustomization.yaml File

Every Kustomize directory must contain a file named exactly `kustomization.yaml`. This file is the manifest of the Kustomize configuration — it lists what resources to include, what patches to apply, what transformations to make, and what generators to run.

Minimal kustomization.yaml:

```
# kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - deployment.yaml
  - service.yaml
```

What this does: Declares that this kustomization manages two resources: `deployment.yaml` and `service.yaml`. Running `kubectl kustomize .` in this directory outputs both files merged and rendered.

29.3 Directory Structure: Base and Overlays

The standard Kustomize project layout separates a **base** (shared configuration) from **overlays** (environment-specific customizations):

```
my-app/
├── base/
│   ├── kustomization.yaml
│   ├── deployment.yaml
│   ├── service.yaml
│   └── configmap.yaml
└── overlays/
    ├── development/
    │   ├── kustomization.yaml
    │   └── patch-replicas.yaml
    ├── staging/
    │   ├── kustomization.yaml
    │   └── patch-replicas.yaml
    └── production/
        ├── kustomization.yaml
        ├── patch-replicas.yaml
        └── patch-resources.yaml
```

What this structure achieves: The `base/` directory contains the complete, environment-agnostic Kubernetes manifests. Each overlay directory contains a `kustomization.yaml` that references the base and applies environment-specific patches. The base files are never modified.

29.4 Building and Applying Kustomizations

The Two Core Commands

Command 1: kubectl kustomize — render without applying

```
kubectl kustomize <directory>
```

What this does: Reads all resources and patches in the kustomization directory, processes them, and outputs the final merged YAML to stdout. Nothing is sent to the cluster. This is essential for reviewing what will be applied before actually applying it.

```
# Render the base
kubectl kustomize ./base

# Render a specific overlay
kubectl kustomize ./overlays/production

# Save rendered output to a file
kubectl kustomize ./overlays/production > rendered-production.yaml

# Review the output (pipe to less for large outputs)
kubectl kustomize ./overlays/production | less
```

Command 2: kubectl apply -k — render and apply in one step

```
kubectl apply -k <directory>
```

What this does: The `-k` flag tells `kubectl apply` to treat the directory as a Kustomize directory instead of a plain YAML file. It renders the kustomization and immediately applies it to the cluster. Equivalent to `kubectl kustomize <dir> | kubectl apply -f -`.

```
# Apply base directly
kubectl apply -k ./base

# Apply production overlay
kubectl apply -k ./overlays/production

# Dry run to see what would change without applying
kubectl apply -k ./overlays/production --dry-run=client

# Dry run against the server (validates against API schema)
kubectl apply -k ./overlays/production --dry-run=server
```

Command 3: kubectl delete -k — delete all resources in a kustomization

```
kubectl delete -k ./overlays/production
```

What this does: Deletes all resources that the kustomization manages. Useful for tearing down an entire environment cleanly.

29.5 Working Example: Base Configuration

Let us build a complete working example from the ground up.

base/deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  labels:
    app: my-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
      - name: app
        image: my-app:latest
        ports:
        - containerPort: 8080
        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"
          limits:
            cpu: "200m"
            memory: "256Mi"
        env:
        - name: LOG_LEVEL
          value: "info"
```

base/service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: my-app
spec:
  selector:
    app: my-app
  ports:
  - port: 80
    targetPort: 8080
  type: ClusterIP
```

base/kustomization.yaml

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - deployment.yaml
  - service.yaml
```

```bash
# Verify the base renders cleanly
kubectl kustomize ./base
```

29.6 Overlays: Customizing for Each Environment

Development Overlay

overlays/development/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

# Reference the base
resources:
  - ../../base

# Add a namespace prefix to all resources
namespace: development

# Add labels to all resources
commonLabels:
  environment: development

# Override the image tag for development
images:
  - name: my-app
    newTag: dev-latest

# Apply patches
patches:
  - path: patch-replicas.yaml
```

overlays/development/patch-replicas.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 1        # Development only needs 1 replica
```

```
# Render the development overlay and inspect
kubectl kustomize ./overlays/development

# Apply to the cluster
kubectl apply -k ./overlays/development
```

Production Overlay

overlays/production/kustomization.yaml

```yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - ../../base

namespace: production

commonLabels:
  environment: production

images:
  - name: my-app
    newName: registry.company.com/my-app   # Different registry in prod
    newTag: v1.5.2                          # Pinned version tag

patches:
  - path: patch-replicas.yaml
  - path: patch-resources.yaml
  - path: patch-env.yaml
```

overlays/production/patch-replicas.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 5        # Production needs 5 replicas for HA
```

overlays/production/patch-resources.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  template:
    spec:
      containers:
      - name: app
        resources:
          requests:
            cpu: "500m"
            memory: "512Mi"
          limits:
            cpu: "1000m"
            memory: "1Gi"
```

overlays/production/patch-env.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  template:
    spec:
      containers:
      - name: app
        env:
        - name: LOG_LEVEL
          value: "warn"           # Less verbose logging in production
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: url
```

29.7 Kustomize Patches: Strategic Merge vs JSON 6902

Kustomize supports two types of patches, each with its own strengths.

Strategic Merge Patch

A strategic merge patch is a partial YAML document that is merged with the target resource. Kubernetes
uses the resource's schema to understand how to merge arrays (e.g., containers merge by `name` key, not by index position).

```yaml
# kustomization.yaml
patches:
  - path: my-patch.yaml
    target:
      kind: Deployment
      name: my-app
```

```yaml
# my-patch.yaml — add a new environment variable without replacing existing ones
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  template:
    spec:
      containers:
      - name: app
        env:
        - name: NEW_FEATURE_FLAG
          value: "enabled"
```

What this does: Because Kubernetes knows the `env` array merges by `name` key, this patch ADDS the new variable without removing the existing ones.

JSON 6902 Patch (Precise Surgical Operations)

JSON 6902 patches allow you to perform precise operations: add, remove, replace, copy, and move at
exact paths within the resource. They are more verbose but more explicit.

```
# kustomization.yaml — inline JSON 6902 patch
patches:
  - target:
      kind: Deployment
      name: my-app
    patch: |-
      - op: replace
        path: /spec/replicas
        value: 3
      - op: add
        path: /spec/template/spec/containers/0/env/-
        value:
          name: FEATURE_FLAG
          value: "true"
      - op: remove
        path: /spec/template/spec/containers/0/env/0
```

The `/spec/template/spec/containers/0/env/-` path: The `-` at the end is a JSON Pointer convention meaning "append to the end of the array". `/env/0` means the first element of the env array.

29.8 Transformers: Global Modifications

Transformers apply changes to ALL resources in a kustomization. They are declared directly in `kustomization.yaml` and are extremely powerful for applying consistent metadata across an entire set of resources.

namePrefix and nameSuffix

```
# kustomization.yaml
namePrefix: prod-        # All resource names get "prod-" prepended
nameSuffix: -v2          # All resource names get "-v2" appended
```

Result: A Deployment named `my-app` becomes `prod-my-app-v2`. This is commonly used to differentiate environments that share the same cluster.

commonLabels

```
# kustomization.yaml
commonLabels:
  app.kubernetes.io/managed-by: kustomize
  app.kubernetes.io/environment: production
  team: platform-engineering
```

What this does: Adds ALL listed labels to every resource AND to Pod template selectors. This means these labels appear on Deployments, Services, ConfigMaps, and all their managed Pods.

> ⚠️ ***Warning:*** `commonLabels` *also modifies* `spec.selector` *on Services and Deployments. If these selectors change on existing resources, Kubernetes may reject the update. Use* `commonAnnotations` *for purely informational metadata to avoid selector conflicts.*

commonAnnotations

```
# kustomization.yaml
commonAnnotations:
  team-contact: "platform@company.com"
  docs-url: "https://wiki.internal/my-app"
  last-deployed-by: "ci-pipeline"
```

What this does: Adds annotations to every resource without affecting selectors. Annotations are safe to add to existing resources.

namespace

```
# kustomization.yaml
namespace: production
```

What this does: Sets `metadata.namespace` on ALL resources in the kustomization. This is the most common use of transformers — ensuring all resources land in the correct namespace.

29.9 Generators: Creating ConfigMaps and Secrets from Files

Kustomize can automatically generate ConfigMaps and Secrets from files and literals, with automatic
hash-based name suffixes. This is one of Kustomize's most powerful features for configuration management.

ConfigMap Generator

```
# kustomization.yaml
configMapGenerator:
  - name: app-config
    files:
      - configs/app.properties      # From a file (key = filename)
      - application.conf=configs/app.conf  # Custom key name
    literals:
      - LOG_LEVEL=info
      - MAX_CONNECTIONS=100
    options:
      disableNameSuffixHash: true    # Set true to disable hash suffix (default: false, hash enabled)
```

What the hash suffix does: By default, Kustomize appends an 8-character hash of the ConfigMap contents to its name (e.g., `app-config-5gh29kd7`). When the content changes, the hash changes, the name changes, and Kubernetes forces a rolling update of all Pods that reference it. This guarantees Pods always use the latest configuration.

configs/app.properties (a plain text config file):

```
database.host=prod-db.internal
database.port=5432
database.name=myapp_production
cache.ttl=3600
```

Secret Generator

```
# kustomization.yaml
secretGenerator:
  - name: db-credentials
    type: Opaque
    files:
      - secrets/db-password.txt     # Content becomes the secret value
    literals:
      - username=myapp_user
    envs:
      - secrets/.env.production     # Load from .env file format
    options:
      disableNameSuffixHash: true   # Stable name for secrets (referenced by pods)
```

secrets/.env.production:

```
DB_PASSWORD=super-secret-password-123
API_KEY=abcdef123456
```

> ⚠️ ***Security Note:*** *Never commit secret files to version control. Use a secrets management system (Vault, AWS Secrets Manager, Sealed Secrets) and only reference them in kustomization.yaml.*

29.10 Image Transformers

The `images` field in kustomization.yaml is a dedicated transformer for updating container image names and tags across ALL resources in a kustomization. This is the primary mechanism for environment-specific image versioning.

```
# kustomization.yaml
images:
  - name: my-app                        # Match this image name in any container
    newName: registry.company.com/my-app  # Replace with this name
    newTag: v1.5.2                        # Replace the tag

  - name: sidecar-image
    newTag: stable                        # Only change the tag, keep the name

  - name: old-image-name
    newName: new-image-name               # Rename without changing tag

  - name: my-app
    digest: sha256:abc123def456...        # Pin to exact digest (immutable)
```

Why use the images transformer instead of editing deployment.yaml directly?

Because the base `deployment.yaml` uses the logical name `my-app:latest`, and each overlay can independently set the exact image version appropriate for that environment. The base file never needs to be modified.

```
# Verify the image override applied correctly
kubectl kustomize ./overlays/production | grep "image:"
```

29.11 Merging Multiple Kustomizations with Components

Kustomize 4.1+ introduced **Components** — reusable pieces of configuration that can be included in multiple overlays. This solves the problem of shared features (e.g., monitoring, TLS) that apply to some but not all overlays.

components/monitoring/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1alpha1
kind: Component

patches:
  - path: add-prometheus-annotations.yaml
```

components/monitoring/add-prometheus-annotations.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  template:
    metadata:
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "9090"
        prometheus.io/path: "/metrics"
```

overlays/production/kustomization.yaml (with component)

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

resources:
  - ../../base

components:
  - ../../components/monitoring   # Include the monitoring component
  - ../../components/tls          # Include the TLS component
```

29.12 Practical Kustomize Workflow Commands

Diffing Before Applying

Before applying a kustomization, it is best practice to see what would change. The combination of `kubectl kustomize` and `kubectl diff` gives you a precise diff against what is currently running:

```
# See what would change in the cluster if you applied the production overlay
kubectl diff -k ./overlays/production
```

What this does: Shows a unified diff between the current state of resources in the cluster and what the kustomization would produce. Lines starting with `-` are what would be removed, lines starting with `+` are what would be added.

Applying and Verifying

```
# Apply and immediately check rollout status
kubectl apply -k ./overlays/production && \
  kubectl rollout status deployment/my-app -n production
```

Viewing All Managed Resources

```
# See all resources that a kustomization manages
kubectl kustomize ./overlays/production | grep "^kind:\|^  name:"
```

Deleting Resources by Kustomization

```
# Tear down all resources from a kustomization
kubectl delete -k ./overlays/development
```

Debugging Kustomize Output

```
# Check for YAML syntax errors in your kustomization
kubectl kustomize ./overlays/production 2>&1 | head -50

# Validate against the Kubernetes API schema without applying
kubectl apply -k ./overlays/production --dry-run=server

# Output as JSON instead of YAML
kubectl kustomize ./overlays/production -o json | jq '.metadata.name'
```

29.13 Using Kustomize with Remote Bases

Kustomize can reference bases stored in remote Git repositories, which is powerful for sharing common configurations across teams and projects.

```
# kustomization.yaml — reference a remote base from GitHub
resources:
  - github.com/company/k8s-base//apps/my-app/base?ref=v1.2.0
  # Format: github.com/<org>/<repo>//<path-in-repo>?ref=<branch-or-tag>
```

```
# Reference a base from a specific Git tag
resources:
  - https://github.com/company/k8s-base//base?ref=v2.0.0
```

> ⚠️ ***Production Note:*** *Always pin remote bases to a specific tag or commit hash, never to a mutable branch like* `main`*. This ensures your deployments are reproducible and not broken by upstream changes.*

29.14 Kustomize in CI/CD Pipelines

In a CI/CD pipeline, the typical pattern is:

```
# 1. Render the overlay for the target environment
kubectl kustomize ./overlays/${ENVIRONMENT} > rendered.yaml

# 2. Validate the rendered output (optional but recommended)
kubectl apply -f rendered.yaml --dry-run=server

# 3. Diff against current cluster state
kubectl diff -f rendered.yaml

# 4. Apply
kubectl apply -f rendered.yaml

# 5. Wait for rollout
kubectl rollout status deployment/my-app -n ${ENVIRONMENT} --timeout=300s
```

Why render to a file first? Storing the rendered manifest as a CI/CD artifact gives you an exact record of what was deployed — useful for auditing, debugging, and disaster recovery.

29.15 Troubleshooting Kustomize

Error: accumulating resources

```
$ kubectl kustomize overlays/production
Error: accumulating resources:
  accumulation err='merging resources from '../base':
  no such file or directory'

# Base path is wrong in kustomization.yaml
# Check relative path from overlay to base:
ls overlays/production/../../base/
# Fix kustomization.yaml:
# resources:
# - ../../base    ← must match actual relative path
```

Error: Strategic merge patch conflict

```
$ kubectl apply -k overlays/production
Error: strategic merge patch: ...
  "spec.template.spec.containers[name="app"]":
  unable to find api field in struct Container for the key "unknown-field"

# The patch references a field that doesn't exist
# Check the patch file - field names must be exact:
kubectl explain deployment.spec.template.spec.containers | grep -i "field\|resource"
# Common mistakes:
# "ressources" instead of "resources"
# "replicas" in the wrong level (it's in spec, not spec.template.spec)
```

Validate kustomize output before applying

```
# Preview what Kustomize will generate (no apply)
kubectl kustomize overlays/production

# Validate YAML syntax
kubectl kustomize overlays/production | kubectl apply --dry-run=client -f -

# Diff against what's running in cluster
kubectl diff -k overlays/production

# Check if Kustomize can resolve all resources
kubectl kustomize overlays/production 2>&1 | grep -i error
```

Common Kustomize gotchas

```
# Gotcha 1: Patches not applying - namePrefix changes the name
# If base has name: app and overlay adds namePrefix: prod-
# then the patch must reference: prod-app (not app)
# Use the final generated name in patches

# Gotcha 2: ConfigMap generators don't update when content changes
# Add a --load-restrictor flag if files are outside kustomize root:
kubectl kustomize --load-restrictor LoadRestrictionsNone ./overlays/prod

# Gotcha 3: Image transformer only applies to containers, not initContainers
# Use a separate patch for initContainers:
# patches:
# - target:
#     kind: Deployment
#   patch: |-
#     - op: replace
#       path: /spec/template/spec/initContainers/0/image
#       value: myimage:v2.0

# Gotcha 4: ConfigMap names get hashed by default
# This is good (prevents stale config) but can break references
# Disable hash for a specific generator:
# configMapGenerator:
# - name: app-config
#   options:
#     disableNameSuffixHash: true  ← Use only when you need stable names
```

Chapter 30 — CKA Certification 2026: Complete Study and Command Reference

30.1 About the CKA Examination

The **Certified Kubernetes Administrator (CKA)** examination is administered by the Cloud Native Computing Foundation (CNCF) and the Linux Foundation. It is a performance-based, hands-on examination — you are given a live Kubernetes cluster and must complete tasks within a time limit using only a terminal. There are no multiple-choice questions.

2026 Examination Details

Attribute	Details
Duration	2 hours
Questions	~17 performance-based tasks
Passing score	66%
Exam mode	Online proctored (browser-based terminal)
Kubernetes version	Current stable (expected: 1.30 or 1.31 in 2026)
Retake policy	1 free retake included
Validity	3 years from passing date
Environment	PSI Bridge secure browser + provided kubeconfig

Allowed Resources During the Exam

The CKA exam allows you to open **one additional browser tab** pointing to the official Kubernetes documentation at `kubernetes.io/docs`. You may NOT use any other external resources. Bookmarking key documentation pages before the exam is strongly recommended.

30.2 CKA 2026 Exam Domains and Weightings

The CKA curriculum is divided into five domains. The percentage indicates how many points in the exam come from each domain.

Domain 1: Cluster Architecture, Installation & Configuration (25%)

This is the heaviest domain. It covers setting up and managing the Kubernetes control plane, RBAC, kubeadm, and etcd.

Key topics:

- kubeadm cluster installation, joining nodes, and upgrading clusters
- Role-Based Access Control (RBAC): Roles, ClusterRoles, RoleBindings, ClusterRoleBindings
- ServiceAccounts and their permissions
- etcd backup and restore
- Kubeconfig file management and multi-cluster contexts
- Kubernetes API server configuration and flags

- High availability control plane concepts

Example exam task: "*Create a ClusterRole named `pod-reader` that allows listing and getting pods, then bind it to the user `jane` in the `development` namespace.*"

```
# Create the ClusterRole
kubectl create clusterrole pod-reader \
  --verb=get,list,watch \
  --resource=pods

# Create a RoleBinding in the development namespace
# (RoleBinding binds a ClusterRole within a namespace scope)
kubectl create rolebinding jane-pod-reader \
  --clusterrole=pod-reader \
  --user=jane \
  --namespace=development
```

```
# Verify the binding works
kubectl auth can-i list pods \
  --namespace=development \
  --as=jane
# Expected output: yes
```

Domain 2: Workloads & Scheduling (15%)

This domain covers deploying and managing applications on Kubernetes.

Key topics:

- Deployments, ReplicaSets, StatefulSets, DaemonSets, Jobs, CronJobs
- Rolling updates and rollbacks
- Resource requests and limits, LimitRanges, ResourceQuotas
- ConfigMaps and Secrets — creating, updating, mounting
- Node selectors, node affinity, taints and tolerations
- Pod priority and preemption
- Horizontal Pod Autoscaler (HPA)
- Multi-container pod patterns (init containers, sidecar)

Example exam task: "*Scale the Deployment `web-app` in namespace `production` to 5 replicas, then update the image to `nginx:1.25`, and verify the rollout completes successfully.*"

```
# Scale
kubectl scale deployment web-app -n production --replicas=5

# Update image
kubectl set image deployment/web-app nginx=nginx:1.25 -n production

# Watch rollout
kubectl rollout status deployment/web-app -n production

# Verify
kubectl get pods -n production -l app=web-app
```

Example exam task: "*Create a Pod that uses a ConfigMap to set environment variables.*"

```
# Create ConfigMap
kubectl create configmap app-config \
  --from-literal=DB_HOST=postgres.default.svc.cluster.local \
  --from-literal=DB_PORT=5432

# Create pod using the configmap
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: app
    image: busybox
    command: ["sh", "-c", "env | grep DB && sleep 3600"]
    envFrom:
    - configMapRef:
        name: app-config
EOF
```

Domain 3: Services & Networking (20%)

This domain covers how applications communicate within and outside the cluster.

Key topics:

- Service types: ClusterIP, NodePort, LoadBalancer, ExternalName
- Ingress resources and Ingress controllers
- NetworkPolicies — restricting Pod-to-Pod communication
- DNS and CoreDNS (fully covered in Chapter 10 of this guide)
- kube-proxy and Service routing fundamentals
- Gateway API (newer addition for 2025/2026 curriculum)

Example exam task: "*Create a NetworkPolicy in namespace* `api` *that allows only Pods with label* `role=frontend` *to communicate with Pods that have label* `role=backend` *on port 8080.*"

```
# network-policy.yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: frontend-to-backend
  namespace: api
spec:
  podSelector:
    matchLabels:
      role: backend         # This policy applies TO backend pods
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: frontend    # Only allow FROM frontend pods
    ports:
    - protocol: TCP
      port: 8080
```

```
kubectl apply -f network-policy.yaml

# Test: create a frontend pod and try to curl the backend
kubectl run frontend-test \
  --image=curlimages/curl \
  --labels="role=frontend" \
  --rm -it --restart=Never \
  -n api \
  -- curl -s http://backend-service:8080/health

# Test: create a pod without the label (should be BLOCKED)
kubectl run blocked-test \
  --image=curlimages/curl \
  --rm -it --restart=Never \
  -n api \
  -- curl -s --max-time 5 http://backend-service:8080/health
# This should timeout/fail — blocked by NetworkPolicy
```

Example exam task: "*Expose the Deployment `my-app` as a NodePort service on port 30080.*"

```
kubectl expose deployment my-app \
  --type=NodePort \
  --port=80 \
  --target-port=8080 \
  --name=my-app-nodeport

# Patch to set the specific NodePort
kubectl patch service my-app-nodeport \
  --type='json' \
  -p='[{"op":"replace","path":"/spec/ports/0/nodePort","value":30080}]'
```

Example exam task: "*Create an Ingress that routes `/api` to the service `api-service:8080` and `/web` to `web-service:80`.*"

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
spec:
  ingressClassName: nginx
  rules:
  - host: myapp.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 8080
      - path: /web
        pathType: Prefix
        backend:
          service:
            name: web-service
            port:
              number: 80
```

```
kubectl apply -f ingress.yaml
kubectl get ingress my-ingress
kubectl describe ingress my-ingress
```

Domain 4: Storage (10%)

This domain covers persistent data storage in Kubernetes.

Key topics:

- PersistentVolumes (PV) and PersistentVolumeClaims (PVC)
- StorageClasses and dynamic provisioning
- Volume access modes: ReadWriteOnce, ReadOnlyMany, ReadWriteMany
- Volume types: hostPath, emptyDir, nfs, configMap, secret
- Mounting volumes in containers

Example exam task: "*Create a PersistentVolume of 1Gi with hostPath* `/data/myapp`*, then create a PVC that claims it, then mount it in a Pod at* `/app/data`."

```
# pv.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: myapp-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
  - ReadWriteOnce
  hostPath:
    path: /data/myapp
  storageClassName: manual
```

```
# pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myapp-pvc
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: manual
```

```yaml
# pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
spec:
  containers:
  - name: app
    image: nginx
    volumeMounts:
    - name: app-storage
      mountPath: /app/data
  volumes:
  - name: app-storage
    persistentVolumeClaim:
      claimName: myapp-pvc
```

```
kubectl apply -f pv.yaml -f pvc.yaml -f pod.yaml

# Verify PVC is bound
kubectl get pvc myapp-pvc
# STATUS should be "Bound"

# Verify the mount
kubectl exec myapp-pod -- df -h /app/data
```

Domain 5: Troubleshooting (30%)

Troubleshooting is the **largest domain** in the CKA and directly maps to the skills covered throughout this entire guide. It requires diagnosing and fixing broken Pods, nodes, cluster components, and networking.

Key topics:

- Diagnosing Pod failures: OOMKilled, CrashLoopBackOff, ImagePullBackOff, Pending
- Node troubleshooting: NotReady nodes, kubelet failures
- Cluster component failures: API server, scheduler, controller-manager, etcd
- Application networking issues
- Log analysis and event inspection

Example exam task: "*The Pod* `broken-app` *in namespace* `qa` *is in CrashLoopBackOff. Find the cause and fix it.*"

```
# Step 1: Describe the Pod
kubectl describe pod broken-app -n qa

# Look at:
# - "Events" section: image pull errors, OOMKill, liveness probe failures
# - "State" of the container: reason, exit code, message
# - "Last State": what happened before the current crash

# Step 2: Check current and previous logs
kubectl logs broken-app -n qa
kubectl logs broken-app -n qa --previous

# Step 3: Based on exit codes:
# Exit 137 (OOMKill) → increase memory limits
# Exit 1 (app error) → check app logs for startup failure
# CrashLoop with no logs → entrypoint/command error

# Step 4: Fix — e.g., fix an incorrect environment variable
kubectl set env deployment/broken-app -n qa DB_HOST=correct-hostname

# Or edit the deployment
kubectl edit deployment broken-app -n qa
```

Example exam task: "*A node is in NotReady state. Investigate and restore it.*"

```
# Step 1: Identify the NotReady node
kubectl get nodes

# Step 2: Describe the node for events
kubectl describe node <node-name>

# Step 3: SSH into the node (if allowed by exam task)
# Check kubelet status
systemctl status kubelet

# Check kubelet logs
journalctl -u kubelet -n 50 --no-pager

# Common fixes:
# kubelet is stopped → systemctl start kubelet
# kubelet config error → check /var/lib/kubelet/config.yaml
# Container runtime issue → systemctl status containerd

# Restart kubelet
systemctl restart kubelet

# Step 4: Verify node recovers
kubectl get nodes -w
```

30.3 etcd Backup and Restore

Backing up and restoring etcd is a critical CKA skill that appears in the Cluster Architecture domain. The `etcdctl` utility is used for this operation.

Example: Back up etcd to a file

```
ETCDCTL_API=3 etcdctl snapshot save /tmp/etcd-backup.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
```

Flag explanations:

- `ETCDCTL_API=3` — Use etcd API version 3 (required for modern etcd)
- `--endpoints` — The etcd endpoint (usually 127.0.0.1:2379 on the control plane node)
- `--cacert`, `--cert`, `--key` — TLS certificates for authenticated access to etcd

Example: Verify the backup

```
ETCDCTL_API=3 etcdctl snapshot status /tmp/etcd-backup.db \
  --write-out=table
```

Output shows: Snapshot hash, revision, total keys, and size. Always verify after backup.

Example: Restore etcd from backup

```
ETCDCTL_API=3 etcdctl snapshot restore /tmp/etcd-backup.db \
  --data-dir=/var/lib/etcd-restored
```

Then update the etcd static Pod manifest to use the new data directory:

```
# Edit /etc/kubernetes/manifests/etcd.yaml
# Change: --data-dir=/var/lib/etcd
# To:     --data-dir=/var/lib/etcd-restored
# And update the hostPath volume to match
```

30.4 Cluster Upgrade with kubeadm

Upgrading a Kubernetes cluster is a key CKA task. The process follows a strict order: upgrade the control plane first, then upgrade worker nodes one by one.

Step 1: Check current version and available upgrades

```
kubeadm upgrade plan
```

What this shows: The current version, the recommended upgrade target, and all components that will be upgraded. Always run this first.

Step 2: Upgrade kubeadm on the control plane node

```
# On the control plane node (as root/sudo)
apt-mark unhold kubeadm
apt-get update
apt-get install -y kubeadm=1.30.0-00
apt-mark hold kubeadm

# Verify
kubeadm version
```

Step 3: Apply the upgrade

```
kubeadm upgrade apply v1.30.0
```

Step 4: Upgrade kubelet and kubectl on the control plane

```
# Drain the control plane node first
kubectl drain <control-plane-node> \
  --ignore-daemonsets \
  --delete-emptydir-data

# Upgrade kubelet and kubectl
apt-mark unhold kubelet kubectl
apt-get install -y kubelet=1.30.0-00 kubectl=1.30.0-00
apt-mark hold kubelet kubectl

# Restart kubelet
systemctl daemon-reload
systemctl restart kubelet

# Uncordon the node
kubectl uncordon <control-plane-node>
```

Step 5: Upgrade each worker node

```
# For each worker node:
# 1. Drain from the control plane
kubectl drain <worker-node> \
  --ignore-daemonsets \
  --delete-emptydir-data

# 2. SSH into the worker node and run:
apt-mark unhold kubeadm
apt-get install -y kubeadm=1.30.0-00
apt-mark hold kubeadm

kubeadm upgrade node   # Note: worker nodes use "node" not "apply"

apt-mark unhold kubelet kubectl
apt-get install -y kubelet=1.30.0-00 kubectl=1.30.0-00
apt-mark hold kubelet kubectl
systemctl daemon-reload
systemctl restart kubelet

# 3. Back on the control plane, uncordon the worker
kubectl uncordon <worker-node>
```

30.5 RBAC — Roles, ClusterRoles, Bindings

RBAC is one of the most frequently tested areas in the CKA. It controls who can do what to which resources in which namespaces.

The RBAC model has four resource types:

Resource	Scope	Purpose
`Role`	Namespace	Defines permissions within one namespace
`ClusterRole`	Cluster-wide	Defines permissions across all namespaces or for cluster-scoped resources
`RoleBinding`	Namespace	Grants a Role or ClusterRole to a user/group/SA within a namespace
`ClusterRoleBinding`	Cluster-wide	Grants a ClusterRole to a user/group/SA across the entire cluster

Example: Create a Role allowing only pod listing in a namespace

```
kubectl create role pod-reader \
  --verb=get,list,watch \
  --resource=pods \
  --namespace=development
```

Example: Bind the Role to a user

```
kubectl create rolebinding dev-pod-reader \
  --role=pod-reader \
  --user=developer1 \
  --namespace=development
```

Example: Create a ServiceAccount with specific permissions

```
# Create the ServiceAccount
kubectl create serviceaccount monitoring-sa -n monitoring

# Create a ClusterRole for reading cluster metrics
kubectl create clusterrole metrics-reader \
  --verb=get,list \
  --resource=nodes,pods,services,endpoints

# Bind to the ServiceAccount
kubectl create clusterrolebinding monitoring-metrics-reader \
  --clusterrole=metrics-reader \
  --serviceaccount=monitoring:monitoring-sa
```

Example: Test permissions using kubectl auth can-i

```
# Test as a specific user
kubectl auth can-i create deployments --namespace=development --as=developer1

# Test as a ServiceAccount
kubectl auth can-i get pods \
  --namespace=monitoring \
  --as=system:serviceaccount:monitoring:monitoring-sa

# List all permissions for the current user
kubectl auth can-i --list

# List all permissions for a user in a specific namespace
kubectl auth can-i --list --namespace=development --as=developer1
```

30.6 Jobs and CronJobs

Example: Create a Job that runs a computation

```
kubectl create job compute-pi \
  --image=perl \
  -- perl -Mbignum=bpi -wle 'print bpi(2000)'
```

```
# Watch the Job run
kubectl get jobs -w

# See the output
kubectl logs -l job-name=compute-pi
```

Example: Create a CronJob that runs every 5 minutes

```
kubectl create cronjob health-check \
  --image=curlimages/curl \
  --schedule="*/5 * * * *" \
  -- curl -s http://my-service/health
```

```
# List CronJobs
kubectl get cronjobs

# Manually trigger a CronJob
kubectl create job --from=cronjob/health-check manual-run-001

# View last few runs
kubectl get jobs -l job-name=health-check --sort-by=.metadata.creationTimestamp
```

30.7 Node Management: Cordoning, Draining, Taints

Example: Cordon a node (prevent new Pods from scheduling)

```
kubectl cordon <node-name>
```

What this does: Marks the node as `SchedulingDisabled`. Existing Pods continue to run, but no new Pods will be scheduled on it. Use before maintenance.

Example: Drain a node (evict all Pods gracefully)

```
kubectl drain <node-name> \
  --ignore-daemonsets \
  --delete-emptydir-data \
  --grace-period=60
```

What this does: Evicts all Pods from the node (except DaemonSet Pods and mirror Pods). The `--ignore-daemonsets` flag is required because DaemonSet Pods cannot be evicted. `--delete-emptydir-data`
is required if any Pods use `emptyDir` volumes (their data will be lost).

Example: Uncordon a node after maintenance

```
kubectl uncordon <node-name>
```

Example: Add a taint to a node

```
# NoSchedule: new pods without toleration won't be scheduled here
kubectl taint nodes <node-name> dedicated=gpu:NoSchedule

# PreferNoSchedule: scheduler tries to avoid this node but will use it if needed
kubectl taint nodes <node-name> dedicated=gpu:PreferNoSchedule

# NoExecute: evicts existing pods that don't tolerate the taint
kubectl taint nodes <node-name> maintenance=true:NoExecute
```

Example: Remove a taint from a node

```
# Add a trailing dash (-) to the taint to remove it
kubectl taint nodes <node-name> dedicated=gpu:NoSchedule-
```

30.8 Pod Scheduling: Affinity, Node Selectors, Tolerations

Example: Schedule a Pod on a specific node using nodeSelector

```
# First, label the target node
kubectl label node <node-name> disktype=ssd

# Then create a Pod with a nodeSelector
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Pod
metadata:
  name: ssd-app
spec:
  nodeSelector:
    disktype: ssd
  containers:
  - name: app
    image: nginx
EOF
```

Example: Schedule a Pod with node affinity (preferred)

```
apiVersion: v1
kind: Pod
metadata:
  name: affinity-pod
spec:
  affinity:
    nodeAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        nodeSelectorTerms:
        - matchExpressions:
          - key: disktype
            operator: In
            values:
            - ssd
      preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 1
        preference:
          matchExpressions:
          - key: region
            operator: In
            values:
            - us-east-1
  containers:
  - name: app
    image: nginx
```

Example: Add a toleration to schedule on a tainted node

```
apiVersion: v1
kind: Pod
metadata:
  name: gpu-pod
spec:
  tolerations:
  - key: "dedicated"
    operator: "Equal"
    value: "gpu"
    effect: "NoSchedule"
  containers:
  - name: app
    image: nvidia/cuda:11.0-base
```

30.9 ResourceQuotas and LimitRanges

Example: Create a ResourceQuota for a namespace

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: dev-quota
  namespace: development
spec:
  hard:
    requests.cpu: "4"
    requests.memory: "8Gi"
    limits.cpu: "8"
    limits.memory: "16Gi"
    pods: "20"
    services: "10"
    persistentvolumeclaims: "5"
```

```
kubectl apply -f quota.yaml

# View quota usage
kubectl describe resourcequota dev-quota -n development
```

Example: Create a LimitRange to set default resource requests

```
apiVersion: v1
kind: LimitRange
metadata:
  name: default-limits
  namespace: development
spec:
  limits:
  - default:              # Default limits (used if container doesn't specify)
      cpu: "500m"
      memory: "256Mi"
    defaultRequest:       # Default requests (used if container doesn't specify)
      cpu: "100m"
      memory: "128Mi"
    max:                  # Maximum allowed values
      cpu: "2"
      memory: "2Gi"
    min:                  # Minimum allowed values
      cpu: "50m"
      memory: "64Mi"
    type: Container
```

```
kubectl apply -f limitrange.yaml

# Verify
kubectl describe limitrange default-limits -n development
```

30.10 CKA Time Management and Exam Strategy

Understanding the technical content is necessary but not sufficient for passing the CKA. The exam requires completing approximately 17 tasks in 2 hours — about 7 minutes per task. Speed and efficiency are as important as knowledge.

25.10.1 Setting Up Your Exam Shell Environment

The very first thing to do when the exam starts is configure your shell for efficiency. This alone can save 5–10 minutes across the exam.

```
# Enable kubectl tab autocompletion (works in the exam environment)
source <(kubectl completion bash)
echo "source <(kubectl completion bash)" >> ~/.bashrc

# Create aliases for the most-used commands
alias k=kubectl
alias kgp='kubectl get pods'
alias kgs='kubectl get services'
alias kgd='kubectl get deployments'
alias kgn='kubectl get nodes'
alias kdp='kubectl describe pod'
alias kns='kubectl config set-context --current --namespace'

# Make the 'k' alias also use autocompletion
complete -F __start_kubectl k

# Set vim as the default editor for kubectl edit
export KUBE_EDITOR=vim

# Apply all aliases immediately
source ~/.bashrc
```

25.10.2 Imperative Commands vs YAML Files

The CKA rewards speed. **Use imperative commands wherever possible** rather than writing YAML
from scratch. Every second saved on file creation is a second spent on harder tasks.

Speed comparison:

```
# ❌ SLOW: Writing a Deployment YAML from scratch
vim deployment.yaml
# (minutes of typing...)

# ✅ FAST: Imperative creation
kubectl create deployment my-app --image=nginx --replicas=3

# ✅ FAST: Generate YAML and edit
kubectl create deployment my-app --image=nginx --dry-run=client -o yaml > my-app.yaml
# (make minimal edits to the generated file)
kubectl apply -f my-app.yaml
```

Dry-run + output: The most powerful exam technique

```
# Generate a Pod YAML without creating it
kubectl run my-pod --image=nginx --dry-run=client -o yaml > pod.yaml

# Generate a Deployment
kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml > dep.yaml

# Generate a Service
kubectl expose deployment my-dep --port=80 --dry-run=client -o yaml > svc.yaml

# Generate a ConfigMap
kubectl create configmap my-cm --from-literal=key=value --dry-run=client -o yaml > cm.yaml

# Generate a Secret
kubectl create secret generic my-secret --from-literal=password=secret --dry-run=client -o yaml > secret
.yaml

# Generate a ServiceAccount
kubectl create serviceaccount my-sa --dry-run=client -o yaml > sa.yaml

# Generate a Role
kubectl create role my-role --verb=get,list --resource=pods --dry-run=client -o yaml > role.yaml

# Generate a ClusterRole
kubectl create clusterrole my-cr --verb=get,list --resource=pods --dry-run=client -o yaml > cr.yaml

# Generate a RoleBinding
kubectl create rolebinding my-rb --role=my-role --user=jane --dry-run=client -o yaml > rb.yaml
```

25.10.3 Context Switching Between Clusters

The CKA exam typically presents multiple clusters. Each task specifies which cluster context to use. **Always switch context as the FIRST step of each task.** Working in the wrong cluster is a very common and costly mistake.

```
# List available contexts
kubectl config get-contexts

# Switch to a specific cluster
kubectl config use-context k8s-cluster-1

# Verify you are in the right context
kubectl config current-context

# Switch namespace within the current context (avoid -n on every command)
kubectl config set-context --current --namespace=kube-system
```

25.10.4 Using kubectl explain for Quick Reference

During the exam, `kubectl explain` is your in-terminal documentation. It shows the fields of any Kubernetes resource, their types, and descriptions.

```
# Explain all Pod fields
kubectl explain pod

# Explain a specific nested field
kubectl explain pod.spec.containers

# Explain tolerations
kubectl explain pod.spec.tolerations

# Explain affinity rules
kubectl explain pod.spec.affinity.nodeAffinity

# Explain PersistentVolumeClaim spec
kubectl explain pvc.spec

# Recursive explanation (shows full tree)
kubectl explain deployment --recursive | head -50
```

When to use it: When you know the resource but can't remember the exact field name or structure. For example, if you forget whether it's `imagePullPolicy` or `image-pull-policy`, `kubectl explain pod.spec.containers` will show you immediately.

25.10.5 Vim Tips for the CKA Exam

Most CKA candidates use vim as their editor. Knowing these shortcuts significantly reduces editing time.

```
# Navigation
gg          — Go to first line
G           — Go to last line
:<number>   — Go to specific line number (e.g., :42)
/text       — Search forward for "text"
n           — Next search result

# Editing
dd          — Delete current line
yy          — Copy (yank) current line
p           — Paste after current line
u           — Undo
Ctrl+r      — Redo

# Indentation (critical for YAML)
>>          — Indent current line
<<          — Dedent current line
Visual mode + > or < — Indent/dedent selected lines

# YAML-specific vim settings (add to ~/.vimrc in exam)
```

```
# Run this to configure vim for YAML editing:
cat >> ~/.vimrc << 'EOF'
set expandtab       " Use spaces instead of tabs
set tabstop=2       " Tab width = 2 spaces
set shiftwidth=2    " Indent width = 2 spaces
set autoindent      " Auto-indent new lines
EOF
```

25.10.6 Checking Your Work After Each Task

After completing a task, always verify it:

```
# After creating a Pod
kubectl get pod <pod-name>
kubectl describe pod <pod-name> | tail -20   # Check events

# After creating a Deployment
kubectl get deployment <name>
kubectl rollout status deployment/<name>

# After creating a Service
kubectl get svc <name>
kubectl describe svc <name>

# After creating RBAC
kubectl auth can-i <verb> <resource> --as=<user> -n <namespace>

# After a node operation
kubectl get nodes

# After etcd backup
ls -la /tmp/etcd-backup.db
ETCDCTL_API=3 etcdctl snapshot status /tmp/etcd-backup.db
```

30.11 Static Pods — A Critical CKA Topic

Static Pods are Pods managed directly by the kubelet on a specific node, not by the Kubernetes API server or any controller. They are defined as YAML files placed in a directory on the node's filesystem, typically `/etc/kubernetes/manifests/`. The control plane components themselves (etcd, kube-apiserver, kube-controller-manager, kube-scheduler) run as static Pods.

Example: Locate the static Pod manifests directory

```
# On the control plane node
ls /etc/kubernetes/manifests/
# Output: etcd.yaml  kube-apiserver.yaml  kube-controller-manager.yaml  kube-scheduler.yaml
```

Example: Create a static Pod on a node

```
# SSH into the target node
# Create a manifest in the static pods directory
cat << EOF > /etc/kubernetes/manifests/static-nginx.yaml
apiVersion: v1
kind: Pod
metadata:
  name: static-nginx
  namespace: default
spec:
  containers:
  - name: nginx
    image: nginx:1.25
    ports:
    - containerPort: 80
EOF
```

What happens: The kubelet detects the new file within seconds and starts the Pod automatically. No `kubectl apply` is needed. The Pod appears in `kubectl get pods` as `static-nginx-<node-name>`.

Example: Delete a static Pod

```
# ❌ This does NOT work — the kubelet immediately recreates it
kubectl delete pod static-nginx-<node-name>

# ✅ Correct way: delete the manifest file on the node
rm /etc/kubernetes/manifests/static-nginx.yaml
```

The kubelet detects the file removal and terminates the Pod.

Example: Find the static Pod path from the kubelet config

```
# If the default path doesn't work, check the kubelet config
cat /var/lib/kubelet/config.yaml | grep staticPodPath
# or
ps aux | grep kubelet | grep staticPodPath
```

30.12 Helm in the Context of CKA

While Helm is more central to the CKAD (Certified Kubernetes Application Developer) certification, the CKA 2025/2026 curriculum does include basic Helm usage. Understanding the relationship between
Helm and raw kubectl/Kustomize commands is important.

Example: Add a Helm repository

```
helm repo add stable https://charts.helm.sh/stable
helm repo update
```

Example: Install a chart

```
helm install my-release stable/nginx-ingress \
  --namespace ingress \
  --create-namespace \
  --set controller.replicaCount=2
```

Example: List installed releases

```
helm list -A
```

Example: Get values of a release

```
helm get values my-release -n ingress
```

Example: Uninstall a release

```
helm uninstall my-release -n ingress
```

30.13 Complete CKA Troubleshooting Scenarios

The CKA exam always includes troubleshooting tasks. These are the most common patterns:

Scenario 1: Fix a broken cluster — node NotReady

```
# Step 1: Identify the problem node
kubectl get nodes
# NAME         STATUS     ROLES    AGE   VERSION
# worker-1     NotReady   <none>   5d    v1.30.0  ← problem!

# Step 2: Check node conditions
kubectl describe node worker-1 | grep -A 10 "Conditions:"

# Step 3: SSH to the node
ssh worker-1

# Step 4: Check kubelet
sudo systemctl status kubelet
# ● kubelet.service - kubelet: The Kubernetes Node Agent
#    Active: failed (Result: exit-code)
sudo journalctl -u kubelet -n 30 --no-pager

# Common error 1: kubelet service not enabled
sudo systemctl enable kubelet
sudo systemctl start kubelet

# Common error 2: kubelet config error
sudo journalctl -u kubelet | grep "error"
# "failed to load Kubelet config file /var/lib/kubelet/config.yaml"
# File may be missing or corrupted - recreate using kubeadm:
sudo kubeadm init phase kubelet-config

# Common error 3: Container runtime down
sudo systemctl status containerd
sudo systemctl start containerd
sudo systemctl restart kubelet
```

Scenario 2: Fix a broken Deployment

```
# Task: "Deployment web-app in namespace frontend is not running. Fix it."

# Step 1: Check deployment status
kubectl get deployment web-app -n frontend
# NAME      READY   UP-TO-DATE   AVAILABLE   AGE
# web-app   0/3     0            0           10m

# Step 2: Describe deployment
kubectl describe deployment web-app -n frontend | tail -30

# Step 3: Check pods
kubectl get pods -n frontend -l app=web-app
# NAME                       READY   STATUS             RESTARTS   AGE
# web-app-7d8f9b6c4-abc12   0/1     ImagePullBackOff   0          10m

# Step 4: Describe a failing pod
kubectl describe pod web-app-7d8f9b6c4-abc12 -n frontend
# Events:
#   Warning  Failed  2m  kubelet  Failed to pull image "nginx:1.99.99":
#     rpc error: ...
#     404 Not Found

# Step 5: Fix image
kubectl set image deployment/web-app web=nginx:1.25 -n frontend
kubectl rollout status deployment/web-app -n frontend
```

Scenario 3: Expose a Deployment

```
# Task: "Create a Service to expose deployment api-server on port 8080"

# Quick imperative creation
kubectl expose deployment api-server --port=8080 --target-port=8080 \
  --name=api-service -n backend

# Verify it works
kubectl get svc api-service -n backend
kubectl get endpoints api-service -n backend  # Should have IPs
kubectl run test --image=curlimages/curl --rm -it --restart=Never \
  -n backend -- curl http://api-service:8080/health
```

Scenario 4: Create RBAC for a user

```
# Task: "User dev-user should be able to get/list pods in namespace dev"

kubectl create role pod-reader \
  --verb=get,list,watch \
  --resource=pods \
  -n dev

kubectl create rolebinding dev-user-pod-reader \
  --role=pod-reader \
  --user=dev-user \
  -n dev

# Verify:
kubectl auth can-i get pods -n dev --as=dev-user   # yes
kubectl auth can-i delete pods -n dev --as=dev-user  # no
kubectl auth can-i get pods -n production --as=dev-user  # no
```

30.14 CKA Exam Practice: Common Task Templates

Task type: etcd backup (appears on almost every CKA exam)

```
# Read the question carefully - it tells you where to save the backup
# Typical task: "Backup etcd to /opt/cluster-backup.db"

ETCDCTL_API=3 etcdctl snapshot save /opt/cluster-backup.db \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key

# Verify:
ETCDCTL_API=3 etcdctl snapshot status /opt/cluster-backup.db -w table
```

Task type: NetworkPolicy (common CKA task)

```
# Task: "Deny all ingress traffic to namespace restricted,
#         but allow traffic from namespace monitoring on port 9090"

# Step 1: Apply deny-all default
cat << 'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-ingress
  namespace: restricted
spec:
  podSelector: {}
  policyTypes:
  - Ingress
EOF

# Step 2: Allow from monitoring namespace
cat << 'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-from-monitoring
  namespace: restricted
spec:
  podSelector: {}
  policyTypes:
  - Ingress
  ingress:
  - from:
    - namespaceSelector:
        matchLabels:
          kubernetes.io/metadata.name: monitoring
    ports:
    - port: 9090
EOF
```

Task type: Node maintenance (drain + taint)

```
# Task: "Node worker-2 needs maintenance.
#         Drain it, then taint it with maintenance=true:NoSchedule"

# Step 1: Cordon (prevent new scheduling)
kubectl cordon worker-2

# Step 2: Drain (evict running pods)
kubectl drain worker-2 --ignore-daemonsets --delete-emptydir-data

# Step 3: Taint for maintenance
kubectl taint node worker-2 maintenance=true:NoSchedule

# After maintenance, restore:
kubectl taint node worker-2 maintenance=true:NoSchedule-  # Remove taint
kubectl uncordon worker-2
```

Task type: Create a multi-container pod

```
# Task: "Create a pod 'web-server' with two containers:
#         nginx:1.25 named 'web' on port 80, and
#         busybox:1.35 named 'logger' that runs 'sleep 3600'"

# Use --dry-run to generate template, then edit:
kubectl run web-server --image=nginx:1.25 --dry-run=client -o yaml > pod.yaml

# Edit pod.yaml to add second container and correct port:
```

```
apiVersion: v1
kind: Pod
metadata:
  name: web-server
spec:
  containers:
  - name: web
    image: nginx:1.25
    ports:
    - containerPort: 80
  - name: logger
    image: busybox:1.35
    command: ["sleep", "3600"]
```

30.15 Exam-Day Tips and Common Mistakes

Setup your shell before starting tasks:

```
# These 4 aliases save significant time on the exam
alias k=kubectl
alias kn='kubectl -n'
alias kg='kubectl get'
alias kd='kubectl describe'
export do='--dry-run=client -o yaml'  # Use: k run test --image=nginx $do

# Enable kubectl autocomplete (should already be set up on exam)
source <(kubectl completion bash)
complete -F __start_kubectl k

# Set current namespace to avoid -n on every command
kubectl config set-context --current --namespace=<task-namespace>
```

Common mistakes that cost points:

```
# Mistake 1: Working in wrong namespace
# Always check: kubectl config view --minify | grep namespace
# Set namespace at start of each task

# Mistake 2: Forgetting --dry-run=client -o yaml for complex resources
# Always generate YAML rather than trying to write it from scratch

# Mistake 3: Not verifying after completing a task
# After every task: kubectl get <resource> and kubectl describe <resource>
# Check that it's actually Running/Bound/Ready before moving on

# Mistake 4: Wasting time on one hard task
# If stuck >5 min: flag it, move to next task, come back
# 17 tasks in 2 hours = ~7 min per task

# Mistake 5: Not reading task fully
# The task might specify: namespace, resource name, labels, specific fields
# Read the whole task before typing anything

# Mistake 6: Using wrong context/cluster
# At start of each task: kubectl config use-context <context-from-task>
```

Appendix A — Quick Reference Cheat Sheet

A.1 Container Exec Commands

```
# Interactive shell
kubectl exec -it <pod> -- /bin/bash
kubectl exec -it <pod> -- /bin/sh                    # Alpine/minimal images

# Target specific container
kubectl exec -it <pod> -c <container> -- bash

# Non-interactive single command
kubectl exec <pod> -- <command>

# Shell interpretation for pipes/redirects
kubectl exec <pod> -- sh -c 'cmd1 | cmd2'

# Exec in specific namespace
kubectl exec -it <pod> -n <ns> -- bash
```

A.2 Log Commands

```
kubectl logs <pod>                                    # All logs
kubectl logs <pod> -c <container>                     # Specific container
kubectl logs -f <pod>                                 # Stream/follow
kubectl logs <pod> --tail=100                         # Last 100 lines
kubectl logs <pod> --since=1h                         # Last 1 hour
kubectl logs <pod> --since-time="2024-06-01T09:00Z"  # Since timestamp
kubectl logs <pod> --previous                         # Crashed container logs
kubectl logs -l app=my-app --all-containers=true     # All pods by label
```

A.3 File Transfer Commands

```
kubectl cp <pod>:/remote/path ./local/path            # Container → local
kubectl cp ./local/path <pod>:/remote/path            # Local → container
kubectl cp <pod>:/path -c <container> ./path          # Specific container
```

A.4 Inspection Commands

```
kubectl describe pod <pod>                            # Human-readable summary
kubectl get pod <pod> -o yaml                         # Full YAML spec
kubectl get pod <pod> -o json                         # Full JSON
kubectl get pod <pod> -o jsonpath='{.status.podIP}'  # Specific field
kubectl get pods -A -o wide                           # All pods, all namespaces
kubectl top pod <pod> --containers                   # Resource usage
```

A.5 Port Forwarding

```
kubectl port-forward <pod> 8080:80                   # Local:Container
kubectl port-forward service/<svc> 8080:80           # Via service
kubectl port-forward deployment/<dep> 8080:80        # Via deployment
kubectl port-forward <pod> 8080:80 9090:9090         # Multiple ports
```

A.6 Ephemeral Test Containers (Create & Auto-Delete)

```
# Interactive busybox shell
kubectl run test --image=busybox --rm -it --restart=Never -- sh

# Ping test
kubectl run test --image=busybox --rm -it --restart=Never -- ping -c 4 <host>

# curl test
kubectl run test --image=curlimages/curl --rm -it --restart=Never -- curl -s <url>

# Full toolkit (netshoot)
kubectl run test --image=nicolaka/netshoot --rm -it --restart=Never -- bash

# Specific namespace
kubectl run test --image=busybox --rm -it --restart=Never -n <ns> -- sh

# TCP port check
kubectl run test --image=busybox --rm -it --restart=Never -- nc -zv <host> <port>

# Traceroute
kubectl run test --image=nicolaka/netshoot --rm -it --restart=Never -- traceroute <host>
```

A.7 CoreDNS Testing

```
# Check CoreDNS pods
kubectl get pods -n kube-system -l k8s-app=kube-dns

# View CoreDNS config
kubectl get configmap coredns -n kube-system -o yaml

# CoreDNS logs
kubectl logs -n kube-system -l k8s-app=kube-dns

# DNS resolution test
kubectl run dns --image=busybox --rm -it --restart=Never -- nslookup kubernetes.default
kubectl run dns --image=busybox --rm -it --restart=Never -- nslookup <svc>.<ns>.svc.cluster.local
kubectl run dns --image=nicolaka/netshoot --rm -it --restart=Never -- dig <svc>.default.svc.cluster.loc
al
kubectl run dns --image=nicolaka/netshoot --rm -it --restart=Never -- dig +short <svc>
kubectl run dns --image=nicolaka/netshoot --rm -it --restart=Never -- dig @<coredns-ip> <svc>

# Check pod resolv.conf
kubectl exec <pod> -- cat /etc/resolv.conf

# External DNS test
kubectl run dns --image=busybox --rm -it --restart=Never -- nslookup google.com
```

A.8 Ephemeral Debug Containers (Stable, K8s 1.23+)

```
# Inject debug container into running pod
kubectl debug -it <pod> --image=nicolaka/netshoot --target=<container> -- bash

# Debug distroless pod
kubectl debug -it <pod> --image=busybox --target=<container> -- sh

# Create debug copy of pod
kubectl debug <pod> -it --copy-to=<debug-pod-name> --image=busybox -- sh
kubectl delete pod <debug-pod-name>    # Cleanup

# Debug a node
kubectl debug node/<node-name> -it --image=nicolaka/netshoot
```

Appendix B — CKA 2026 Command Speed Reference

This appendix is designed as a rapid-fire revision card for all CKA domain commands.

B.1 Cluster Architecture & RBAC

```
# Context management
kubectl config get-contexts
kubectl config use-context <context>
kubectl config set-context --current --namespace=<ns>
kubectl config current-context

# RBAC creation
kubectl create role <name> --verb=<verbs> --resource=<resources> -n <ns>
kubectl create clusterrole <name> --verb=<verbs> --resource=<resources>
kubectl create rolebinding <name> --role=<role> --user=<user> -n <ns>
kubectl create rolebinding <name> --clusterrole=<cr> --user=<user> -n <ns>
kubectl create clusterrolebinding <name> --clusterrole=<cr> --user=<user>
kubectl create clusterrolebinding <name> --clusterrole=<cr> --serviceaccount=<ns>:<sa>

# Permission testing
kubectl auth can-i <verb> <resource> --as=<user> -n <ns>
kubectl auth can-i --list --as=<user> -n <ns>

# ServiceAccount
kubectl create serviceaccount <name> -n <ns>

# etcd
ETCDCTL_API=3 etcdctl snapshot save /path/backup.db --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key
ETCDCTL_API=3 etcdctl snapshot status /path/backup.db --write-out=table
```

B.2 Workloads & Scheduling

```
# Deployments
kubectl create deployment <name> --image=<img> --replicas=<n>
kubectl scale deployment <name> --replicas=<n>
kubectl set image deployment/<name> <container>=<new-image>
kubectl rollout status deployment/<name>
kubectl rollout undo deployment/<name>
kubectl rollout history deployment/<name>

# Pods (imperative)
kubectl run <name> --image=<img>
kubectl run <name> --image=<img> --env="KEY=VALUE" --labels="k=v"
kubectl run <name> --image=<img> --requests='cpu=100m,memory=128Mi'
kubectl run <name> --image=<img> --limits='cpu=500m,memory=512Mi'

# Jobs and CronJobs
kubectl create job <name> --image=<img> -- <command>
kubectl create cronjob <name> --image=<img> --schedule="* * * * *" -- <command>

# ConfigMaps and Secrets
kubectl create configmap <name> --from-literal=key=val --from-file=<file>
kubectl create secret generic <name> --from-literal=key=val

# Node scheduling
kubectl label node <node> <key>=<val>
kubectl cordon <node>
kubectl drain <node> --ignore-daemonsets --delete-emptydir-data
kubectl uncordon <node>
kubectl taint nodes <node> <key>=<val>:<effect>
kubectl taint nodes <node> <key>=<val>:<effect>-   # Remove taint
```

B.3 Services & Networking

```
# Services
kubectl expose deployment <name> --port=<port> --target-port=<tp> --type=<type>
kubectl expose pod <name> --port=<port>

# Check endpoints
kubectl get endpoints <svc>

# DNS test
kubectl run dns --image=busybox --rm -it --restart=Never -- nslookup <service>
kubectl run dns --image=busybox --rm -it --restart=Never -- nslookup <svc>.<ns>.svc.cluster.local

# NetworkPolicy: list all
kubectl get networkpolicies -A
```

B.4 Storage

```
# PV and PVC
kubectl get pv
kubectl get pvc -A
kubectl describe pvc <name>

# Verify volume is mounted
kubectl exec <pod> -- df -h <mountpath>
kubectl exec <pod> -- mount | grep <mountpath>
```

B.5 Troubleshooting

```
# Pod troubleshooting
kubectl describe pod <pod>
kubectl logs <pod> --previous
kubectl logs <pod> -c <container>
kubectl get events -n <ns> --sort-by=.lastTimestamp

# Node troubleshooting (on node)
systemctl status kubelet
systemctl restart kubelet
journalctl -u kubelet -n 100 --no-pager

# Check all non-running pods cluster-wide
kubectl get pods -A | grep -v Running | grep -v Completed

# Get pod on a specific node
kubectl get pods -A -o wide | grep <node-name>

# Resource usage
kubectl top nodes
kubectl top pods -A --sort-by=memory
```

Appendix C — Kustomize Pattern Reference

C.1 Complete kustomization.yaml Field Reference

```yaml
# kustomization.yaml — All common fields
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization

# Resources: list of YAML files or remote references to include
resources:
  - deployment.yaml
  - service.yaml
  - ../../base                                # Relative path to another kustomization
  - github.com/org/repo//path?ref=v1.0.0      # Remote GitHub reference

# Components: reusable kustomization fragments
components:
  - ../../components/monitoring
  - ../../components/tls

# Namespace: override namespace on all resources
namespace: production

# Name transformations
namePrefix: prod-
nameSuffix: -v2

# Common labels (also modifies selectors)
commonLabels:
  environment: production
  managed-by: kustomize

# Common annotations (does NOT modify selectors — safer for updates)
commonAnnotations:
  team: platform-engineering

# Image transformations
images:
  - name: my-app
    newName: registry.company.com/my-app
    newTag: v1.5.2
  - name: sidecar
    newTag: stable
  - name: my-app
    digest: sha256:abcdef...   # Pin to digest (immutable)

# Patches (strategic merge or JSON 6902)
patches:
  - path: patch-replicas.yaml                 # File-based patch
  - path: patch-env.yaml
    target:
      kind: Deployment
      name: my-app
  - target:                                    # Inline JSON 6902 patch
      kind: Deployment
      name: my-app
    patch: |-
      - op: replace
        path: /spec/replicas
        value: 5

# ConfigMap generator
configMapGenerator:
  - name: app-config
    files:
      - config.properties
    literals:
      - LOG_LEVEL=info
```

```
    options:
      disableNameSuffixHash: false

# Secret generator
secretGenerator:
  - name: app-secrets
    type: Opaque
    literals:
      - username=admin
    envs:
      - .env.production
    options:
      disableNameSuffixHash: true
```

C.2 Common Kustomize Command Patterns

```
# Render and review (always do this before applying)
kubectl kustomize ./overlays/production | less

# Diff against live cluster
kubectl diff -k ./overlays/production

# Apply overlay
kubectl apply -k ./overlays/production

# Apply with server-side dry run (validates against API schema)
kubectl apply -k ./overlays/production --dry-run=server

# Delete all resources in a kustomization
kubectl delete -k ./overlays/production

# Save rendered output for CI/CD artifact
kubectl kustomize ./overlays/production > rendered-$(date +%Y%m%d-%H%M%S).yaml

# Apply rendered output (useful when you want to keep the artifact)
kubectl kustomize ./overlays/production | kubectl apply -f -

# Watch rollout after apply
kubectl apply -k ./overlays/production && \
  kubectl rollout status deployment/my-app -n production --timeout=300s
```

Appendix D — CKA Gap-Fill Quick Reference

This appendix provides a condensed command reference specifically for the nine topic areas that were identified as missing or thin in the original document.

D.1 Cluster Installation Quick Reference

```
# Disable swap
swapoff -a && sed -i '/ swap / s/^/#/' /etc/fstab

# Load kernel modules
modprobe overlay && modprobe br_netfilter

# Install containerd + set SystemdCgroup=true
apt install containerd && containerd config default > /etc/containerd/config.toml
sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml

# Install kubeadm/kubelet/kubectl
apt install -y kubelet kubeadm kubectl && apt-mark hold kubelet kubeadm kubectl

# Init control plane
kubeadm init --pod-network-cidr=192.168.0.0/16 --apiserver-advertise-address=<IP>

# Configure kubectl
mkdir -p ~/.kube && cp /etc/kubernetes/admin.conf ~/.kube/config

# Install CNI (Calico)
kubectl apply -f https://docs.projectcalico.org/manifests/calico.yaml

# Join worker
kubeadm join <CP_IP>:6443 --token <token> --discovery-token-ca-cert-hash sha256:<hash>

# Generate new join command (if token expired)
kubeadm token create --print-join-command

# Check certificates
kubeadm certs check-expiration && kubeadm certs renew all
```

D.2 HA Control Plane Quick Reference

```
# Init HA control plane
kubeadm init \
  --control-plane-endpoint "LB_DNS:6443" \
  --upload-certs \
  --pod-network-cidr=192.168.0.0/16

# Join additional control plane nodes
kubeadm join LB_DNS:6443 --token <t> \
  --discovery-token-ca-cert-hash sha256:<h> \
  --control-plane --certificate-key <k>

# Check etcd member list
kubectl exec -it etcd-cp1 -n kube-system -- sh -c \
  'ETCDCTL_API=3 etcdctl member list \
   --endpoints=https://127.0.0.1:2379 \
   --cacert=/etc/kubernetes/pki/etcd/ca.crt \
   --cert=/etc/kubernetes/pki/etcd/server.crt \
   --key=/etc/kubernetes/pki/etcd/server.key'
```

D.3 CRI/CNI/CSI Quick Reference

```
# CRI — check runtime version on node
kubectl get node <n> -o jsonpath='{.status.nodeInfo.containerRuntimeVersion}'

# crictl — interact with runtime directly
sudo crictl ps                      # Running containers
sudo crictl pods                    # All pods
sudo crictl logs <container-id>     # Container logs
sudo crictl pull nginx:1.25         # Pull image

# CNI — check plugin pods
kubectl get pods -n kube-system | grep -E "calico|flannel|cilium"

# CSI — check drivers
kubectl get csidriver
kubectl get csistoragecapacities -A
kubectl get csinode
```

D.4 CRD and Operator Quick Reference

```
# List CRDs
kubectl get crd
kubectl api-resources --api-group=mycompany.com

# Install CRD
kubectl apply -f crd.yaml

# Create CR instance
kubectl apply -f custom-resource.yaml

# List instances
kubectl get <plural-name> -A
kubectl describe <kind> <name>

# Delete CRD (DELETES ALL INSTANCES)
kubectl delete crd <name>
```

D.5 Gateway API Quick Reference

```
# Install Gateway API CRDs
kubectl apply -f https://github.com/kubernetes-sigs/gateway-api/releases/download/v1.1.0/standard-
install.yaml

# Check resources
kubectl get gatewayclass
kubectl get gateway -A
kubectl get httproute -A

# Describe for troubleshooting
kubectl describe gateway my-gateway -n production
kubectl describe httproute my-route -n production
```

D.6 HPA Quick Reference

```
# Create HPA imperatively
kubectl autoscale deployment <name> --min=2 --max=10 --cpu-percent=50

# View HPA
kubectl get hpa
kubectl describe hpa <name>

# Delete HPA
kubectl delete hpa <name>

# Generate load (test HPA)
kubectl run load --image=busybox --rm -it --restart=Never -- \
  sh -c "while true; do wget -q -O- http://<svc>/; done"

# Check metrics
kubectl top pod --sort-by=cpu
```

D.7 Probes and PDB Quick Reference

```
# Debug probe failures
kubectl describe pod <n> | grep -A 10 "Liveness\|Readiness\|Startup"
kubectl get pod <n> -o jsonpath='{.status.containerStatuses[0].restartCount}'
kubectl logs <n> --previous

# PDB commands
kubectl create -f pdb.yaml
kubectl get pdb
kubectl describe pdb <name>
kubectl delete pdb <name>
```

D.8 Storage Quick Reference

```
# StorageClass
kubectl get sc
kubectl describe sc <name>

# PV and PVC
kubectl get pv
kubectl get pvc -A
kubectl describe pvc <name>

# Expand PVC
kubectl patch pvc <name> -p '{"spec":{"resources":{"requests":{"storage":"50Gi"}}}}'

# Release a stuck PV (clear claimRef)
kubectl patch pv <name> -p '{"spec":{"claimRef": null}}'

# Check reclaim policy
kubectl get pv -o custom-columns=NAME:.metadata.name,POLICY:.spec.persistentVolumeReclaimPolicy
```

D.9 ConfigMap and Secret Quick Reference

```
# ConfigMap
kubectl create configmap <n> --from-literal=key=val --from-file=file.conf
kubectl get configmap <n> -o yaml
kubectl edit configmap <n>
kubectl delete configmap <n>

# Secret
kubectl create secret generic <n> --from-literal=user=admin --from-literal=pass=secret
kubectl create secret tls <n> --cert=tls.crt --key=tls.key
kubectl get secret <n> -o yaml
kubectl get secret <n> -o jsonpath='{.data.password}' | base64 --decode
kubectl delete secret <n>
```

References and Further Reading

- **Kubernetes Official Docs:** https://kubernetes.io/docs/
- **CKA Curriculum (CNCF):** https://github.com/cncf/curriculum
- **CKA Program Changes (Feb 2025):** https://training.linuxfoundation.org/certified-kubernetes-administrator-cka-program-changes/
- **Linux Foundation CKA Exam Page:** https://training.linuxfoundation.org/certification/certified-kubernetes-administrator-cka/
- **CoreDNS Official Docs:** https://coredns.io/manual/toc/
- **Kustomize Official Docs:** https://kubectl.docs.kubernetes.io/references/kustomize/
- **Kustomize GitHub:** https://github.com/kubernetes-sigs/kustomize
- **netshoot Image (GitHub):** https://github.com/nicolaka/netshoot
- **kubectl Cheat Sheet (Official):** https://kubernetes.io/docs/reference/kubectl/cheatsheet/
- **Kubernetes DNS Specification:** https://github.com/kubernetes/dns/blob/master/docs/specification.md
- **Ephemeral Containers Guide:** https://kubernetes.io/docs/concepts/workloads/pods/ephemeral-containers/
- **etcdctl Documentation:** https://etcd.io/docs/v3.5/op-guide/maintenance/
- **kubeadm Upgrade Guide:** https://kubernetes.io/docs/tasks/administer-cluster/kubeadm/kubeadm-upgrade/
- **RBAC Guide:** https://kubernetes.io/docs/reference/access-authn-authz/rbac/
- **NetworkPolicy Guide:** https://kubernetes.io/docs/concepts/services-networking/network-policies/
- **Gateway API Docs:** https://gateway-api.sigs.k8s.io/
- **HPA Docs:** https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/
- **StorageClass Docs:** https://kubernetes.io/docs/concepts/storage/storage-classes/
- **CRD Docs:** https://kubernetes.io/docs/tasks/extend-kubernetes/custom-resources/custom-resource-definitions/
- **kubeadm HA Guide:** https://kubernetes.io/docs/setup/production-environment/tools/kubeadm/high-availability/
- **Container Runtime Interface:** https://kubernetes.io/docs/concepts/architecture/cri/
- **CNI Specification:** https://github.com/containernetworking/cni

---

This document is a comprehensive technical reference guide covering the complete CKA 2026 exam curriculum, Kubernetes container interaction, ephemeral test containers, CoreDNS diagnostics, container removal and modification, kubectl Kustomize, cluster installation from scratch, HA control plane setup, extension interfaces (CNI/CSI/CRI), CRDs and Operators, Gateway API, HPA autoscaling, self-healing probes and PodDisruptionBudgets, dynamic storage provisioning with StorageClasses, and a complete ConfigMap and Secrets reference.